Министерство просвящения Республики Башкортостан
Государственное автономное профессиональное образовательное учреждение
Уфимский колледж статистики, информатики и
вычислительной техники

УТВЕРЖДАЮ
Заместитель директора
по учебной работе
_____З.З. Курмашева
«___» _____ 2026 г.

РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ ДЛЯ УЧЁТА ПОСЕЩЕНИЙ В ВЕТЕРИНАРНОЙ КЛИНИКЕ

Пояснительная записка к курсовому проекту

Руководитель
_____ О.В. Фатхулова
«___» _____ 2026 г.

Студент гр. 23П-4
_____Х.А.Альбертович
«___» _____ 2026 г.

2026

Министерство образования и науки Республики Башкортостан

Государственное автономное профессиональное образовательное учреждение

Уфимский колледж статистики, информатики и вычислительной техники

УТВЕРЖДАЮ

Заместитель директора

по учебной работе

_____ З.З.

Курмашева

«__» _____ 2026 г.

ЗАДАНИЕ

на курсовой проект студенту дневного отделения, группы 23П-4, специальности 09.02.07 Информационные системы и программирование.

Фамилия, имя, отчество: Хабибуллин Артём Альбертович

Тема: «Разработка информационной системы для учёта посещений в ветеринарной клинике»

Текст задания:

при выполнении курсового проекта должны быть решены следующие задачи:

а) спроектирована база данных;

б) разработана архитектура и структура программы;

в) реализована регистрация и авторизация пользователей с разделением ролей (клиент, врач) на основе JWT-токенов;

г) для клиента реализованы функции: запись на приём (выбор питомца, врача, услуги и времени), просмотр истории визитов с экспортом протокола в PDF, ведение карточек питомцев, оформление заказов в интернет-магазине товаров;

д) для врача реализованы функции: просмотр расписания, подтверждение/отклонение записей на приём, завершение приёма с фиксацией веса питомца, диагнозов, лечения и медикаментов, просмотр картотеки пациентов;

е) реализован справочник диагнозов и дашборд со статистикой и графиками динамики веса питомцев.

В результате выполнения курсового проекта должны быть представлены:

а) пояснительная записка, состоящая из следующих разделов:

Введение

1 Постановка задачи

2 Экспериментальный раздел

Заключение

Список использованных источников

б) электронный носитель, содержащий разработанный программный продукт, пояснительную записку;

в) графическая часть - 6 листов формата А4;

г) презентация дипломного проекта в электронном виде.

Список рекомендуемых источников:

1 Хорстманн К. Принципы программирования на Go : учебник / Хорстманн К. — Москва : ДМК Пресс, 2022.

2 Дронов В.А. JavaScript и Vue.js : разработка веб-приложений : учебник для студ. учреждений сред. проф. образования. — Москва : Издательский центр «Академия», 2023.

3 Кумскова И.А. Базы данных: учебник / Кумскова И.А. — Москва : КноРус, 2021. — 400 с. — ISBN 978-5-406-08303-1. — URL: https://book.ru/book/940108. — Текст: электронный.

4 GORM Documentation: сайт, 2024 — URL: https://gorm.io/docs/ — Режим доступа: свободный. — Текст: электронный.

5 Vue.js Documentation: сайт, 2024 — URL: https://vuejs.org/guide/ — Режим доступа: свободный. — Текст: электронный

Задание к выполнению получил «30» марта 2026г.

Студент_____Хабибуллин Артём Альбертович

Срок окончания «27» июня 2026г.

Руководитель_____О.В. Фатхулова

Задание рассмотрено на заседании цикловой комиссии информатики

«25» марта 2026г. протокол № 5

Председатель цикловой комиссии_____О.В. Фатхулова

АННОТАЦИЯ

Пояснительная записка к курсовому проекту содержит постановку и программу решения задачи «Разработка информационной системы для учёта посещений в ветеринарной клинике».

Серверное приложение (REST API) написано на языке программирования Go с использованием фреймворка Gin и библиотеки GORM в среде программирования Visual Studio Code, с использованием системы управления базами данных MySQL, и отлажено на данных контрольного примера.

Веб-приложение для клиентов и врачей клиники написано на языке программирования TypeScript с использованием фреймворка Vue 3 (Composition API) в среде программирования Visual Studio Code, предназначено для работы в браузере на любой платформе, и отлажено на данных контрольного примера.

СОДЕРЖАНИЕ

	лист
Введение	7
1 Постановка задачи	8
1.1 Описание предметной области	8
1.2 Проектирование бизнес-процессов предметной области	13
1.3 Описание входной информации	15
1.4 Описание выходной информации	16
1.5 Общие требования к программному продукту	18
1.6 Описание структуры базы данных	19
2 Экспериментальный раздел	25
2.1 Описание программы	25
2.2 Протокол тестирования программного продукта	29
2.3 Руководство пользователя	44
Заключение	58
Приложение А. Контрольный пример	59
Приложение Б. Исходный код	61
Приложение В. Результат работы	145
Список использованныъ источников	153

ВВЕДЕНИЕ

В условиях стремительного развития цифровых технологий и роста спроса на качественные ветеринарные услуги особую актуальность приобретает автоматизация процессов учёта и обслуживания клиентов ветеринарных клиник. Современная ветеринарная практика требует оперативного учёта питомцев, истории их посещений, диагнозов и назначенного лечения, а также удобного взаимодействия между клиентом, врачом и администрацией клиники.

Актуальность темы обусловлена тем, что традиционные методы ведения учёта посещений — бумажные карты пациентов, журналы записи на приём, устная передача информации между сотрудниками — приводят к потере данных, затрудняют поиск истории болезни питомца и снижают скорость обслуживания клиентов. Внедрение информационной системы позволяет устранить эти недостатки, обеспечив централизованное хранение медицинских данных, автоматизацию записи на приём и оперативный доступ к истории визитов.

Цель проекта — разработка информационной системы для учёта посещений в ветеринарной клинике, обеспечивающей автоматизацию записи на приём, ведение электронных медицинских карт питомцев и удобное взаимодействие между клиентами и врачами клиники.

Задачи проекта:

- спроектировать структуру базы данных, обеспечивающую хранение информации о клиентах, питомцах, врачах, услугах, записях на приём и медицинских протоколах;
- разработать архитектуру серверного приложения (REST API) с разделением на слои моделей, репозиториев, бизнес-логики и обработчиков запросов;
- реализовать регистрацию и авторизацию пользователей с разделением ролей (клиент, врач) на основе JWT-токенов;
- реализовать для клиента функции записи на приём с выбором питомца,

врача, услуги и времени, просмотра истории визитов с экспортом медицинского протокола в PDF, ведения карточек питомцев и оформления заказов в интернет-магазине товаров;

- реализовать для врача функции просмотра расписания, подтверждения и отклонения записей на приём, завершения приёма с фиксацией веса питомца, диагнозов, лечения и медикаментов, а также просмотра картотеки пациентов;
- реализовать справочник диагнозов с возможностью выбора нескольких диагнозов при завершении приёма;
- реализовать дашборд со статистикой посещений и графиками динамики веса питомцев;
- выполнить тестирование разработанной системы на корректность работы основных сценариев.

Область применения результатов работы охватывает частные ветеринарные клиники, заинтересованные в автоматизации внутренних процессов учёта посещений. Внедрение разработанной системы позволит сократить время на регистрацию и поиск медицинской информации, повысить точность ведения истории болезни питомцев и улучшить качество обслуживания клиентов клиники.

1.1 Описание предметной области

Пусть требуется разработать информационную систему для учёта посещений в ветеринарной клинике. Система должна обеспечивать автоматизацию следующих задач: регистрация клиентов через веб-приложение, ведение карточек питомцев, запись на приём к врачу, ведение медицинских протоколов посещений с диагнозами и назначенным лечением, продажа товаров для животных через интернет-магазин.

Для регистрации в системе клиент самостоятельно создаёт учётную запись через форму регистрации, указывая контактные данные. После регистрации клиент может добавить одного или нескольких питомцев и записаться на приём к врачу.

Пользователь системы с ролью «клиент» имеет следующие параметры:

- уникальный номер;
- адрес электронной почты;
- пароль;
- фамилия;
- имя;
- номер телефона;
- роль;
- список питомцев.

Пользователь системы с ролью «врач» имеет следующие параметры:

- уникальный номер;
- адрес электронной почты;
- пароль;
- фамилия;
- имя;
- номер телефона;
- роль;
- специализация.

Питомец имеет следующие параметры:

- уникальный номер;
- владелец (клиент);
- имя;
- вид животного;
- порода;
- дата рождения;
- вес;
- фотография.

Для добавления новой записи на приём клиент использует функционал веб-приложения, выбирая питомца, врача, услугу и удобное время приёма. Запись на приём имеет следующие параметры:

- уникальный номер;
- клиент;
- питомец;
- врач;
- услуга;
- дата и время приёма;
- статус записи (ожидает, принята, отклонена, завершена);
- комментарий клиента.

После завершения приёма врач заполняет медицинский протокол, который имеет следующие параметры:

- уникальный номер;
- запись на приём, к которой относится протокол;
- вес питомца на момент приёма;
- список диагнозов;
- назначенное лечение;
- назначенные медикаменты.

Диагноз имеет следующие параметры:

- уникальный номер;
- название диагноза.

Одной записи на приём может соответствовать несколько диагнозов, поскольку у питомца при одном визите может быть выявлено более одного заболевания. Аналогично, один и тот же диагноз может быть поставлен в рамках разных приёмов разным питомцам, поэтому между записью на приём и диагнозом установлена связь типа «многие ко многим».

Услуга, оказываемая клиникой, имеет следующие параметры:

- уникальный номер;
- название услуги;
- стоимость.

Товар интернет-магазина имеет следующие параметры:

- уникальный номер;
- название;
- описание;
- стоимость;
- категория;
- количество на складе.

Для оформления заказа клиент добавляет товары в корзину. Элемент корзины имеет следующие параметры:

- уникальный номер;
- клиент;
- товар;
- количество.

При оформлении заказа создаётся заказ, имеющий следующие параметры:

- уникальный номер;
- клиент;
- список позиций заказа (товар, количество, цена);
- итоговая сумма;
- статус заказа (оплачен, подтверждён, доставлен).

Для корректной работы системы предусмотрены следующие ограничения:

- каждый пользователь, питомец, запись на приём, услуга, диагноз, товар и заказ должны иметь уникальные идентификаторы;
- адрес электронной почты и номер телефона пользователя должны быть уникальными в системе;
- запись на приём не может быть создана на время, занятое другим приёмом того же врача;
- медицинский протокол может быть создан только после перевода записи на приём в статус «завершена».

Система будет работать для двух групп пользователей:

- клиент;
- врач.

Для клиента должен быть доступен следующий функционал:

- регистрация и авторизация в системе;
- ведение карточек питомцев (добавление, редактирование, удаление);
- запись на приём с выбором питомца, врача, услуги и времени;
- просмотр занятых слотов времени врача;
- просмотр истории посещений с медицинскими протоколами и их экспорт в PDF;
- просмотр статистики по питомцам и динамики веса;
- просмотр каталога товаров, управление корзиной, оформление заказов и просмотр истории заказов.

Для врача должен быть доступен следующий функционал:

- авторизация в системе;
- просмотр собственного расписания приёмов;
- подтверждение и отклонение записей на приём;
- завершение приёма с фиксацией веса питомца, диагнозов, лечения и медикаментов;

– поиск пациентов (клиентов) по имени или номеру телефона;
– просмотр медицинской истории клиента по всем его питомцам.

1.2 Проектирование бизнес-процессов предметной области

Диаграмма прецедентов (англ. Use Case Diagram) в языке UML представляет собой графическое отображение взаимодействия пользователей (актёров) с системой через различные сценарии использования (прецеденты). Она позволяет на концептуальном уровне отразить основные функции системы и то, как с ними взаимодействуют участники.

В разрабатываемой системе выделены два актёра: «Клиент» и «Врач». Клиент взаимодействует с системой через прецеденты «Регистрация», «Авторизация», «Управление питомцами», «Запись на приём», «Просмотр истории посещений», «Покупка товаров». Врач взаимодействует с системой через прецеденты «Авторизация», «Просмотр расписания», «Подтверждение/отклонение записи», «Завершение приёма», «Поиск пациентов», «Просмотр медицинской истории клиента».

Диаграмма прецедентов представлена на рисунке 1.2.1.

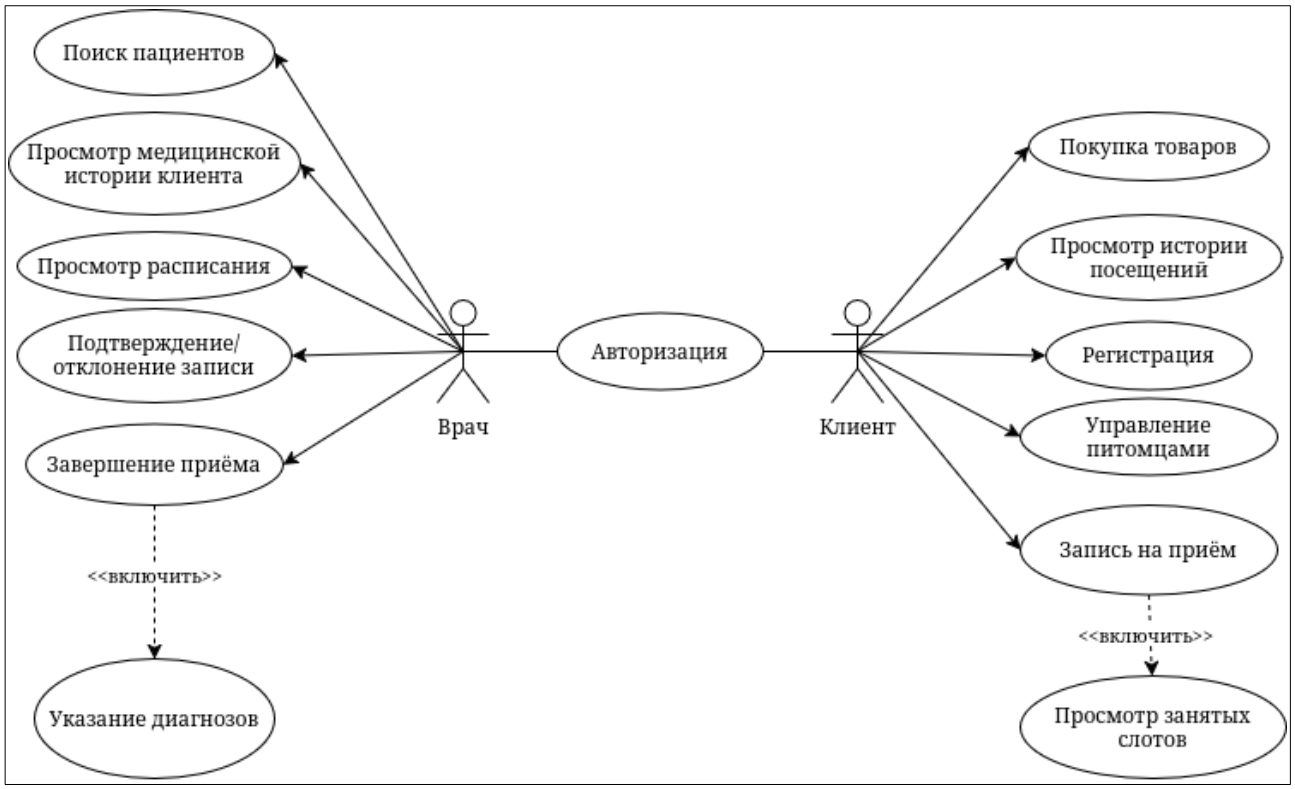


Рисунок 1.2.1 – Диаграмма прецедентов

Диаграмма развёртывания (англ. Deployment Diagram) отражает физическую архитектуру системы — распределение программных компонентов по аппаратным узлам и их взаимодействие. В разрабатываемой системе клиентское веб-приложение (Vue 3) взаимодействует с сервером через HTTP-запросы к REST API (Go, Gin), которое в свою очередь обращается к серверу базы данных MySQL посредством библиотеки GORM.

Диаграмма развёртывания представлена на рисунке 1.2.2.

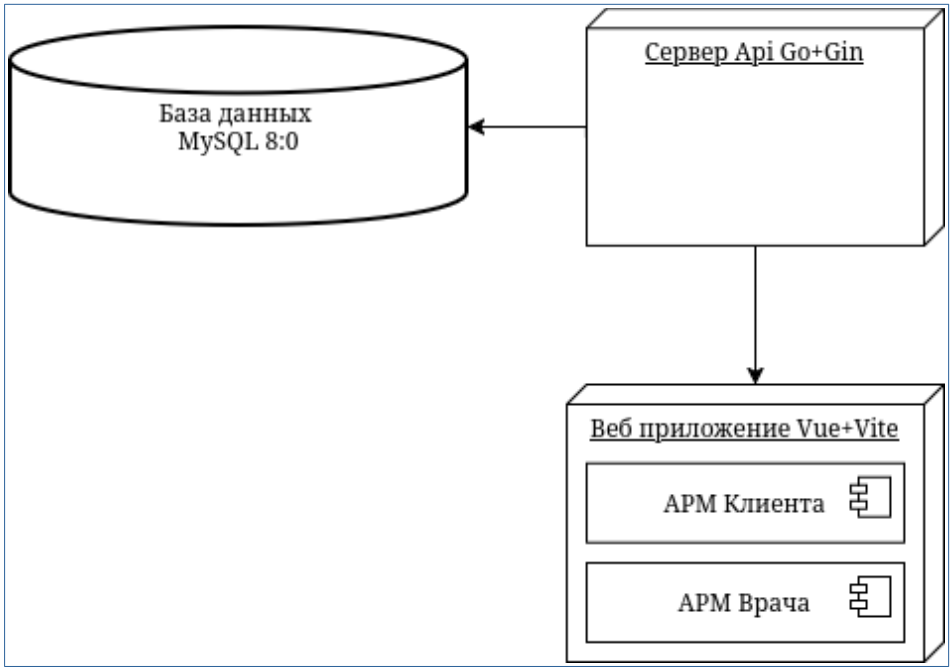


Рисунок 1.2.2 – Диаграмма развёртывания

Диаграмма последовательности (англ. Sequence Diagram) отражает порядок взаимодействия объектов системы во времени для конкретного сценария. На рисунке 1.2.3 представлена диаграмма последовательности для сценария «Запись на приём и завершение визита врачом»: клиент отправляет запрос на создание записи, сервер сохраняет её в статусе «ожидает», врач подтверждает запись, после приёма врач заполняет медицинский протокол с диагнозами, сервер сохраняет протокол и переводит запись в статус «завершена».

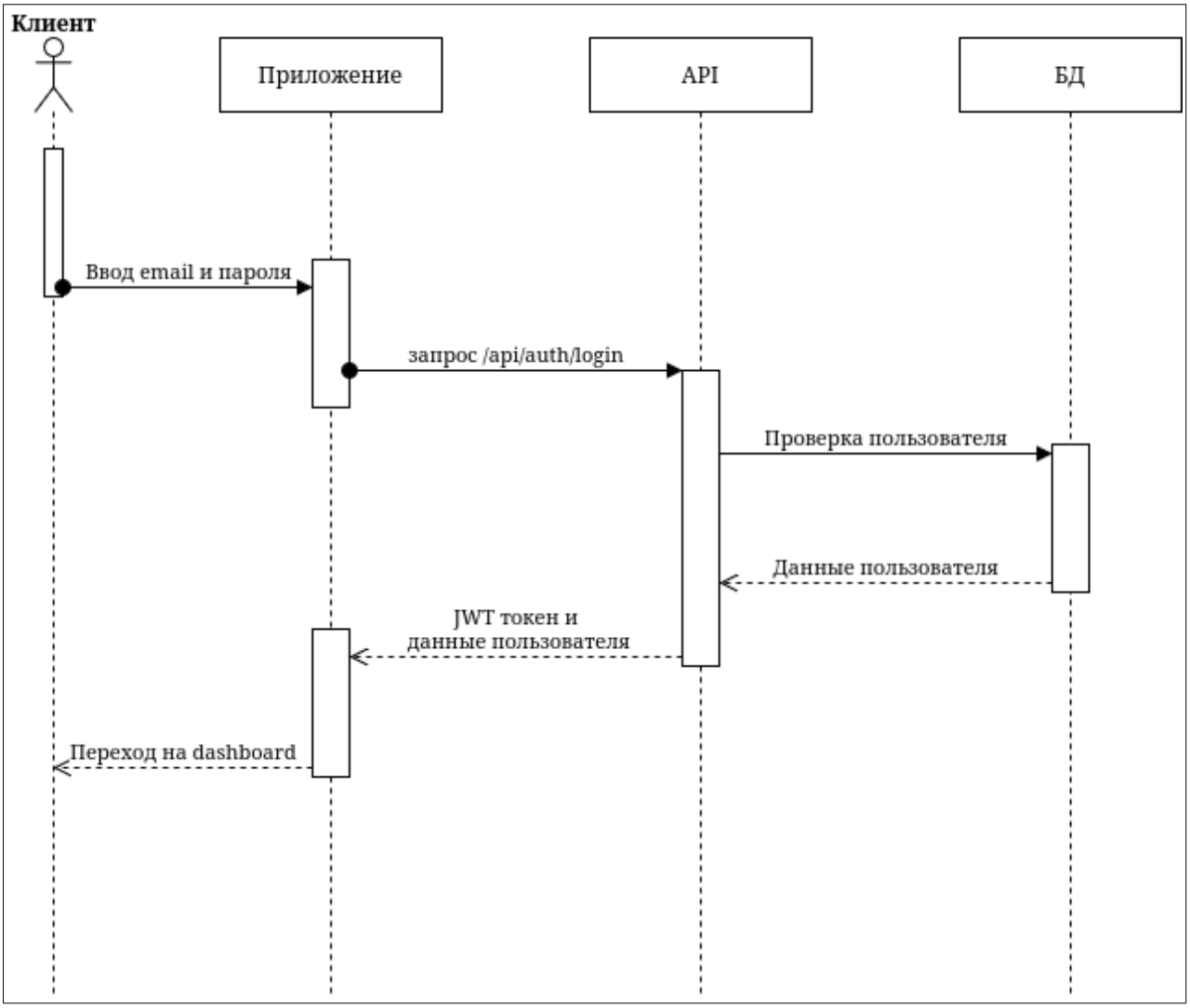


Рисунок 1.2.3 – Диаграмма последовательности

1.3 Описание входной информации

Входной информацией при регистрации клиента является:

- фамилия;
- имя;
- адрес электронной почты;
- номер телефона;
- пароль.

Входной информацией при добавлении питомца является:

- имя питомца;

– вид животного;
– порода;
– дата рождения;
– вес;
– фотография.

Входной информацией для записи на приём является:

– питомец, записываемый на приём;
– врач, к которому записывается клиент;
– услуга;
– дата и время приёма;
– комментарий клиента (необязательно).

Входной информацией для завершения приёма врачом является:

– вес питомца на момент приёма;
– список идентификаторов диагнозов;
– описание назначенного лечения;
– список назначенных медикаментов.

Входной информацией для оформления заказа в интернет-магазине является:

– список товаров и их количество, добавленных в корзину.

1.4 Описание выходной информации

Выходной информацией системы является медицинский протокол приёма в формате PDF, данные статистики и графики динамики веса питомца на дашборде клиента, а также списки записей и истории посещений, отображаемые в табличном виде в веб-приложении.

Шаблон медицинского протокола (заключения), формируемого при экспорте в PDF, представлен на рисунке 1.4.1.



ВЕТЕРИНАРНЫЙ ЦЕНТР "ВетКлиника"

Лицензия № ВЕТ-77-01-000842 | petcare-clinic.ru

г. Москва, ул. Ветеринарная, 12 | +7 (495) 000-00-00

МЕДИЦИНСКОЕ ЗАКЛЮЧЕНИЕ №11

Пациент: Барсик

Вид животного: Домашнее животное

Дата приема: 19.06.2026

Вес пациента: 3.5 кг

Лечащий врач: Смирнов Иван

1. АНАМНЕЗ И ДИАГНОЗ

Клинически здоров

2. НАЗНАЧЕННОЕ ЛЕЧЕНИЕ И РЕКОМЕНДАЦИИ

Чаще мыть

3. МЕДИКАМЕНТОЗНЫЕ НАЗНАЧЕНИЯ

Лекарство 1мл 5д

ДЛЯ ДОКУМЕНТОВ

Подпись врача: _____ / Смирнов Иван /

Рисунок 1.4.1 – Шаблон медицинского заключения

Пример отображения истории посещений клиента представлен на рисунке 1.4.2.

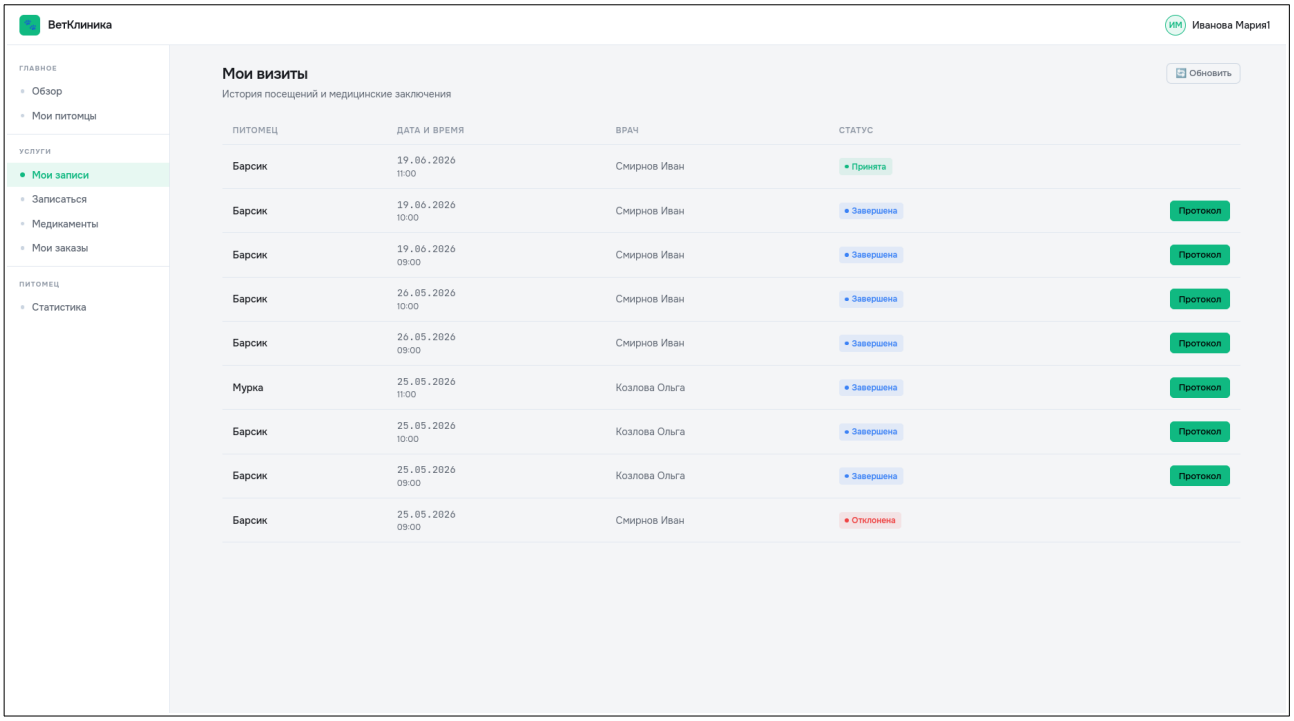


Рисунок 1.4.2 – Пример вывода истории посещений

1.5 Общие требования к программному продукту

Для работы с веб-приложением пользователи должны обладать базовыми навыками работы с ПК и веб-браузером.

Надёжность системы должна быть обеспечена выполнением следующих пунктов:

- разграничение прав доступа пользователей на основе ролей (клиент, врач) с использованием JWT-токенов;
- хранение паролей пользователей в зашифрованном виде;
- возможность создания и восстановления резервной копии базы данных.

Отказы программы вследствие некорректных действий пользователя при взаимодействии с системой недопустимы.

Минимальные системные требования для компьютера, на котором планируется использование веб-приложения:

- объём ОЗУ не менее 4 Гб;

- доступ в интернет не менее 1 Мб/с;
- процессор Intel Pentium(R) 2.16 GHz и выше;
- операционная система Windows 10 или выше;
- не менее 500 Мб свободного места на жёстком диске;
- современный веб-браузер (Google Chrome, Mozilla Firefox, Microsoft Edge) с поддержкой JavaScript.

Необходимые виды периферийных устройств: оптический манипулятор типа «мышь», клавиатура, монитор.

1.6 Описание структуры базы данных

При проектировании базы данных использовалась СУБД MySQL 8.0. Управление схемой базы данных осуществляется средствами ORM-библиотеки GORM на основе моделей данных, описанных в программном коде. Описание структуры базы данных представлено в таблицах 1.6.1 – 1.6.11.

Таблица 1.6.1 – users (Список пользователей)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор пользователя	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Электронная почта	email	VARCHAR(191)	обязательное, уникальное поле
Пароль	password	VARCHAR(255)	обязательное поле, хранится в хешированном виде
Имя	first_name	VARCHAR(255)	обязательное поле
Фамилия	last_name	VARCHAR(255)	обязательное поле
Номер телефона	phone	VARCHAR(191)	обязательное,

Содержание поля	Имя поля	Тип, длина	Примечания
			уникальное поле
Роль	role	ENUM('client','doctor')	обязательное поле
Специализация	specialization	VARCHAR(255)	необязательное поле, заполняется для врача

Таблица 1.6.2 – pets (Список питомцев)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор питомца	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Идентификатор владельца	owner_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к users)
Имя питомца	name	VARCHAR(255)	обязательное поле
Вид животного	species	VARCHAR(255)	обязательное поле
Порода	breed	VARCHAR(255)	необязательное поле
Дата рождения	birth_date	DATETIME	обязательное поле
Вес	weight	DOUBLE	обязательное поле
Фотография	avatar	VARCHAR(255)	необязательное поле, путь к файлу

Таблица 1.6.3 – services (Список услуг)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор услуги	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления	updated_at	DATETIME	обязательное поле

Содержание поля	Имя поля	Тип, длина	Примечания
записи			
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Название услуги	name	VARCHAR(255)	обязательное поле
Стоимость	price	BIGINT UNSIGNED	обязательное поле

Таблица 1.6.4 – appointments (Список записей на приём)

Содержание поля	Имя поля	Тип, длина	Примечания
1	2	3	4
Идентификатор записи	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Идентификатор клиента	client_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к users)

Продолжение таблицы 1.6.4

1	2	3	4
Идентификатор питомца	pet_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к pets)
Идентификатор врача	doctor_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к users)
Идентификатор услуги	service_id	BIGINT UNSIGNED	необязательное поле, внешний ключ (к services)
Дата и время приёма	scheduled_at	DATETIME	обязательное поле
Статус записи	status	VARCHAR(20)	обязательное поле, по умолчанию «waiting»
Комментарий клиента	comment	VARCHAR(255)	необязательное поле

Таблица 1.6.5 – diagnoses (Справочник диагнозов)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор диагноза	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Название диагноза	name	VARCHAR(255)	обязательное поле

Таблица 1.6.6 – appointment_diagnoses (Связь записей на приём с диагнозами)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор записи на приём	appointment_id	BIGINT UNSIGNED	составной первичный ключ, внешний ключ (к appointments)
Идентификатор диагноза	diagnosis_id	BIGINT UNSIGNED	составной первичный ключ, внешний ключ (к diagnoses)

Таблица реализует связь типа «многие ко многим» между записями на приём и диагнозами: одной записи на приём может соответствовать несколько диагнозов, а один диагноз может быть поставлен в рамках разных приёмов.

Таблица 1.6.7 – medical_protocols (Список медицинских протоколов)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор протокола	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Идентификатор записи на приём	appointment_id	BIGINT UNSIGNED	обязательное, уникальное поле, внешний ключ (к appointments)
Вес питомца на приёме	weight_at_visit	DOUBLE	обязательное поле

Содержание поля	Имя поля	Тип, длина	Примечания
Назначенное лечение	treatment	VARCHAR(255)	необязательное поле
Назначенные медикаменты	medications	VARCHAR(255)	необязательное поле

Таблица 1.6.8 – products (Список товаров)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор товара	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Название товара	name	VARCHAR(255)	обязательное поле
Описание	description	VARCHAR(255)	необязательное поле
Стоимость	price	INT	обязательное поле
Категория	category	VARCHAR(255)	необязательное поле
Количество на складе	stock	INT	обязательное поле

Таблица 1.6.9 – cart_items (Список товаров в корзине)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор позиции	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Идентификатор пользователя	user_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к users)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор товара	product_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к products)
Количество	quantity	INT UNSIGNED	обязательное поле

Таблица 1.6.10 – orders (Список заказов)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор заказа	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Идентификатор пользователя	user_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к users)
Итоговая сумма	total_amount	INT UNSIGNED	обязательное поле
Статус заказа	status	ENUM('paid','confirmed','delivered')	обязательное поле

Таблица 1.6.11 – order_items (Список позиций заказа)

Содержание поля	Имя поля	Тип, длина	Примечания
Идентификатор позиции	id	BIGINT UNSIGNED	первичный, суррогатный ключ, авто инкремент
Дата создания записи	created_at	DATETIME	обязательное поле
Дата обновления записи	updated_at	DATETIME	обязательное поле
Дата удаления записи	deleted_at	DATETIME	необязательное поле, мягкое удаление
Идентификатор заказа	order_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к orders)
Идентификатор товара	product_id	BIGINT UNSIGNED	обязательное поле, внешний ключ (к products)
Количество	quantity	INT UNSIGNED	обязательное поле

Содержание поля	Имя поля	Тип, длина	Примечания
Цена на момент покупки	price	INT	обязательное поле

На рисунке 1.6.1 представлена схема отношений базы данных.

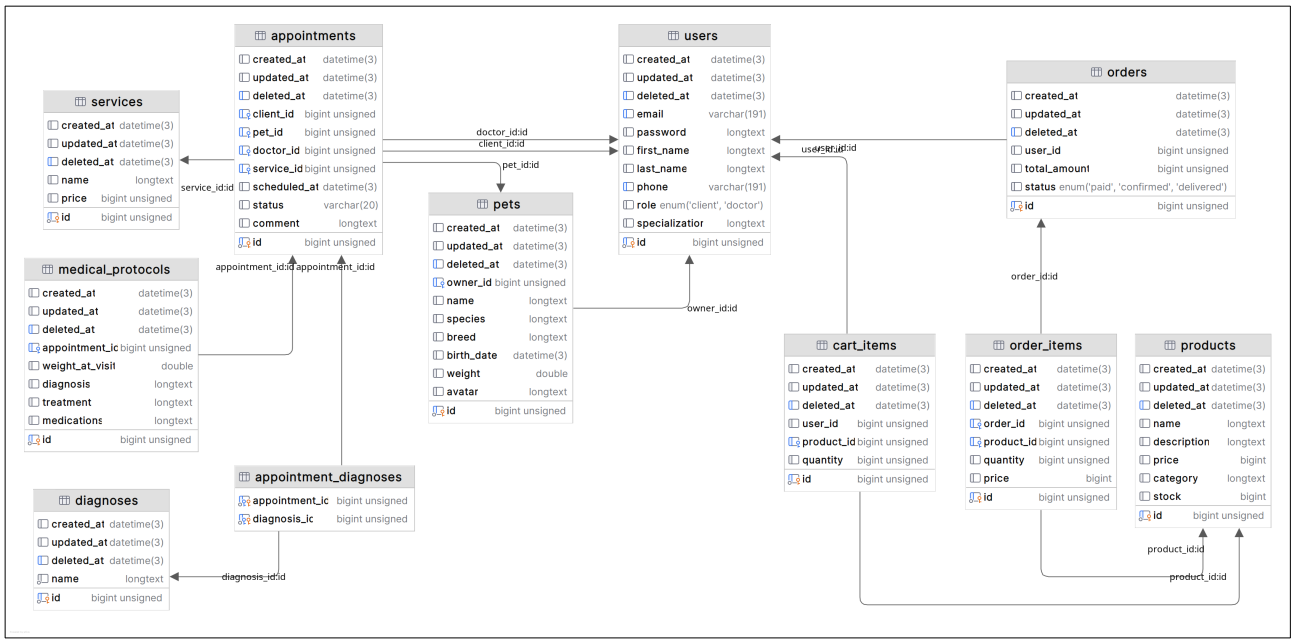


Рисунок 1.6.1 – Схема отношений базы данных

2.1 Описание программы

Разработанная информационная система состоит из двух взаимодействующих программных продуктов: серверного приложения (REST API), реализующего бизнес-логику и доступ к данным, и клиентского веб-приложения, обеспечивающего интерфейс взаимодействия с пользователем.

Серверное приложение разработано на языке программирования Go с использованием веб-фреймворка Gin и ORM-библиотеки GORM. Архитектура серверного приложения построена по слоистому принципу и включает следующие уровни: модели данных (models), репозитории (repository), отвечающие за взаимодействие с базой данных, сервисы (service), реализующие бизнес-логику, обработчики запросов (handler), принимающие HTTP-запросы и формирующие ответы, а также объекты передачи данных (dto), используемые для сериализации запросов и ответов. Авторизация пользователей реализована

на основе JWT-токенов средствами модуля middleware.

Клиентское веб-приложение разработано на языке программирования TypeScript с использованием фреймворка Vue 3 (Composition API) и сборщика Vite. Приложение реализует два рабочих пространства: для клиента клиники и для врача, переключение между которыми осуществляется на основе роли авторизованного пользователя.

Модульная схема программного продукта представлена на рисунке 2.1.1. и 2.1.2

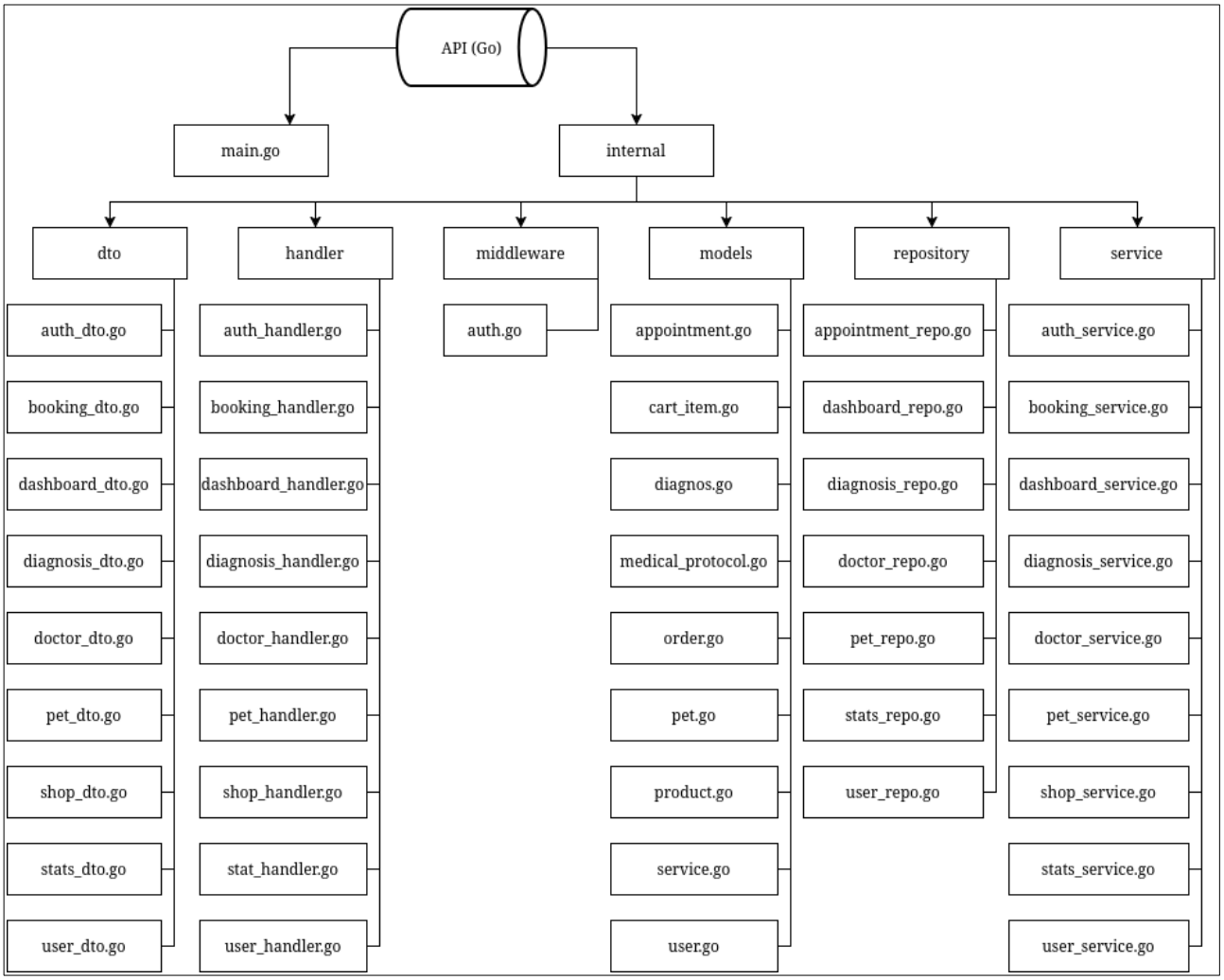


Рисунок 2.1.1 – Модульная схема серверного приложения

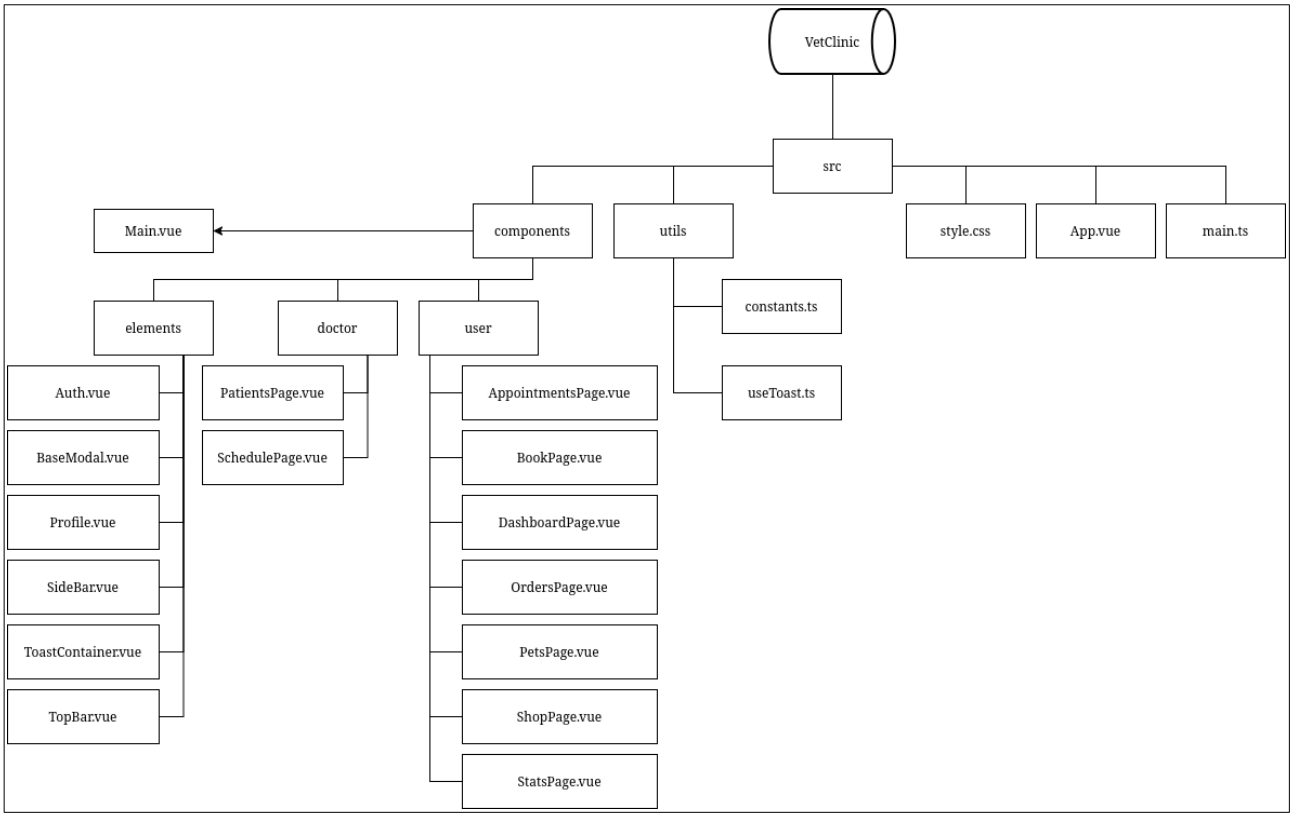


Рисунок 2.1.2 — Модульная схема клиентского веб-приложения

Описание основных модулей серверной части представлено в таблице 2.1.1.

Таблица 2.1.1 – Описание основных модулей серверной части (API)

Модуль	Эндпоинт	Назначение
auth	POST /auth/register	Регистрация нового пользователя
	POST /auth/login	Авторизация пользователя, выдача JWT-токена
user	GET /users/:id	Получение профиля пользователя
	PUT /users/:id	Обновление профиля пользователя
	PUT /users/:id/password	Смена пароля пользователя
pet	GET /pets/owner/:userId	Получение списка питомцев клиента
	POST /pets	Добавление питомца
	PUT /pets/:petId	Редактирование питомца
	DELETE /pets/:petId	Удаление питомца

Модуль	Эндпоинт	Назначение
dashboard	GET /dashboard/:userId	Получение сводной статистики клиента
booking	GET /book/init/:userId	Инициализация формы записи (списки питомцев, врачей, услуг)
	GET /appointments/busy-slots	Получение занятых слотов времени врача
	POST /appointments	Создание записи на приём
	GET /appointments/client/:userId	Получение истории визитов клиента
doctor	GET /doctor/schedule	Получение расписания врача
	PATCH /appointments/:id/complete	Завершение приёма (вес, диагнозы, лечение)
	PATCH /appointments/:id/status	Подтверждение/отклонение записи
	GET /doctor/patients	Поиск пациентов по имени или телефону
	GET /doctor/patients/:id/history	Получение медицинской истории клиента
diagnosis	GET /diagnoses	Получение справочника диагнозов
shop	GET /medications	Получение каталога товаров
	GET /cart/:userId	Получение содержимого корзины
	POST /cart/:userId	Добавление товара в корзину
	PUT /cart/:itemId	Изменение количества товара в корзине
	DELETE /cart/:itemId	Удаление товара из корзины
	POST /orders	Оформление заказа
	GET /orders/:userId	Получение истории заказов
stats	GET /stats/pets/:userId	Получение списка питомцев для статистики
	GET /stats/weight/:petId	Получение данных динамики веса питомца

Описание основных компонентов клиентской части представлено в таблице 2.1.2.

Таблица 2.1.2 – Описание основных компонентов клиентской части

Компонент	Назначение
Main.vue	Корневой компонент, переключение между страницами клиента и врача по роли пользователя
components/elements/Auth.vue	Форма авторизации и регистрации
components/elements/ SideBar.vue	Боковое меню навигации
components/elements/ TopBar.vue	Верхняя панель с профилем пользователя
components/elements/Profile.vue	Просмотр и редактирование профиля
components/elements/ BaseModal.vue	Универсальный компонент модального окна
components/user/ DashboardPage.vue	Дашборд клиента: статистика, ближайшая запись, список питомцев
components/user/PetsPage.vue	Управление карточками питомцев
components/user/BookPage.vue	Форма записи на приём
components/user/ AppointmentsPage.vue	История визитов клиента, просмотр медицинского протокола, экспорт в PDF
components/user/StatsPage.vue	Графики динамики веса питомцев
components/user/ShopPage.vue	Каталог товаров, корзина
components/user/OrdersPage.vue	История заказов клиента
components/doctor/ SchedulePage.vue	Расписание врача, подтверждение/отклонение записей, завершение приёма с указанием диагнозов
components/doctor/ PatientsPage.vue	Картотека пациентов, поиск, просмотр медицинской истории

2.2 Протокол тестирования программного продукта

В ходе тестирования программного продукта на корректных и некорректных данных не было обнаружено ошибок, влияющих на работу программы и системы в целом.

В таблицах 2.2 - 2.12 представлены протоколы тестирования.

Таблица 2.2.1 – Протокол тестирования успешной регистрации клиента

Дата теста	15.06.2026
	«Зарегистрироваться»
Тестовые данные	Имя: Иван, Фамилия: Иванов, Email: ivanov@gmail.com, Телефон: +79991112233, Пароль: 12345678
Ожидаемый результат	Учётная запись создана, переход на страницу авторизации
Фактический результат	Учётная запись создана, переход на страницу авторизации. Рез

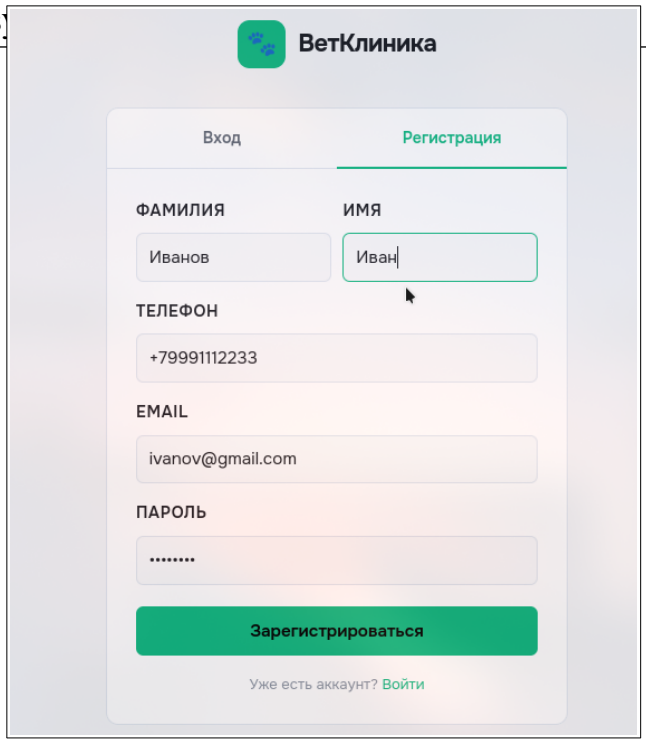


Рисунок 2.2.1.1 – Запись данных в поля для регистраций

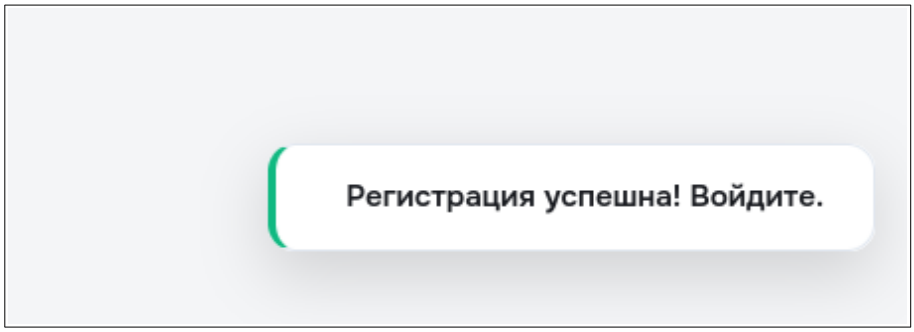


Рисунок 2.2.1.2 — Результат тестирования регистрации при корректных данных

Таблица 2.2.2 – Протокол тестирования регистрации с занятым email

Дата теста	15.06.2026
Приоритет	Средний

Дата теста	15.06.2026
тестирования	
Заголовок/ название теста	Тестирование регистрации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при попытке регистрации с уже существующим email
Этапы теста	Открыть форму регистрации; заполнить поля, указав email уже зарегистрированного пользователя; нажать кнопку «Зарегистрироваться»
Тестовые данные	Email: ivanov@gmail.com (уже существует в системе)
Ожидаемый результат	Вывод сообщения об ошибке, регистрация не выполнена
Фактический результат	Вывод сообщения об ошибке. Результат представлен на рисунке 2.2.2.2

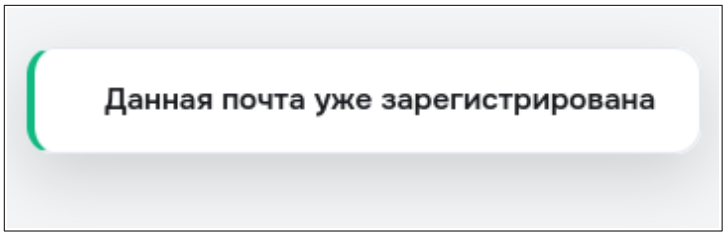


Рисунок 2.2.2.1 – Запись данных в поля для регистраций

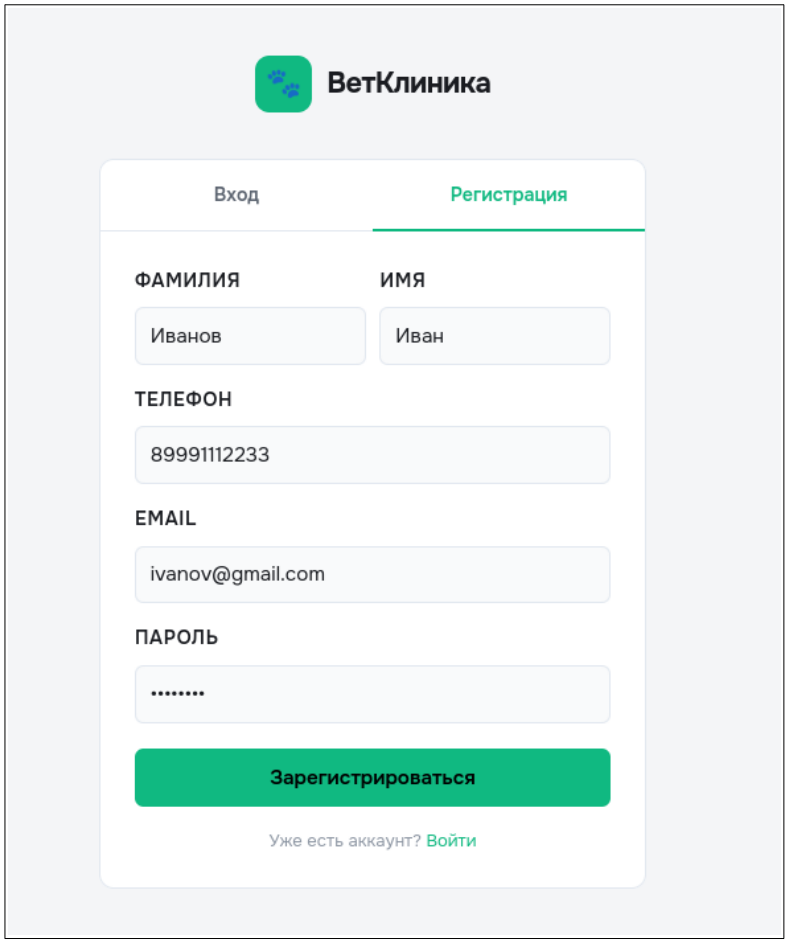


Рисунок 2.2.2.2 – Результат тестирования регистрации при некорректных данных

Таблица 2.2.3 - Протокол тестирования успешной аутентификации

Дата теста	15.06.2026
Приоритет тестирования	Высокий
Заголовок/ название теста	Тестирование аутентификации при корректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при вводе корректных данных
Этапы теста	Ввести Email в текстовое поле «Email»; ввести пароль в текстовое поле «Пароль»; нажать кнопку «Войти»
Тестовые данные	Email: ivanov@gmail.com, Пароль: 12345678
Ожидаемый результат	Успешная авторизация, переход на дашборд
Фактический	Переход на дашборд. Результат представлен на рисунке

результат	2.2.3.2

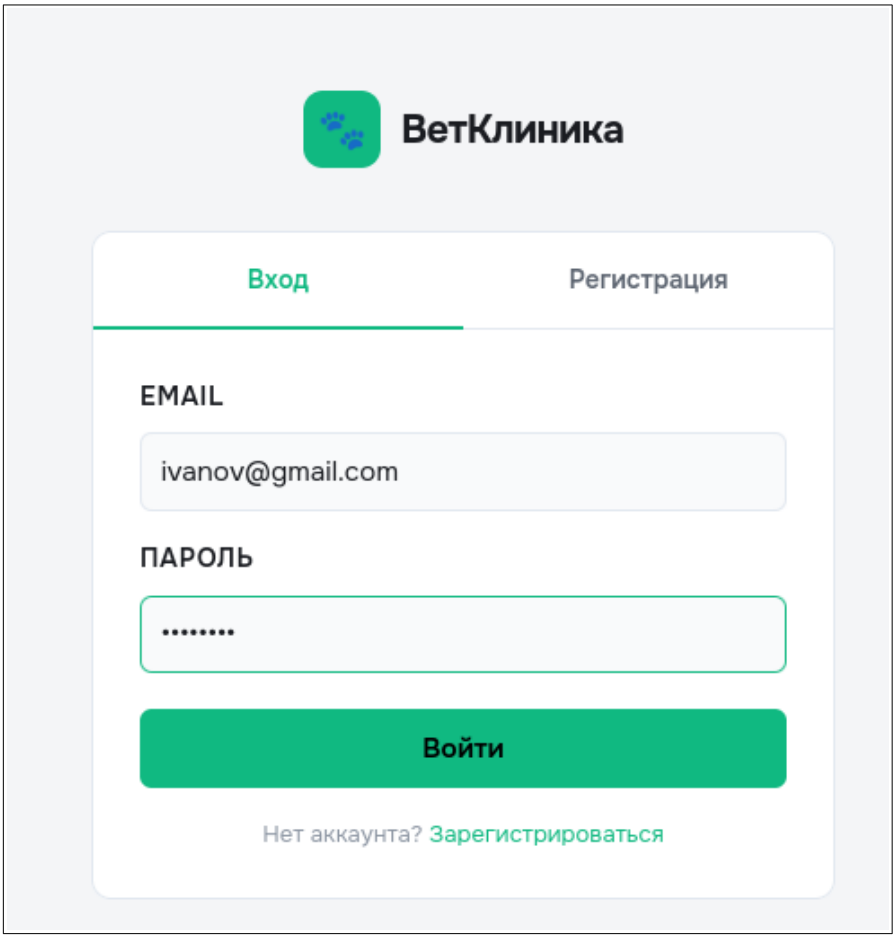


Рисунок 2.2.3.1 – Запись данных в поля для авторизаций

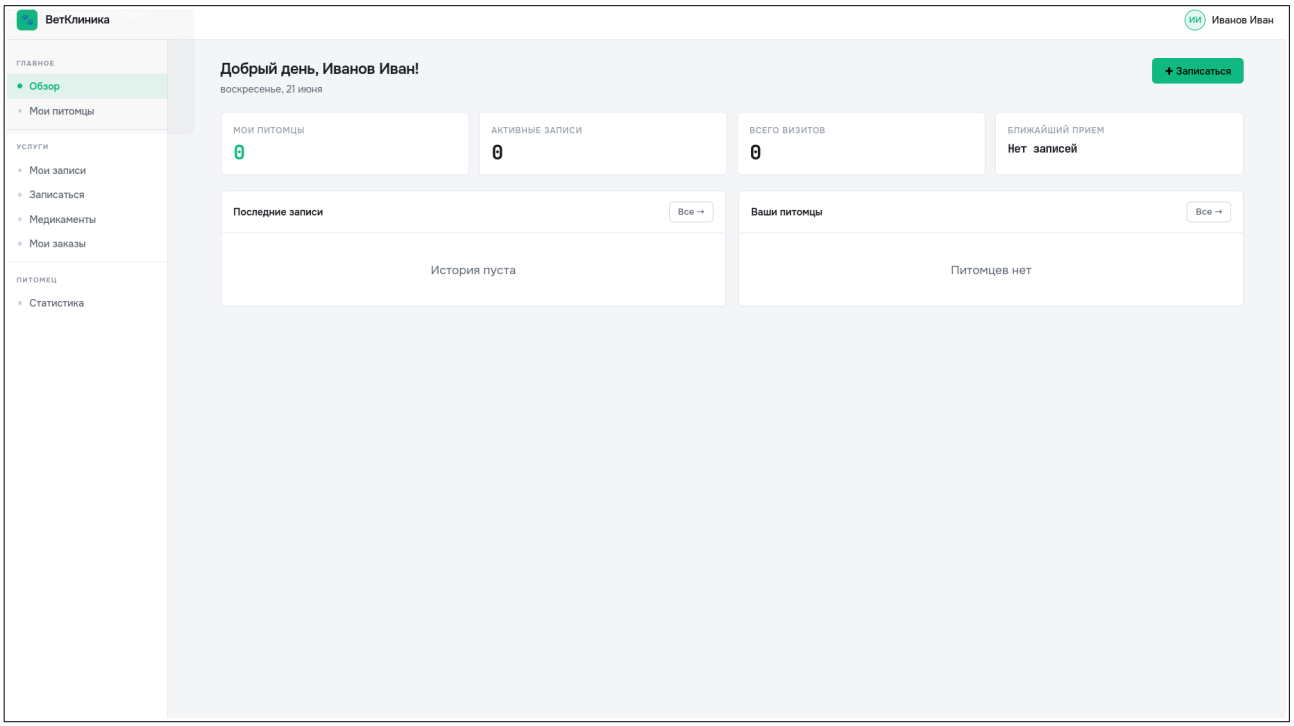


Рисунок 2.2.3.2– Результат тестирования аутентификации при корректных данных

Таблица 2.2.4 – Протокол тестирования неудачной аутентификации

Дата теста	15.06.2026
Приоритет тестирования	Высокий
Заголовок/ название теста	Тестирование аутентификации при некорректных данных
Резюме испытания	Необходимо добиться корректного поведения программы при вводе неверного пароля
Этапы теста	Ввести Email в текстовое поле «Email»; ввести неверный пароль в текстовое поле «Пароль»; нажать кнопку «Войти»
Тестовые данные	Email: ivanov@gmail.com, Пароль: wrongpass
Ожидаемый результат	Вывод сообщения об ошибке авторизации
Фактический результат	Вывод сообщения об ошибке авторизации. Результат представлен на рисунке 2.2.4.1

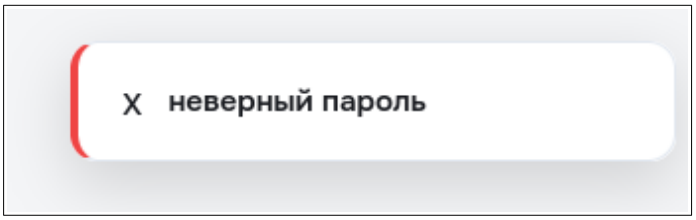


Рисунок 2.2.4.1 – Результат тестирования аутентификации при некорректных данных

Таблица 2.2.5 – Протокол тестирования добавления питомца

Дата теста	15.06.2026
Приоритет тестирования	Средний

Дата теста	15.06.2026
Заголовок/ название теста	Тестирование добавления карточки питомца
Резюме испытания	Необходимо добиться корректного добавления питомца в систему
Этапы теста	Перейти на страницу «Питомцы»; нажать кнопку «Добавить питомца»; заполнить поля «Имя», «Вид», «Порода», «Дата рождения», «Вес»; нажать кнопку «Сохранить»
Тестовые данные	Имя: Барсик, Вид: Кошка, Порода: Британская, Дата рождения: 12.05.2021, Вес: 4.2
Ожидаемый результат	Питомец добавлен, отображается в списке питомцев клиента
Фактический результат	Питомец добавлен и отображается в списке. Результат представлен на рисунке 2.2.5.2

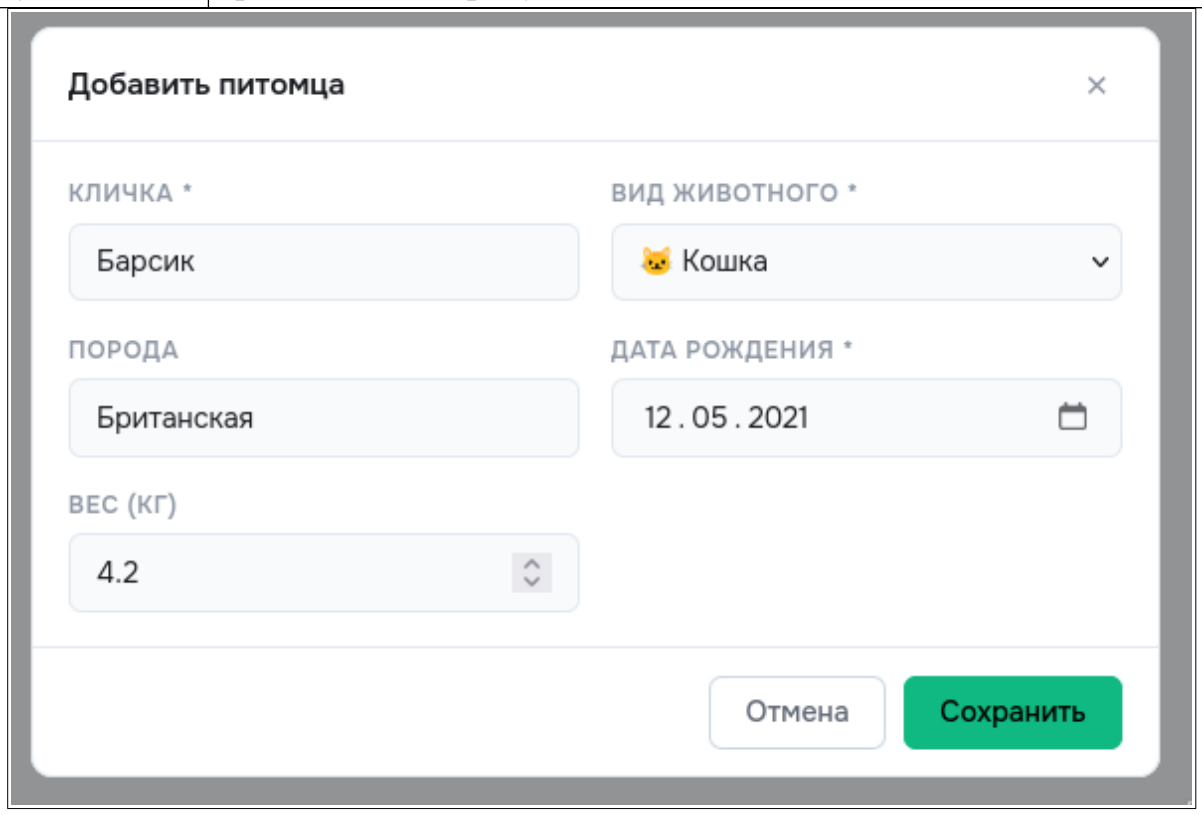


Рисунок 2.2.5.1 – Добавление данных о питомце в поля

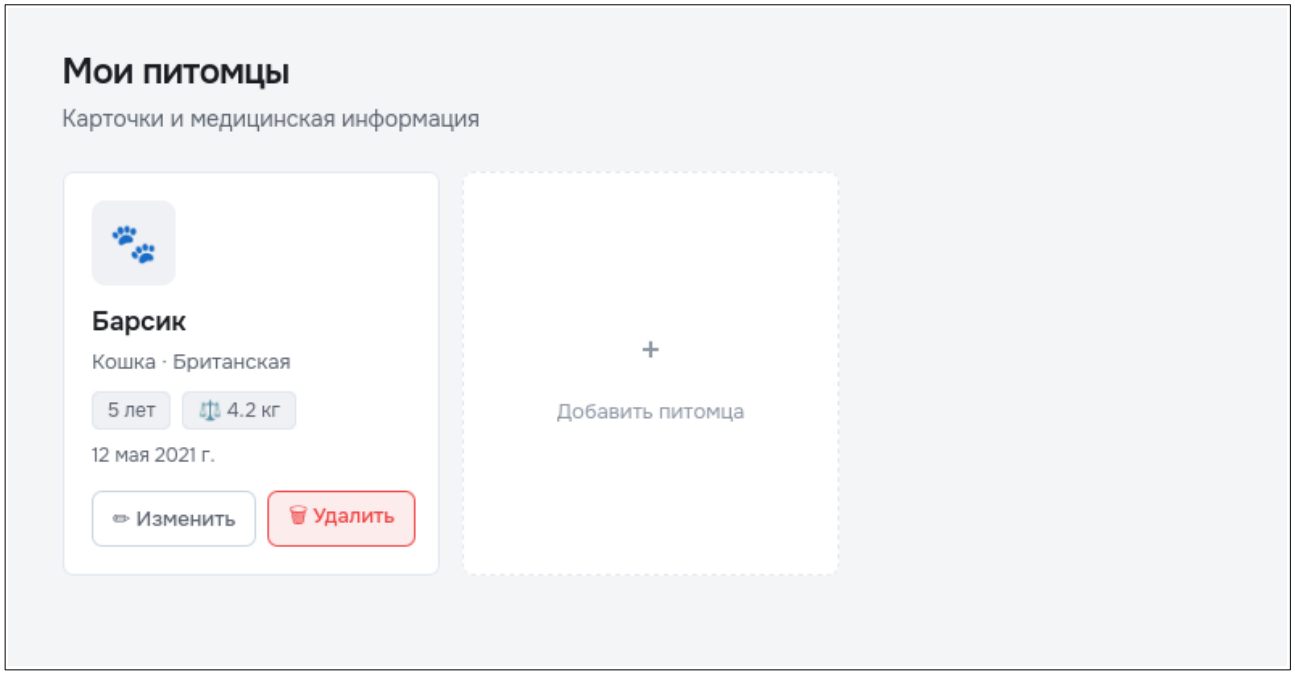


Рисунок 2.2.5.2 – Результат тестирования добавления питомца

Таблица 2.2.6 – Протокол тестирования записи на приём

Дата теста	15.06.2026
Приоритет тестирования	Высокий
Заголовок/ название теста	Тестирование записи клиента на приём
Резюме испытания	Необходимо добиться корректного поведения программы при создании записи на приём
Этапы теста	Перейти на страницу «Запись на приём»; выбрать питомца, врача, услугу и время приёма; нажать кнопку «Записаться»
Тестовые данные	Питомец: Барсик, Врач: Смирнов Иван, Услуга: Первичный осмотр и консультация, Время: 22.06.2026 10:00
Ожидаемый результат	Запись создана со статусом «ожидает»
Фактический результат	Запись создана со статусом «ожидает», отображается в истории визитов. Результат представлен на рисунке 2.2.6.2

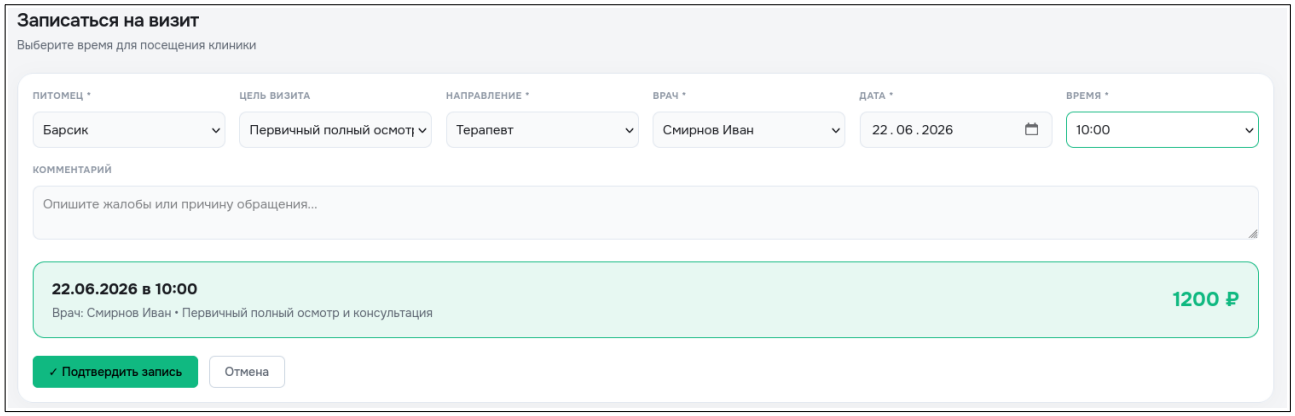


Рисунок 2.2.6.1 – Добавление данных для записи на приём

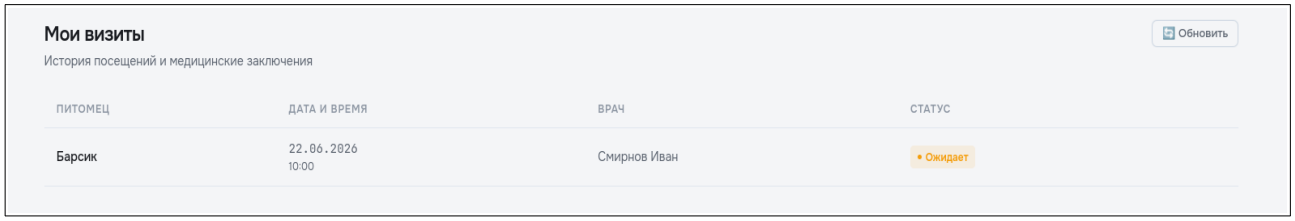


Рисунок 2.2.6.2 – Результат тестирования записи на приём

Таблица 2.2.7 – Протокол тестирования записи на занятое время

Дата теста	15.06.2026
Приоритет тестирования	Средний
Заголовок/ название теста	Тестирование блокировки занятого времени приёма
Резюме испытания	Необходимо убедиться, что время, занятое другой записью к тому же врачу, недоступно для выбора
Этапы теста	Перейти на страницу «Запись на приём»; выбрать врача, у которого уже есть запись на 10:00; открыть список доступного времени
Тестовые данные	Врач: Смирнова А.А., Дата: 22.06.2026
Ожидаемый результат	Время 10:00 отсутствует в списке доступных слотов
Фактический результат	Время 10:00 отсутствует в списке доступных слотов. Результат представлен на рисунке 2.2.7.1

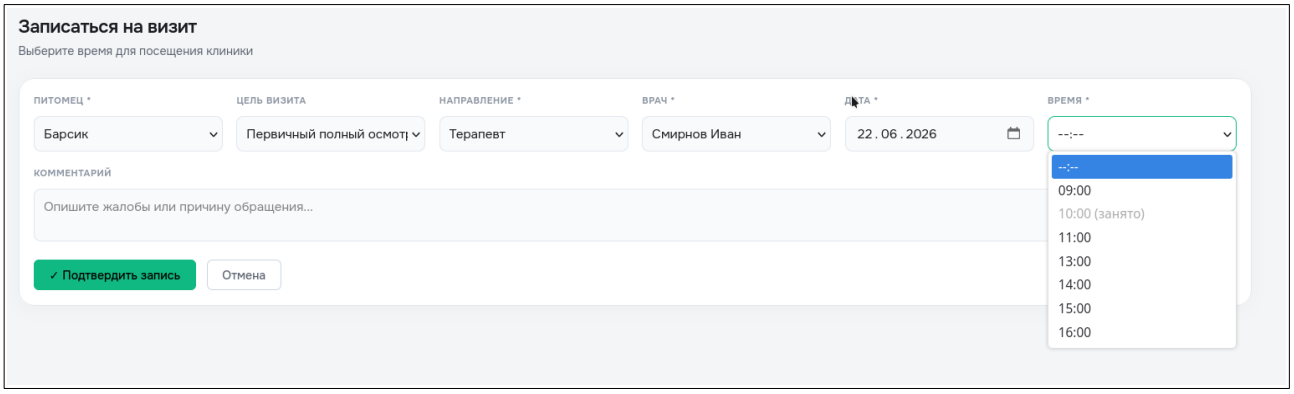


Рисунок 2.2.7.1 – Результат тестирования блокировки занятого времени

Таблица 2.2.8 – Протокол тестирования подтверждения записи врачом

Дата теста	15.06.2026
Приоритет тестирования	Высокий
Заголовок/ название теста	Тестирование подтверждения записи на приём
Резюме испытания	Необходимо добиться корректного изменения статуса записи при подтверждении врачом

Дата теста	15.06.2026
Этапы теста	Перейти на страницу «Приём пациентов»; найти запись со статусом «Ожидает»; нажать кнопку «Принять»
Тестовые данные	Запись: Барсик, 22.06.2026 10:00, статус «waiting»
Ожидаемый результат	Статус записи изменён на «Принята»
Фактический результат	Статус записи изменён на «Принята». Результат представлен на рисунке 2.2.8.2

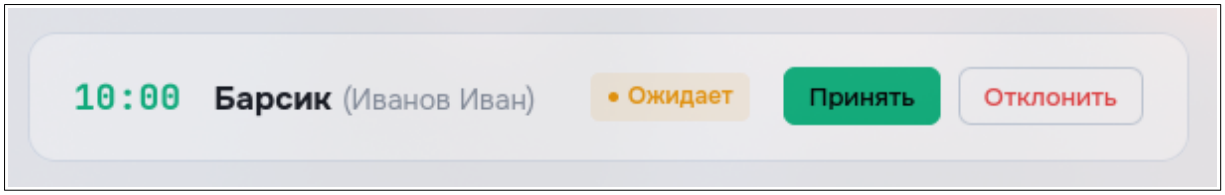


Рисунок 2.2.8.1 – Ожидание принятия питомца

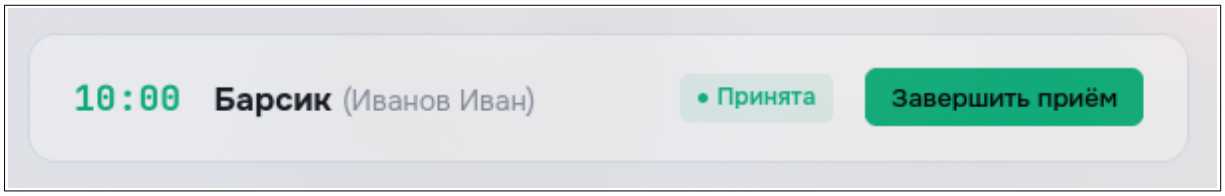


Рисунок 2.2.8.2 – Результат тестирования подтверждения записи

Таблица 2.2.9 – Протокол тестирования отклонения записи врачом

Дата теста	15.06.2026
Приоритет тестирования	Средний
Заголовок/ название теста	Тестирование отклонения записи на приём
Резюме испытания	Необходимо добиться корректного изменения статуса записи при отклонении врачом
Этапы теста	Перейти на страницу «Приём пациентов»; найти запись со статусом «Ожидает»; нажать кнопку «Отклонить»
Тестовые данные	Запись: Барсик, 22.06.2026 11:00, статус «waiting»
Ожидаемый	Статус записи изменён на «Отклонена»

Дата теста	15.06.2026
результат	
Фактический результат	Статус записи изменён на «Отклонена». Результат представлен на рисунке 2.2.9.1

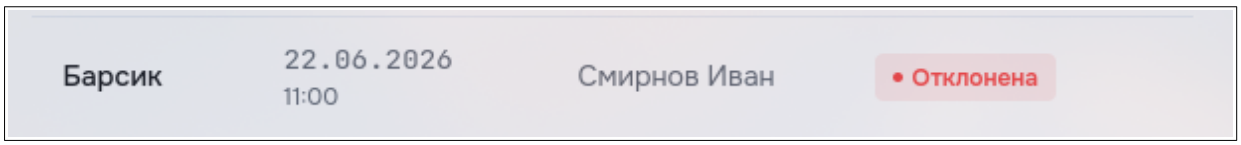


Рисунок 2.2.9.1– Результат тестирования отклонения записи

Таблица 2.2.10 – Протокол тестирования завершения приёма с указанием диагнозов

Дата теста	15.06.2026
Приоритет тестирования	Высокий
Заголовок/ название теста	Тестирование завершения приёма с фиксацией диагнозов
Резюме испытания	Необходимо добиться корректного сохранения медицинского протокола с диагнозами
Этапы теста	Открыть подтверждённую запись; нажать кнопку «Завершить приём»; указать вес; добавить диагнозы через кнопку «+»; заполнить лечение и медикаменты; нажать «Подтвердить и закрыть визит»
Тестовые данные	Вес: 5.4, Диагнозы: Острый гастроэнтерит, Хроническая почечная недостаточность (ХПН), Лечение: Уход, Медикаменты: Лекарства
Ожидаемый результат	Протокол сохранён, статус записи изменён на «Завершена», диагнозы отображаются в истории
Фактический результат	Протокол сохранён корректно. Результат представлен на рисунке 2.2.10.2

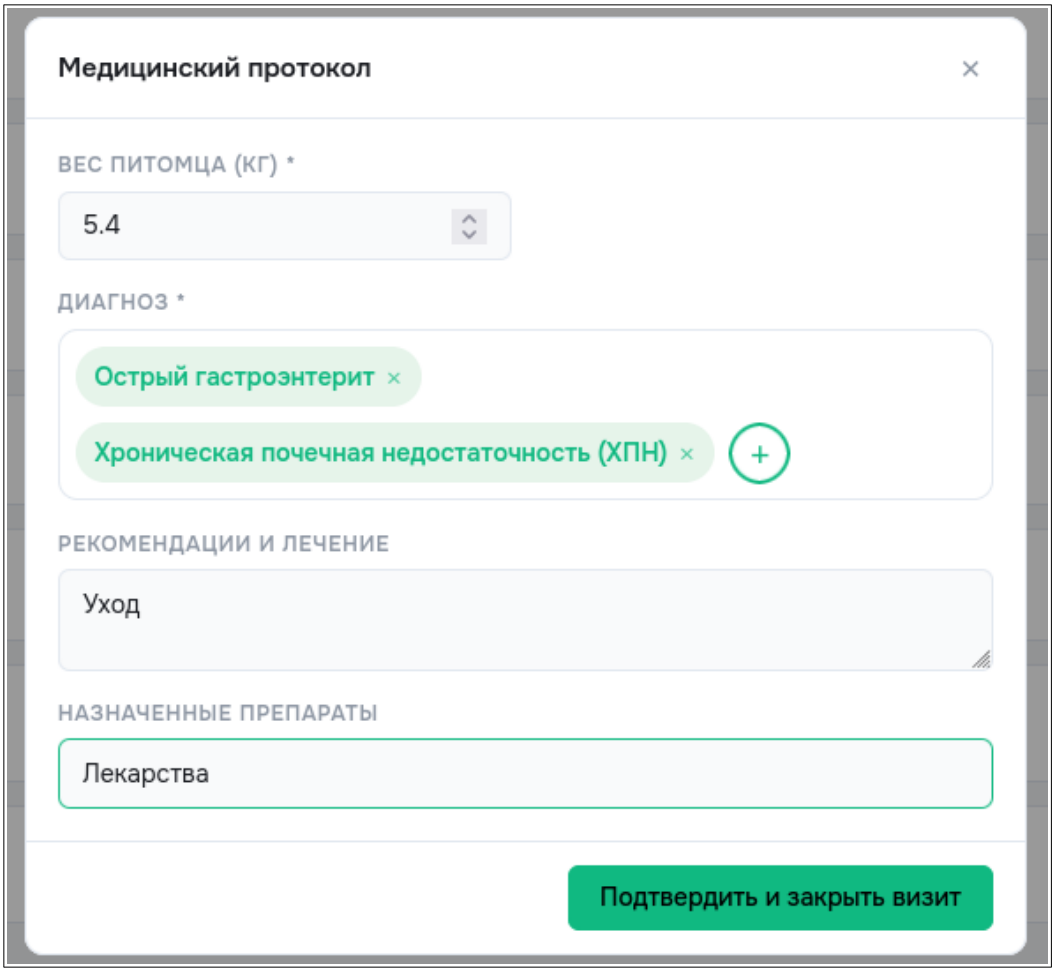


Рисунок 2.2.10.1 – Заполнение данных визита



Рисунок 2.2.10.2– Результат тестирования завершения приёма с диагнозами

Таблица 2.2.11 – Протокол тестирования завершения приёма без указания диагноза

Дата теста	15.06.2026
Приоритет тестирования	Средний
Заголовок/ название теста	Тестирование валидации обязательности диагноза
Резюме	Необходимо убедиться, что протокол не сохраняется без

Дата теста	15.06.2026
испытания	указания диагноза
Этапы теста	Открыть подтверждённую запись; нажать «Завершить приём»; указать вес, не выбирая диагноз; нажать «Подтвердить и закрыть визит»
Тестовые данные	Вес: 3.9, Диагнозы: не выбраны
Ожидаемый результат	Сохранение блокируется, выводится сообщение о необходимости указать диагноз
Фактический результат	Сохранение заблокировано, выведено сообщение «Поле "Вес" и хотя бы один диагноз обязательны для заполнения». Результат представлен на рисунке 2.2.11.1

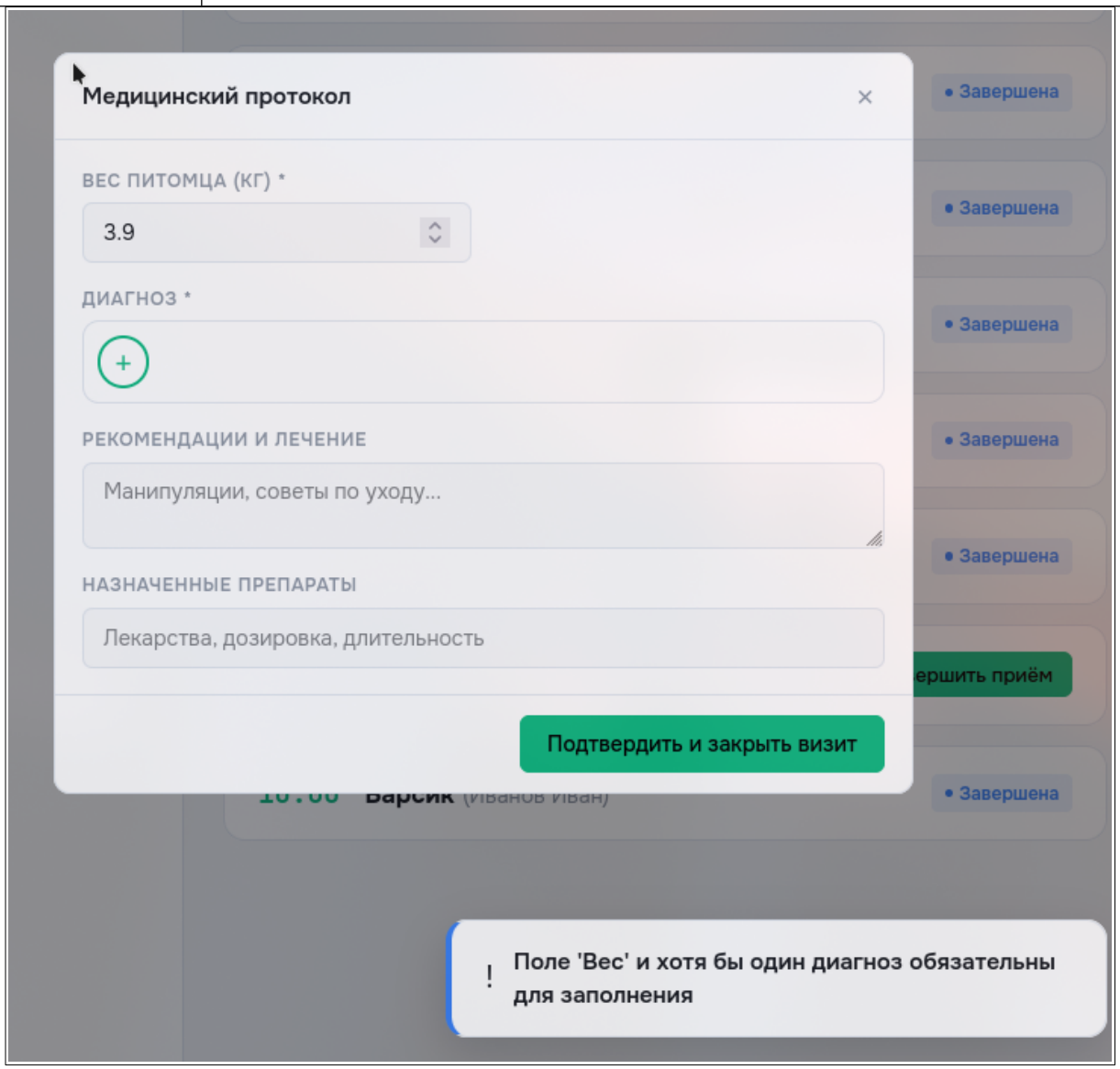


Рисунок 2.2.11.1 – Результат тестирования валидации обязательности диагноза

Таблица 2.2.12 – Протокол тестирования просмотра истории посещений клиентом

Дата теста	15.06.2026
Приоритет тестирования	Средний
Заголовок/ название теста	Тестирование отображения истории визитов
Резюме испытания	Необходимо убедиться, что клиент видит корректную историю своих визитов с диагнозами
Этапы теста	Авторизоваться под клиентом; перейти на страницу «Мои визиты»; открыть протокол завершённого визита
Тестовые данные	Клиент с одним завершённым визитом
Ожидаемый результат	Отображение визита с диагнозами, лечением и медикаментами
Фактический результат	Визит отображается корректно. Результат представлен на рисунке 2.2.12.1

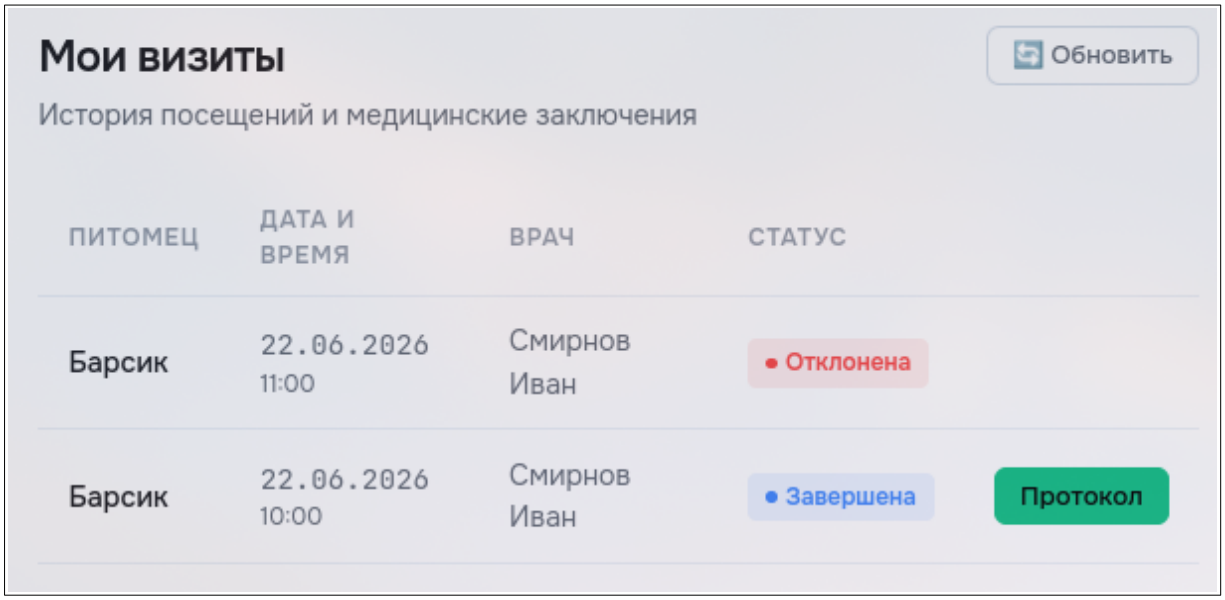


Рисунок 2.2.12.1 – Результат тестирования просмотра истории посещений

2.3 Руководство пользователя

2.3.1 Общие сведения

Для работы с информационной системой пользователю необходим компьютер с

доступом в интернет и установленным современным веб-браузером (Google Chrome, Mozilla Firefox, Microsoft Edge). Специальная установка программного обеспечения на компьютер пользователя не требуется — система работает через веб-браузер.

Система предусматривает два типа пользователей с различным функционалом: клиент (владелец питомца) и врач клиники.

2.3.2 Регистрация и авторизация

Для начала работы с системой новому пользователю необходимо зарегистрироваться. Для этого следует открыть веб-приложение в браузере и на стартовой странице перейти по ссылке «Регистрация».

В открывшейся форме необходимо заполнить поля: «Имя», «Фамилия», «Email», «Телефон», «Пароль» — и нажать кнопку «Зарегистрироваться». Форма регистрации представлена на рисунке 2.3.1.

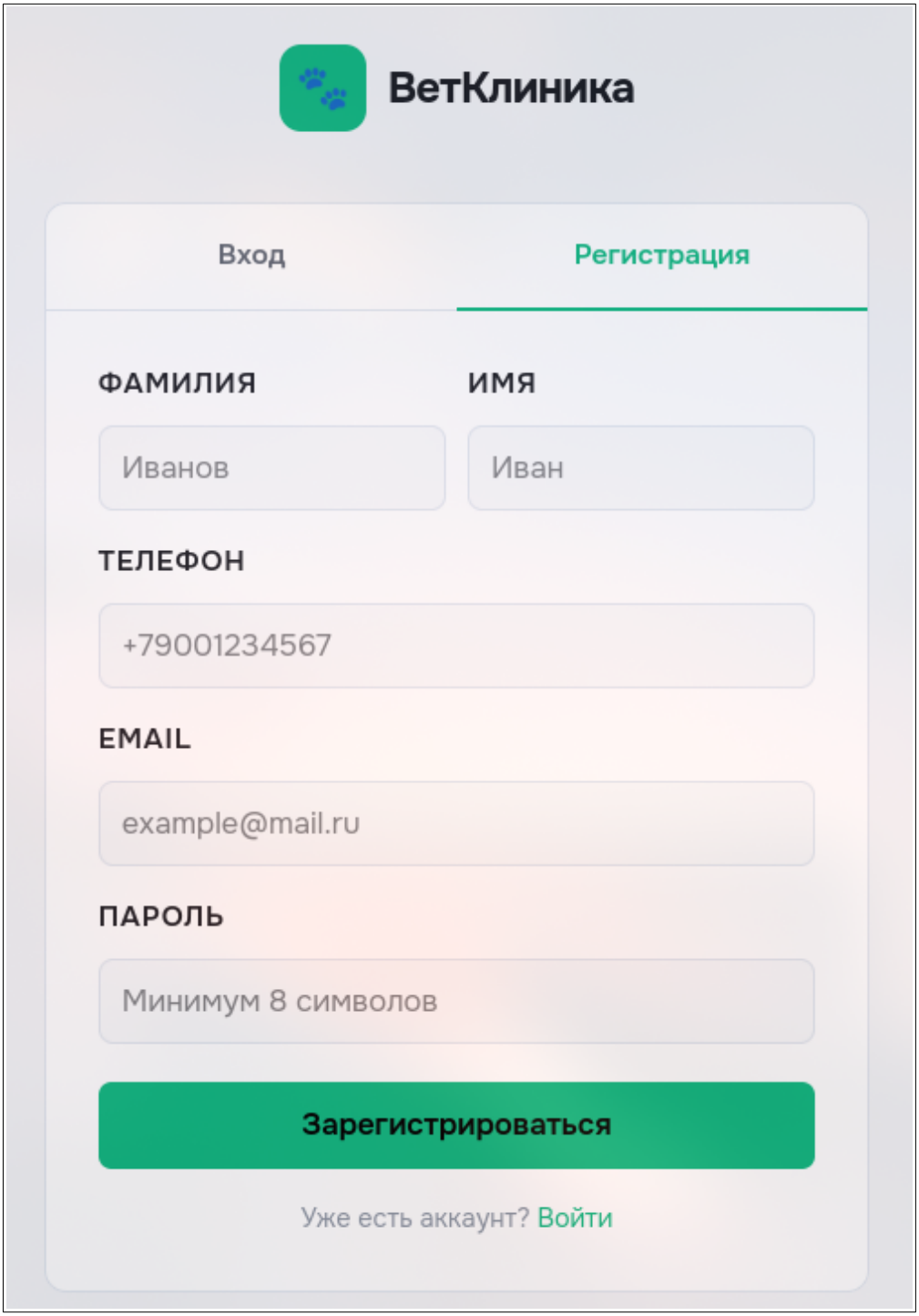


Рисунок 2.3.1 – Форма регистрации

После успешной регистрации пользователь автоматически попадает на форму авторизации, где необходимо ввести email и пароль, указанные при регистрации, и нажать кнопку «Войти». Форма авторизации представлена на рисунке 2.3.2.

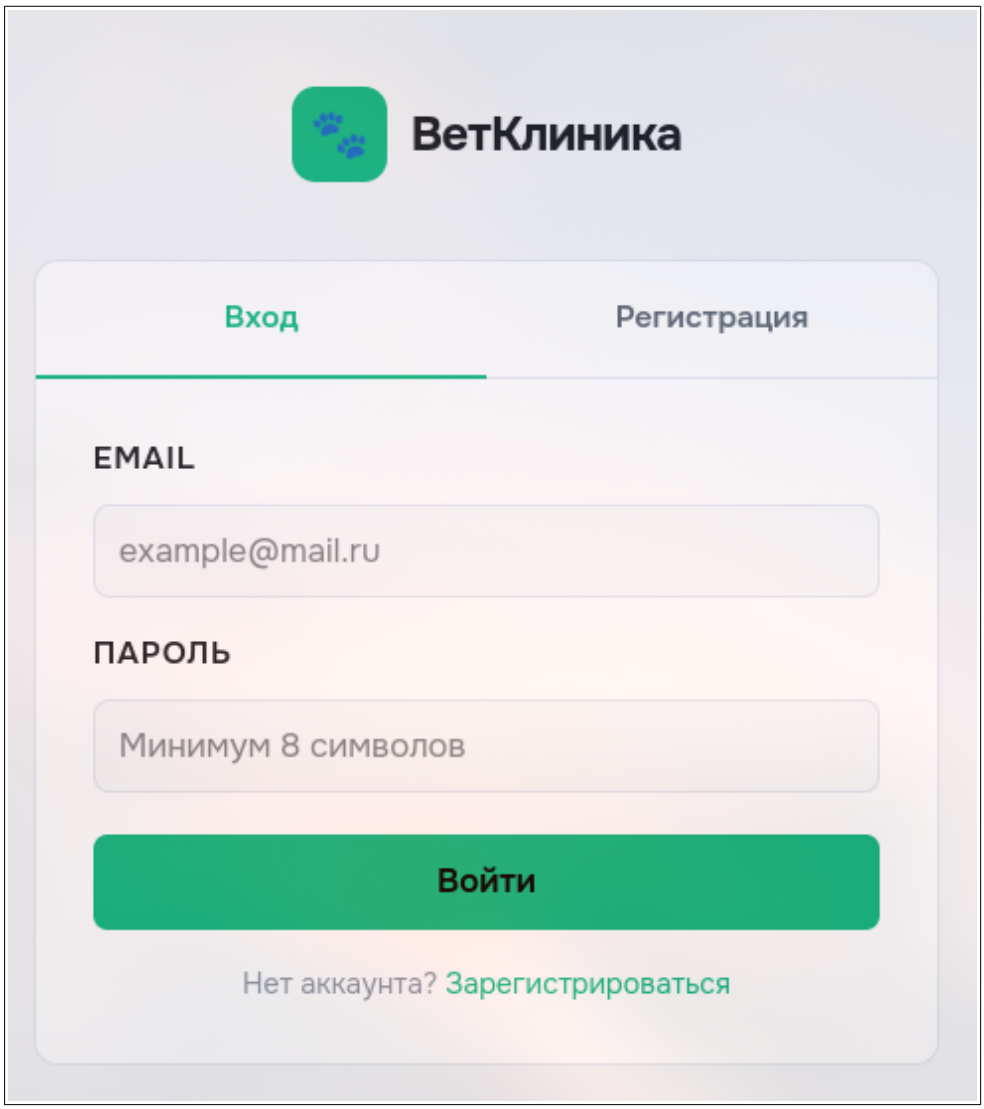


Рисунок 2.3.2 – Форма авторизации

После успешного входа пользователь попадает в рабочее пространство, соответствующее его роли: клиент — на дашборд, врач — на страницу расписания приёмов.

2.3.3 Работа клиента с системой

Дашборд. После авторизации клиент видит главную страницу с общей статистикой: количество питомцев, количество предстоящих записей, количество завершённых визитов, информация о ближайшем приёме и список питомцев. Внешний вид дашборда представлен на рисунке 2.3.3.

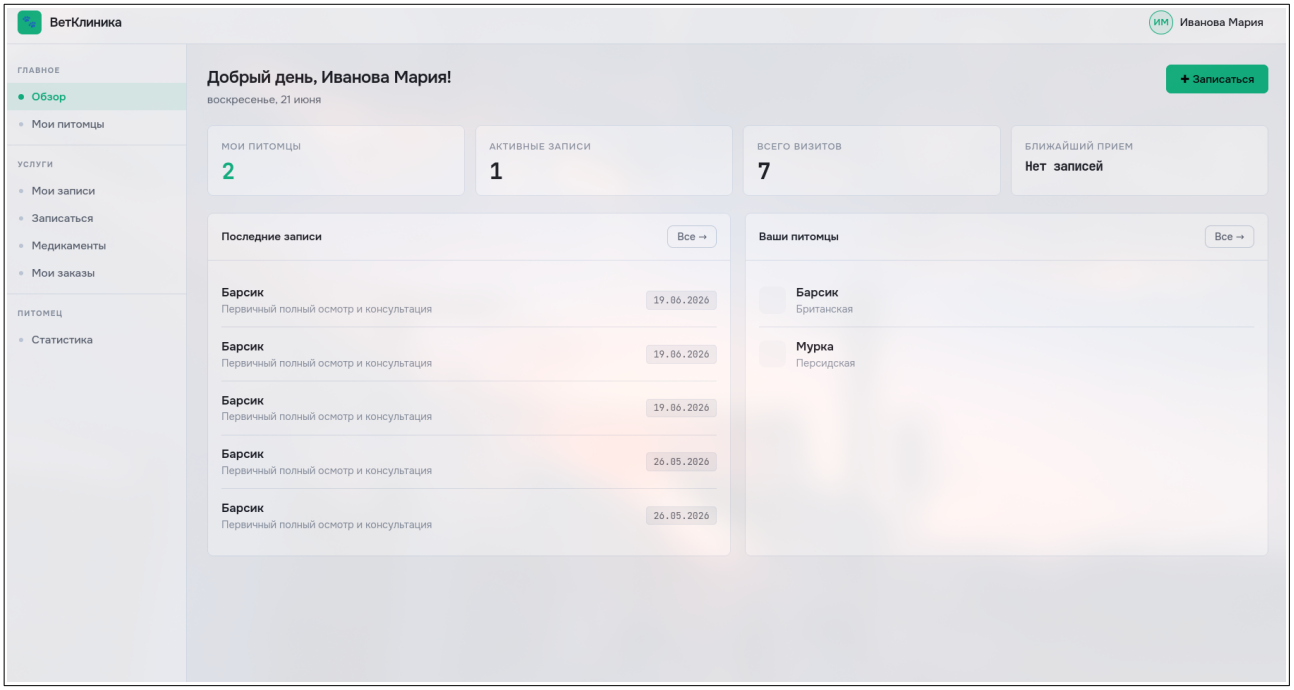


Рисунок 2.3.3 – Дашборд клиента

Управление питомцами. Для добавления питомца необходимо перейти в раздел «Питомцы» через боковое меню, нажать кнопку «Добавить питомца» и заполнить открывшуюся форму: имя, вид животного, порода, дата рождения, вес, фотография. Для редактирования или удаления питомца следует воспользоваться соответствующими кнопками на карточке питомца. Страница управления питомцами представлена на рисунке 2.3.4.

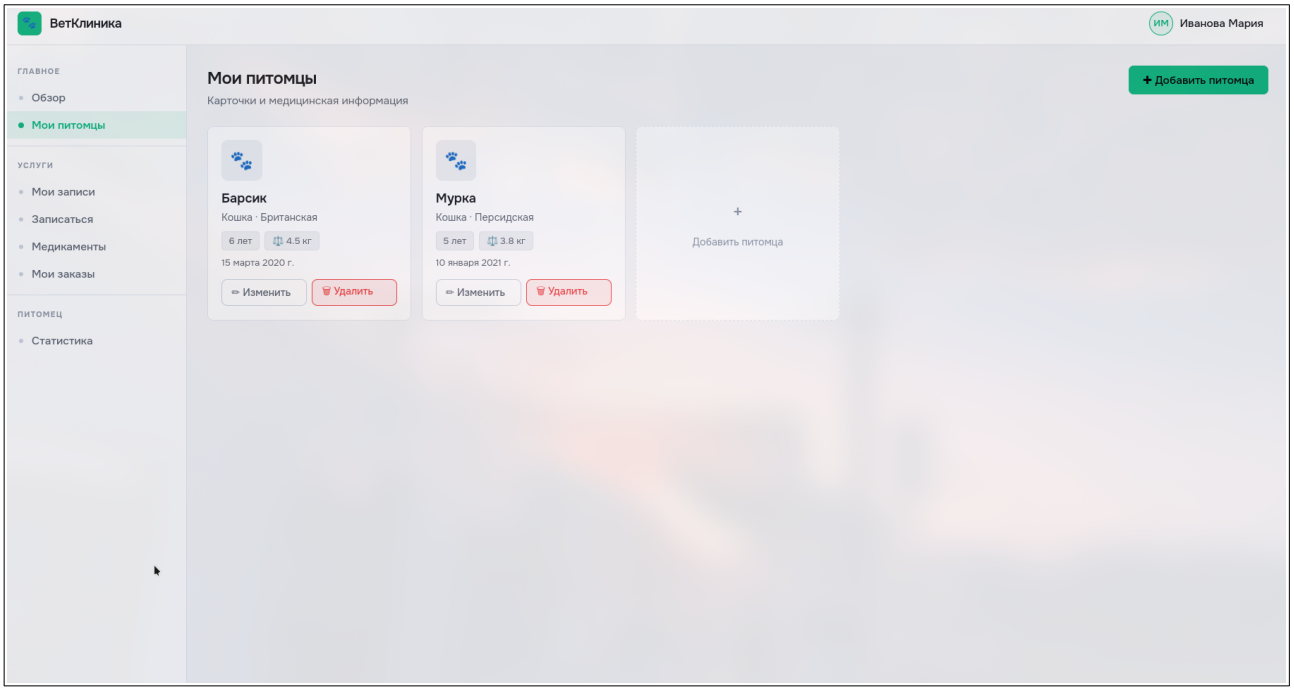


Рисунок 2.3.4 – Страница «Питомцы»

Запись на приём. Для записи на приём необходимо перейти в раздел «Запись на приём», выбрать питомца, врача, услугу и удобное время из списка свободных слотов, при необходимости заполнить комментарий, после чего нажать кнопку «Записаться». Форма записи на приём представлена на рисунке 2.3.5.

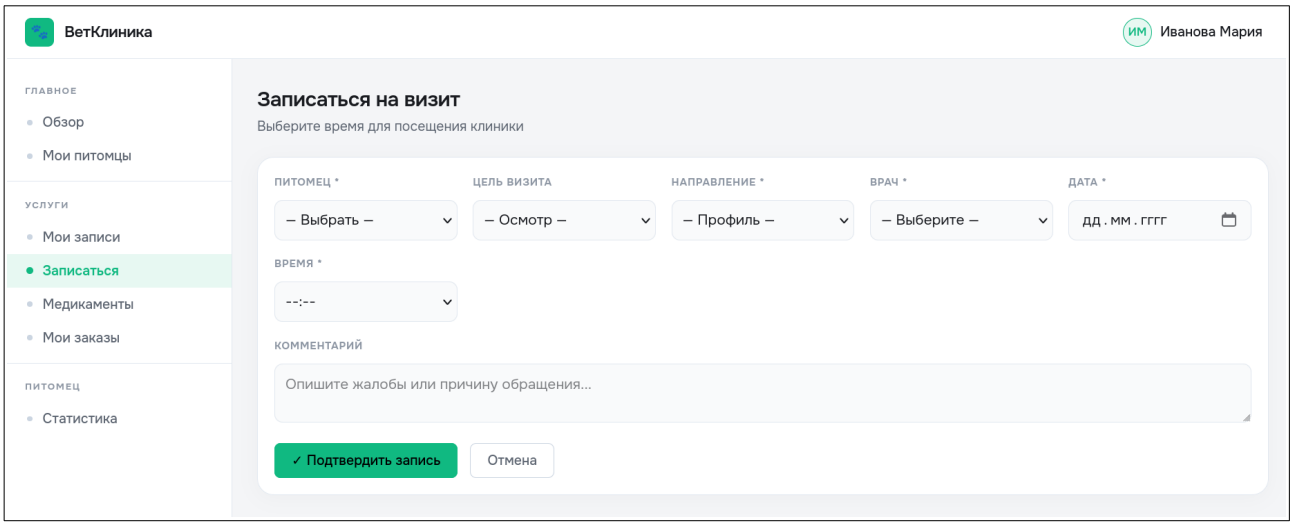


Рисунок 2.3.5 – Форма записи на приём

Просмотр истории посещений. Для просмотра истории визитов необходимо перейти в раздел «Мои визиты». В таблице отображаются все записи клиента с

указанием питомца, даты, времени, врача и статуса. Для завершённых визитов доступна кнопка «Протокол», по нажатию на которую открывается медицинское заключение с указанием диагнозов, лечения и медикаментов. Страница истории визитов представлена на рисунке 2.3.6.

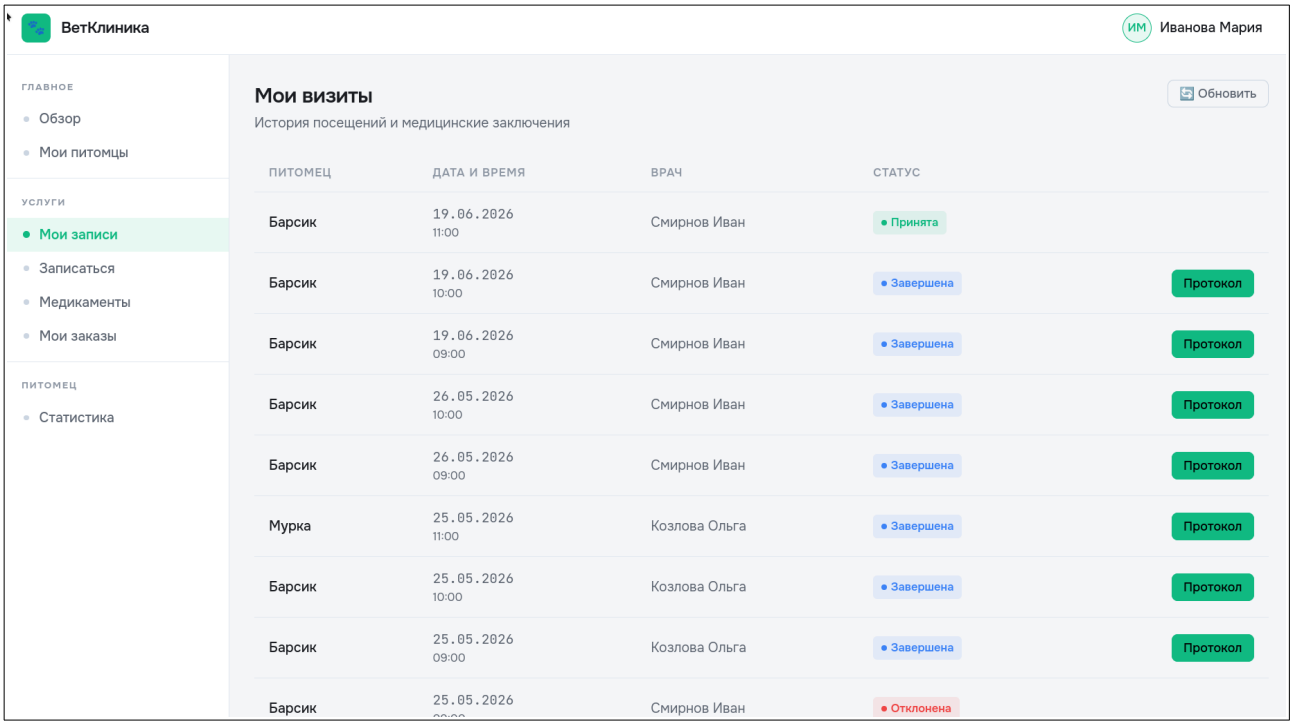


Рисунок 2.3.6 – Страница «Мои визиты»

Для сохранения медицинского заключения в виде документа необходимо в открытом протоколе нажать кнопку «Сохранить в PDF». Окно с протоколом визита представлено на рисунке 2.3.7.



Рисунок 2.3.7 – Медицинское заключение визита

Статистика. Для просмотра динамики веса питомца необходимо перейти в раздел «Статистика», выбрать питомца — система отобразит график изменения веса по датам визитов. Страница статистики представлена на рисунке 2.3.8.

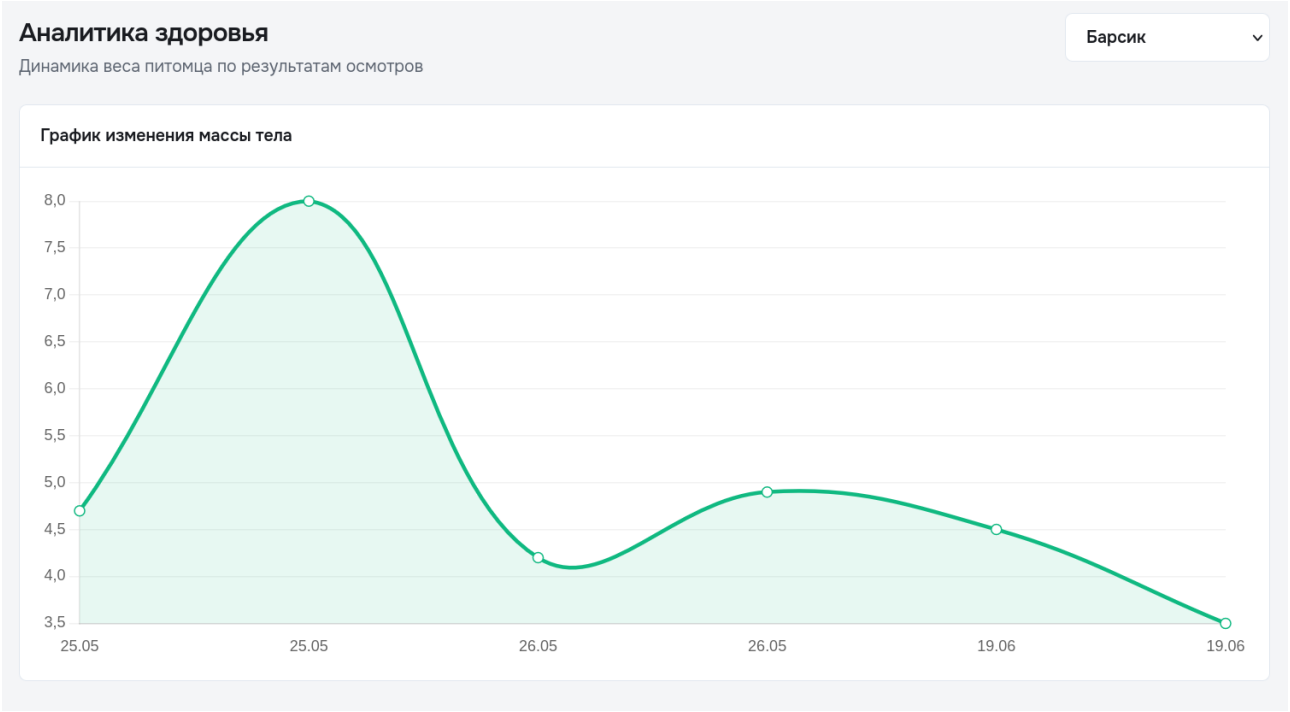


Рисунок 2.3.8 – Страница статистики питомца

Магазин товаров. Для покупки товаров необходимо перейти в раздел «Магазин», выбрать товар и нажать кнопку «В корзину». Содержимое корзины доступно по ссылке «Корзина», где можно изменить количество товара, удалить позицию или оформить заказ кнопкой «Оформить заказ». История оформленных заказов доступна в разделе «Мои Заказы». Страница магазина представлена на рисунке 2.3.9.

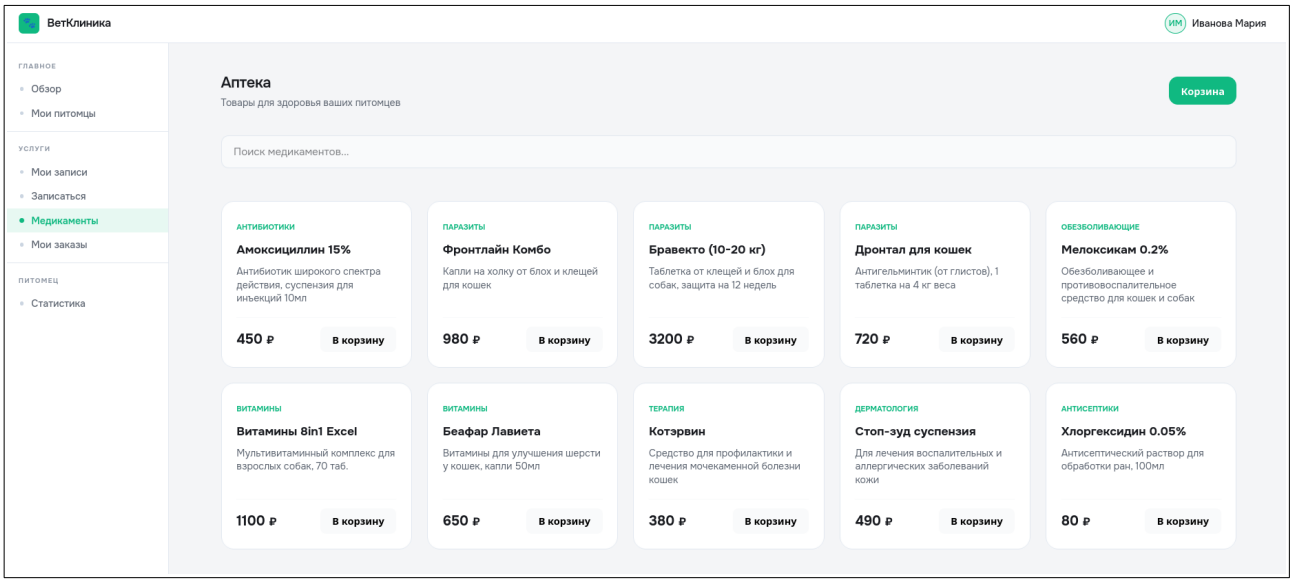


Рисунок 2.3.9 – Страница «Магазин»

2.3.4 Работа врача с системой

Расписание приёма. После авторизации врач попадает на страницу «Приём пациентов», где отображается список записей на текущий день с указанием времени, питомца, владельца и статуса записи. Для новой записи со статусом «Ожидает» доступны кнопки «Принять» и «Отклонить». Страница расписания представлена на рисунке 2.3.10.

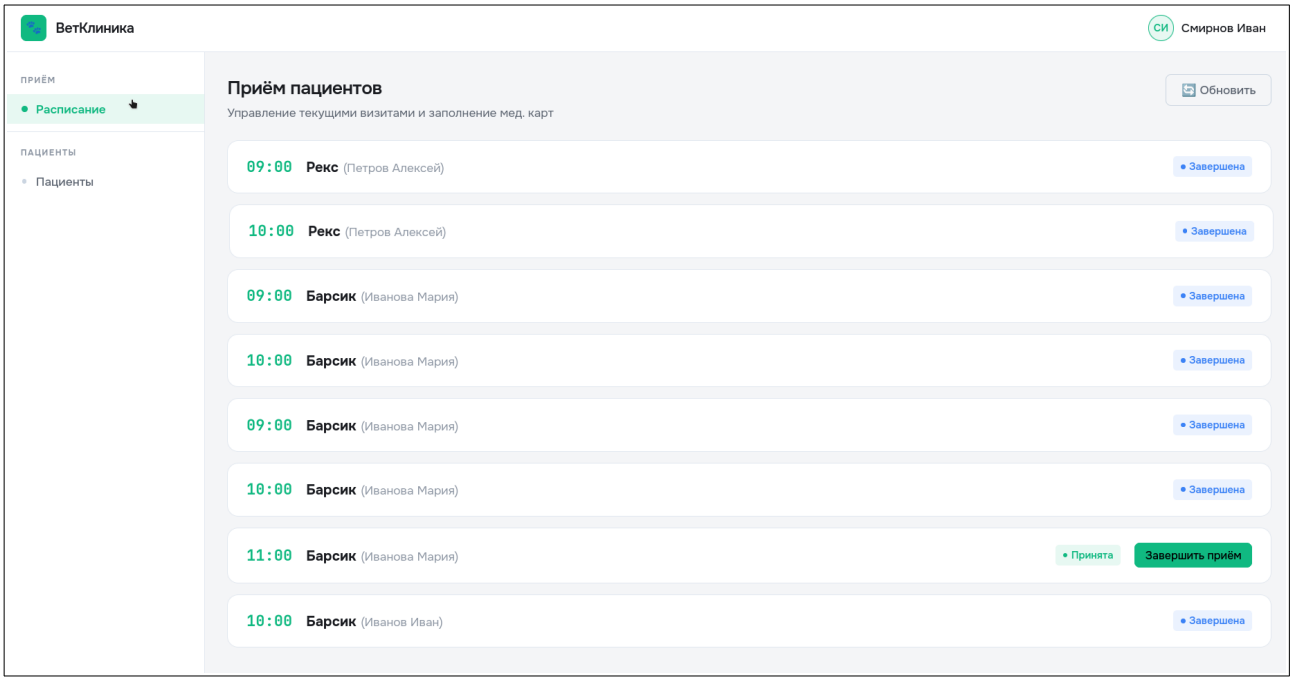


Рисунок 2.3.10 – Страница расписания врача

Завершение приёма. Для подтверждённой записи доступна кнопка «Завершить приём», по нажатию на которую открывается форма медицинского протокола. В форме необходимо указать вес питомца, добавить один или несколько диагнозов с помощью кнопки «+» (выбор из справочника диагнозов), заполнить поля «Рекомендации и лечение» и «Назначенные препараты», после чего нажать кнопку «Подтвердить и закрыть визит». Форма завершения приёма представлена на рисунке 2.3.11.

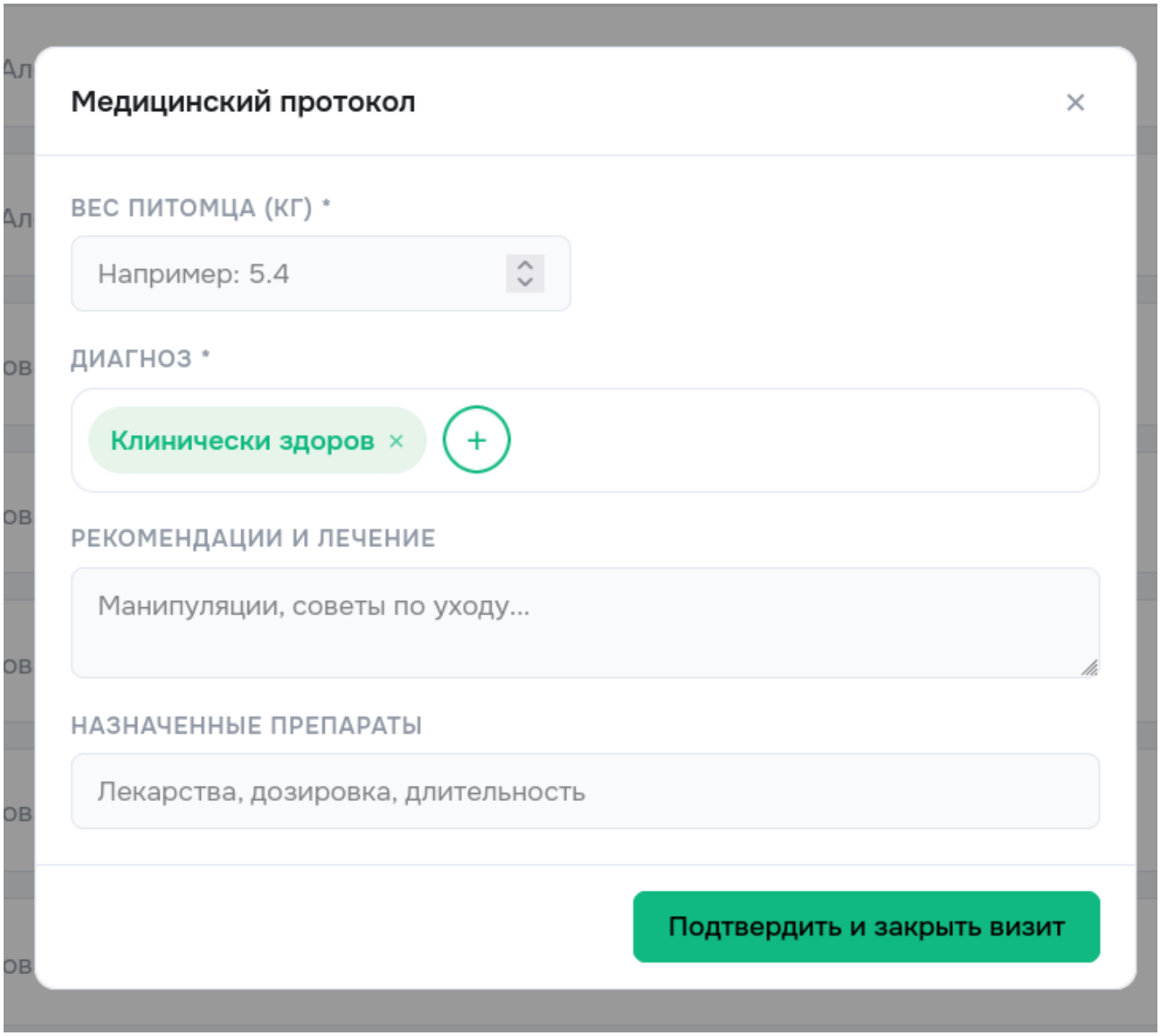


Рисунок 2.3.11 – Форма завершения приёма

Картотека пациентов. Для поиска клиента необходимо перейти в раздел «Картотека пациентов» и ввести в поле поиска фамилию или номер телефона. Для просмотра медицинской истории клиента следует нажать кнопку «Медкарта» на строке найденного клиента — откроется история визитов по всем его питомцам с указанием диагнозов и лечения. Страница картотеки пациентов представлена на рисунке 2.3.12.

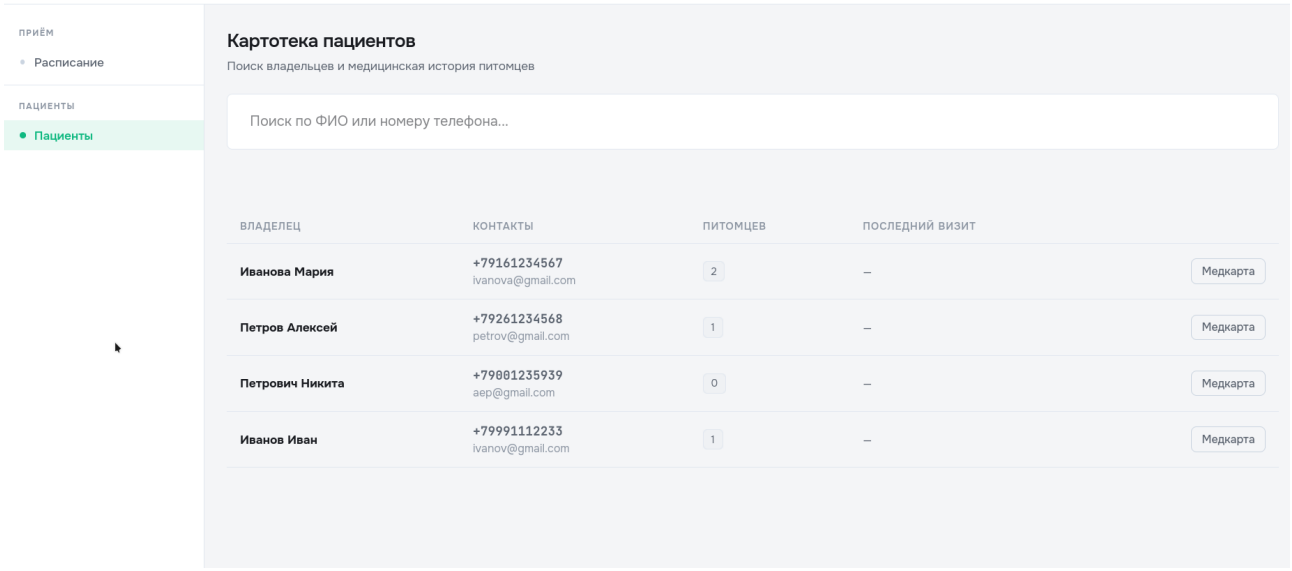


Рисунок 2.3.12 – Страница «Картотека пациентов»

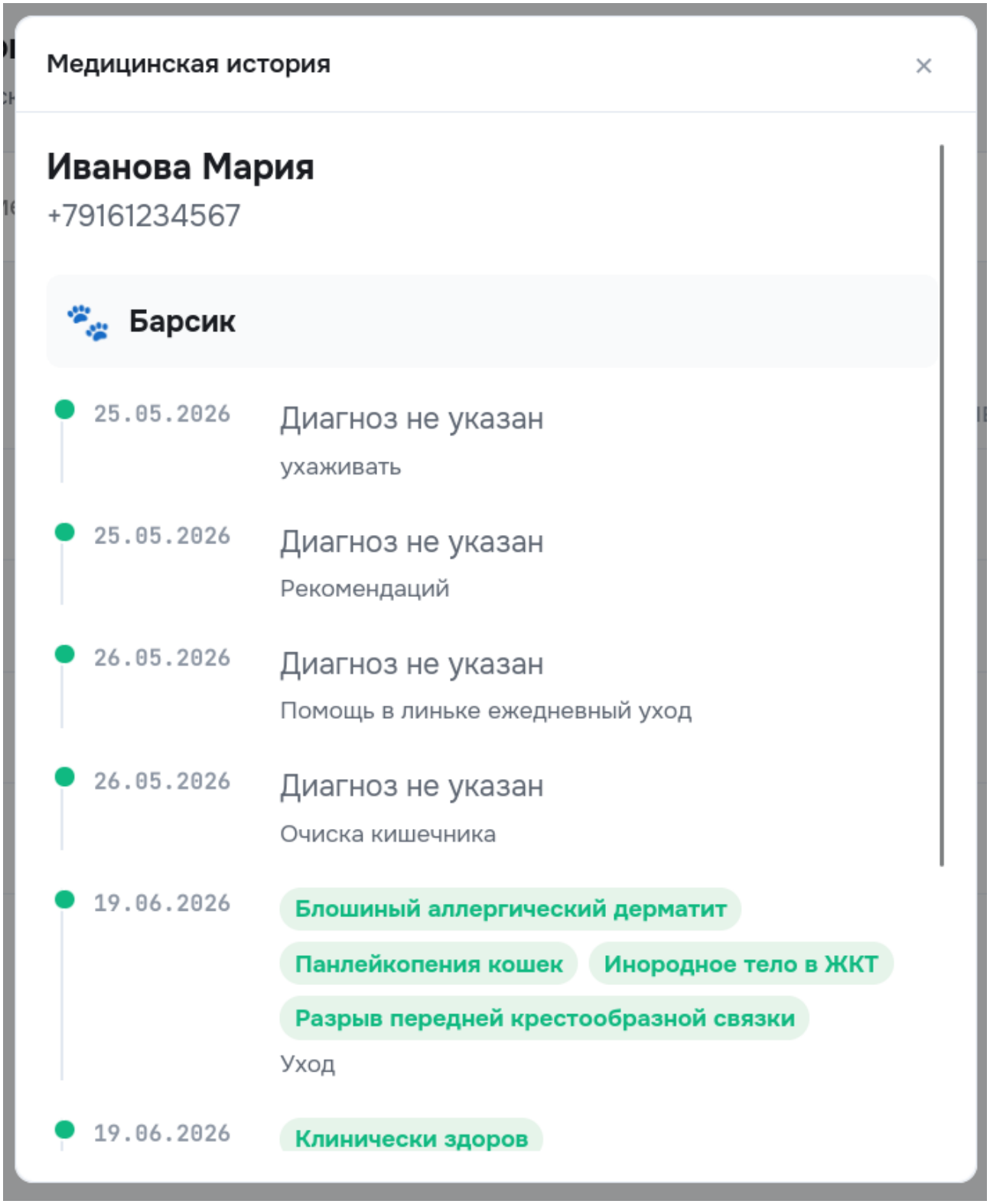


Рисунок 2.3.13 – Медицинская история клиента

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана информационная система для учёта посещений в ветеринарной клинике, реализующая автоматизацию основных процессов взаимодействия клиентов и врачей клиники.

В рамках выполнения проекта были решены следующие задачи:

- спроектирована структура базы данных, обеспечивающая хранение информации о клиентах, питомцах, врачах, услугах, записях на приём, диагнозах и медицинских протоколах;
- разработана архитектура серверного приложения на языке программирования Go с использованием фреймворка Gin и библиотеки GORM, построенная по слоистому принципу с разделением на модели, репозитории, сервисы и обработчики запросов;
- реализована система регистрации и авторизации пользователей с разделением ролей (клиент, врач) на основе JWT-токенов;
- реализован функционал записи клиента на приём с выбором питомца, врача, услуги и времени, с проверкой доступности времени врача;
- реализован функционал завершения приёма врачом с фиксацией веса питомца, диагнозов, лечения и медикаментов, а также справочник диагнозов с возможностью выбора нескольких диагнозов для одного приёма;
- реализован функционал ведения карточек питомцев, просмотра истории посещений с экспортом медицинского протокола в PDF, а также статистики динамики веса питомца;
- реализован функционал поиска пациентов и просмотра медицинской истории клиента для врача;
- реализован интернет-магазин товаров для животных с функциями корзины и оформления заказов;
- разработано клиентское веб-приложение на языке программирования TypeScript с использованием фреймворка Vue 3, реализующее два рабочих пространства — для клиента и для врача клиники;

- проведено тестирование разработанной системы на корректных и некорректных данных, подтвердившее её работоспособность.

Разработанная информационная система решает поставленные задачи по автоматизации учёта посещений ветеринарной клиники, обеспечивая удобное взаимодействие между клиентами и врачами, централизованное хранение медицинских данных питомцев и сокращение времени на ведение документации.

В качестве направлений дальнейшего развития системы можно выделить: добавление push-уведомлений о предстоящих приёмах, реализацию онлайн-оплаты услуг и товаров, добавление ролей администратора и регистратора клиники, а также разработку мобильного приложения для платформ Android и iOS.

ПРИЛОЖЕНИЕ А
Контрольный пример

Таблица А.1 – Входные данные для таблицы «users» (Пользователи)

Электронная почта	Пароль	Имя	Фамилия	Номер телефона	Роль	Специализация
ivanova@gmail.com	12345678	Мария	Иванова	+79161234567	client	—
petrov@gmail.comz	12345678	Алексей	Петров	+79261234568	client	—
doktor1@vetclinic.ru	12345678	Иван	Смирнов	+79301234569	doctor	Терапевт
doktor2@vetclinic.ru	12345678	Ольга	Козлова	+79401234570	doctor	Хирург

Таблица А.2 – Входные данные для таблицы «pets» (Питомцы)

Идентификатор питомца	Идентификатор владельца	Имя питомца	Вид животного	Порода	Дата рождения	Вес
1	1	Барсик	Кот	Британская	2021-05-	4.2

Идентификатор питомца	Идентификатор владельца	Имя питомца	Вид животного	Порода	Дата рождения	Вес
					12	
2	1	Шарик	Собака	Дворняга	2019-03-01	12.5
3	2	Мурка	Кот	Сиамская	2022-08-20	3.8

Таблица А.3 – Входные данные для таблицы «services» (Услуги)

Идентификатор услуги	Название услуги	Стоимость
1	Первичный полный осмотр и консультация	1200
2	Профилактический осмотр перед вакцинацией	600
3	Стрижка когтей (кошки и мелкие собаки)	450

Таблица А.4 – Входные данные для таблицы «diagnoses» (Диагнозы)

Идентификатор диагноза	Название диагноза
1	Клинически здоров
2	Острый гастроэнтерит
3	Мочекаменная болезнь (МКБ)

Таблица А.5 – Входные данные для таблицы «appointments» (Записи на приём)

Идентификатор клиента	Идентификатор питомца	Идентификатор врача	Идентификатор услуги	Дата и время приёма	Статус записи
1	1	3	1	2026-06-22 10:00:00	confirmed
2	3	4	2	2026-06-22 11:30:00	waiting

Таблица А.6 – Выходные данные при формировании медицинского заключения

Питомец	Дата и время	Диагнозы	Лечение	Медикаменты	Вес
Барсик	22.06.2026 10:00	Клинически здоров	Промывание ушной раковины	Капли ушные	4.3
Мурка	22.06.2026 11:30	Острый гастроэнтерит	Диета, антигистаминные препараты	Супрастин	3.9

ПРИЛОЖЕНИЕ Б

Исходный код программного продукта

Полный исходный код проекта содержит более 30 файлов и доступен в полном объёме на электронном носителе, приложенном к курсовому проекту. В настоящем приложении представлены ключевые листинги, демонстрирующие архитектуру и основной функционал системы, сгруппированные по модулям в соответствии со структурой проекта, представленной на рисунках 2.1.1 и 2.1.2.

Серверное приложение (Go API)

main.go

```
package main


import (
	"api/internal/handler"
	"api/internal/middleware"
	"api/internal/repository"
	"api/internal/service"

	_ "api/docs"
```

```
	swaggerFiles "github.com/swaggo/files"
	ginSwagger "github.com/swaggo/gin-swagger"

	database "api/utils"

	"github.com/gin-gonic/gin"
)

// @title Veterinary Clinic API
// @version 1.0
// @description API server for Veterinary Clinic management system.
// @host localhost:3000
// @BasePath /api
// @securityDefinitions.apikey ApiKeyAuth
// @in header
// @name Authorization
func main() {
	db := database.InitDB()
```

```
userRepo := repository.NewUserRepository(db)
authSvc := service.NewAuthService(userRepo)
authHandler := handler.NewAuthHandler(authSvc)

petRepo := repository.NewPetRepository(db)
petSvc := service.NewPetService(petRepo)
petHandler := handler.NewPetHandler(petSvc)

dashRepo := repository.NewDashboardRepository(db)
dashSvc := service.NewDashboardService(dashRepo, petRepo)
dashHandler := handler.NewDashboardHandler(dashSvc)

appRepo := repository.NewAppointmentRepository(db)
bookingSvc := service.NewBookingService(appRepo, petRepo)
bookingHandler := handler.NewBookingHandler(bookingSvc)

shopRepo := repository.NewShopRepository(db)
shopSvc := service.NewShopService(shopRepo)
shopHandler := handler.NewShopHandler(shopSvc)
```

```
statsRepo := repository.NewStatsRepository(db)

statsSvc := service.NewStatsService(statsRepo)

statsHandler := handler.NewStatsHandler(statsSvc, petSvc)


docRepo := repository.NewDoctorRepository(db)

docSvc := service.NewDoctorService(docRepo)

docHandler := handler.NewDoctorHandler(docSvc)


diagRepo := repository.NewDiagnosisRepository(db)

diagSvc := service.NewDiagnosisService(diagRepo)

diagHandler := handler.NewDiagnosisHandler(diagSvc)


userSvc := service.NewUserService(userRepo)

userHandler := handler.NewUserHandler(userSvc)


r := gin.Default()

r.GET("/swagger/*any", ginSwagger.WrapHandler(swaggerFiles.Handler))


api := r.Group("/api")

{
```

```
        api.POST("/auth/register", authHandler.Register)
        api.POST("/auth/login", authHandler.Login)

        protected := api.Group("")
        protected.Use(middleware.AuthMiddleware())
        {
            protected.GET("/users/:id", userHandler.GetProfile)
            protected.PUT("/users/:id", userHandler.UpdateProfile)
            protected.PUT("/users/:id/password",
userHandler.ChangePassword)

            protected.GET("/pets/owner/:userId", petHandler.GetByOwner)
            protected.POST("/pets", petHandler.Create)
            protected.PUT("/pets/:petId", petHandler.Update)
            protected.DELETE("/pets/:petId", petHandler.Delete)

            protected.GET("/dashboard/:userId", dashHandler.GetDashboard)

            protected.GET("/book/init/:userId", bookingHandler.GetInit)
            protected.GET("/appointments/busy-slots",
```

```
bookingHandler.GetBusySlots)

        protected.POST("/appointments", bookingHandler.Create)

        protected.GET("/appointments/client/:userId",
bookingHandler.GetHistory)



        protected.GET("/doctor/schedule", docHandler.GetSchedule)

        protected.PATCH("/appointments/:id/complete",
docHandler.CompleteVisit)

        protected.PATCH("/appointments/:id/status",
docHandler.UpdateStatus)

        protected.GET("/doctor/patients", docHandler.GetPatients)

        protected.GET("/doctor/patients/:id/history",
docHandler.GetHistory)

        protected.GET("/diagnoses", diagHandler.GetAll)



        protected.GET("/medications", shopHandler.GetProducts)

        protected.GET("/cart/:userId", shopHandler.GetCart)

        protected.POST("/cart/:userId", shopHandler.AddToCart)

        protected.PUT("/cart/:itemId", shopHandler.UpdateCart)

        protected.DELETE("/cart/:itemId", shopHandler.DeleteCart)

        protected.POST("/orders", shopHandler.Checkout)
```

```
                protected.GET("/orders/:userId", shopHandler.GetOrders)


                protected.GET("/stats/pets/:userId", statsHandler.GetUserPets)

                protected.GET("/stats/weight/:petId",
statsHandler.GetWeightData)

            }

        }


        r.Run(":3000")

}
```

internal/models

user.go

```
package models

import "gorm.io/gorm"

type UserRole string

const (
        UserClient UserRole = "client"
        UserDoctor UserRole = "doctor"
)

type User struct {
        gorm.Model
```

```
	Email          string   `gorm:"type:varchar(191);uniqueIndex" json:"email"`
	Password       string   `json:"-"`
	FirstName      string   `json:"firstName"`
	LastName       string   `json:"lastName"`
	Phone          string   `gorm:"type:varchar(191);unique" json:"phone"`
	Role           UserRole `gorm:"type:enum('client','doctor')" json:"role"`
	Specialization string   `json:"specialization,omitempty"`
	Pets           []Pet    `gorm:"foreignKey:OwnerID" json:"pets,omitempty"`
}
```

models/pet.go

```
package models

import (
	"time"

	"gorm.io/gorm"
)

type Pet struct {
	gorm.Model
	OwnerID      uint          `json:"ownerId"`
	Name         string        `json:"name"`
	Species      string        `json:"species"`
	Breed        string        `json:"breed"`
	BirthDate    time.Time     `json:"dob"`
	Weight       float64       `json:"weight"`
	Avatar       string        `json:"avatar"`
	Appointments []Appointment `gorm:"foreignKey:PetID"
json:"appointments,omitempty"`
}
```

models/appointment.go

```
package models

import (
	"time"

	"gorm.io/gorm"
)

type StatusAppointment string

const (
	StatusWaiting   StatusAppointment = "waiting"
	StatusConfirmed StatusAppointment = "confirmed"
	StatusRejected  StatusAppointment = "rejected"
	StatusDone      StatusAppointment = "done"
)

type Appointment struct {
	gorm.Model

	ClientID    uint            `json:"clientId"`
	PetID       uint            `json:"petId"`
	DoctorID    uint            `json:"doctorId"`
	ServiceID   *uint           `json:"serviceId"`
	ScheduledAt time.Time       `json:"scheduledAt"`
	Status                                StatusAppointment        `json:"status"
gorm:"type:varchar(20);default:'waiting'"`
	Comment     string          `json:"comment"`
	Pet         Pet             `gorm:"foreignKey:PetID" json:"pet"`
	Client      User            `gorm:"foreignKey:ClientID" json:"client"`
	Doctor      User            `gorm:"foreignKey:DoctorID" json:"doctor"`
```

```
	Service     Service          `gorm:"foreignKey:ServiceID" json:"service"`
	Diagnoses   []Diagnosis      `gorm:"many2many:appointment_diagnoses;"
json:"diagnoses"`
	Protocol    *MedicalProtocol `gorm:"foreignKey:AppointmentID"
json:"protocol"`
}
```

models/diagnos.go

```
package models

import "gorm.io/gorm"

type Diagnosis struct {
	gorm.Model
	Name string `json:"name" gorm:"not null"`
}
```

models/medical_protocol.go

```
package models

import "gorm.io/gorm"

type MedicalProtocol struct {
	gorm.Model
	AppointmentID uint    `gorm:"uniqueIndex" json:"appointmentId"`
	WeightAtVisit float64 `json:"weight"`
	Treatment     string  `json:"treatment"`
	Medications   string  `json:"medications"`
}
```

models/service.go

```
package models

import "gorm.io/gorm"
```

```
type Service struct {
	gorm.Model
	Name  string `json:"name"`
	Price uint   `json:"price"`
}
```

models/product.go

```
package models

import "gorm.io/gorm"

type Product struct {
	gorm.Model
	Name        string `json:"name"`
	Description string `json:"description"`
	Price       int    `json:"price"`
	Category    string `json:"category"`
	Stock       int    `json:"stock"`
}
```

models/cart_item.go

```
package models

import "gorm.io/gorm"

type CartItem struct {
	gorm.Model
	UserID    uint    `json:"userId"`
	ProductID uint    `json:"productId"`
	Quantity  uint    `json:"quantity"`
	Product   Product `gorm:"foreignKey:ProductID" json:"product"`
}
```

models/order.go

```
package models

import "gorm.io/gorm"

type OrderStatus string

const (
	OrderPaid      OrderStatus = "paid"
	OrderConfirmed OrderStatus = "confirmed"
	OrderDelivered OrderStatus = "delivered"
)

type OrderItem struct {
	gorm.Model
	OrderID   uint    `json:"orderId"`
	ProductID uint    `json:"productId"`
	Product   Product `gorm:"foreignKey:ProductID" json:"product"`
	Quantity  uint    `json:"quantity"`
	Price     int     `json:"price"`
}

type Order struct {
	gorm.Model
	UserID      uint        `json:"userId"`
	TotalAmount uint        `json:"totalAmount"`
	Status      OrderStatus `gorm:"type:enum('paid','confirmed','delivered')"
json:"status"`
	Items       []OrderItem `gorm:"foreignKey:OrderID" json:"items"`
}
```

internal/repository

repository/appointment_repo.go

```
package repository

import ( "api/internal/models" "time"

"gorm.io/gorm"
)

type AppointmentRepository interface { Create(app *models.Appointment)
error GetBusySlots(doctorID uint, date string) ([]string,
error) GetByClientID(clientID uint) ([]models.Appointment, error) GetDoctors()
([]models.User, error) GetServices() ([]models.Service,
error) SetDiagnoses(appointmentID uint, diagnosisIDs []uint) error }

type appointmentRepo struct { db *gorm.DB }

func NewAppointmentRepository(db *gorm.DB) AppointmentRepository { return
&appointmentRepo{db} }

func (r *appointmentRepo) Create(app *models.Appointment) error { return
r.db.Create(app).Error }

func (r *appointmentRepo) GetBusySlots(doctorID uint, date string) ([]string, error)
{ times := make([]time.Time, 0) err :=
r.db.Model(&models.Appointment{}). Where("doctor_id = ? AND
DATE(scheduled_at) = ? AND status != ?", doctorID, date,
"rejected"). Pluck("scheduled_at", &times).Error

slots := make([]string, 0)
for _, t := range times {
   slots = append(slots, t.Format("15:04"))
}
return slots, err
}

func (r *appointmentRepo) GetByClientID(clientID uint) ([]models.Appointment,
error) { apps := []models.Appointment{}

err := r.db.Preload("Pet").
   Preload("Protocol").
   Preload("Doctor").
   Preload("Service").
   Preload("Diagnoses").
   Where("client_id = ?", clientID).
   Order("scheduled_at desc").
   Find(&apps) Error
```

```
return apps, err
}

func (r *appointmentRepo) GetDoctors() ([]models.User, error) { doctors :=
make([]models.User, 0) err := r.db.Where("role = ?",
"doctor").Find(&doctors).Error return doctors, err }

func (r *appointmentRepo) GetServices() ([]models.Service, error) { services :=
make([]models.Service, 0) err := r.db.Find(&services).Error return services, err }

func (r *appointmentRepo) SetDiagnoses(appointmentID uint, diagnosisIDs []uint)
error { var appointment models.Appointment

if err := r.db.First(&appointment, appointmentID).Error; err != nil {
   return err
}

var diagnoses []models.Diagnosis

if err := r.db.Where("id IN ?", diagnosisIDs).Find(&diagnoses).Error; err != nil {
   return err
}

return r.db.Model(&appointment).
   Association("Diagnoses").
   Replace(diagnoses)
}
```

repository/diagnosis_repo.go

```
package repository



import (

        "api/internal/models"



        "gorm.io/gorm"

)
```

```
type DiagnosisRepository interface {
	GetAll() ([]models.Diagnosis, error)
	SetDiagnoses(appointment *models.Appointment, diagnoses
[]models.Diagnosis) error
}

type diagnosisRepo struct {
	db *gorm.DB
}

func NewDiagnosisRepository(db *gorm.DB) DiagnosisRepository {
	return &diagnosisRepo{
		db: db,
	}
}

func (r *diagnosisRepo) GetAll() ([]models.Diagnosis, error) {
	diagnoses := make([]models.Diagnosis, 0)

	err := r.db.Find(&diagnoses).Error

	return diagnoses, err
}

func (r *diagnosisRepo) SetDiagnoses(
```

```
        appointment *models.Appointment,
        diagnoses []models.Diagnosis,
) error {
        return r.db.Model(appointment).
                Association("Diagnoses").
                Replace(diagnoses)
}
```

repository/doctor_repo.go

```
package repository
import ( "api/internal/models"
"gorm.io/gorm"
)
type DoctorRepository interface { GetSchedule(doctorID uint)
([]models.Appointment, error) CompleteAppointment(appID uint, protocol
*models.MedicalProtocol) error SearchClients(query string) ([]models.User,
error) GetClientHistory(clientID uint) ([]models.Pet,
error) UpdateAppointmentStatus(appID uint, status string) error SetDiagnoses(appID
uint, diagnosisIDs []uint) error }
type doctorRepo struct { db *gorm.DB }
func NewDoctorRepository(db *gorm.DB) DoctorRepository { return
&doctorRepo{db} }
func (r *doctorRepo) GetSchedule(doctorID uint) ([]models.Appointment, error) { //
Добавлен error apps := make([]models.Appointment, 0) err :=
r.db.Preload("Pet").Preload("Client").Preload("Service"). Where("doctor_id = ? AND
status != ?", doctorID, "rejected"). Order("scheduled_at asc").Find(&apps).Error
return apps, err
}
func (r *doctorRepo) CompleteAppointment(appID uint, protocol
*models.MedicalProtocol) error { return r.db.Transaction(func(tx *gorm.DB) error
{ if err := tx.Create(protocol).Error; err != nil { return err } return
tx.Model(&models.Appointment{}).Where("id = ?", appID).Update("status",
"done").Error }) }
```

```
func (r *doctorRepo) SearchClients(query string) ([]models.User, error) { users :=
make([]models.User, 0) dbQuery := r.db.Preload("Pets").Where("role = ?",
"client") if query != "" { q := "%" + query + "%" dbQuery =
dbQuery.Where("first_name LIKE ? OR last_name LIKE ? OR phone LIKE ?", q, q,
q) } err := dbQuery.Find(&users).Error return users, err }

func (r *doctorRepo) GetClientHistory(clientID uint) ([]models.Pet, error) { pets :=
make([]models.Pet, 0) err := r.db.Preload("Appointments", "status = ?",
"done"). Preload("Appointments.Protocol"). Preload("Appointments.Diagnoses"). W
here("owner_id = ?", clientID).Find(&pets).Error return pets, err }

func (r *doctorRepo) UpdateAppointmentStatus(appID uint, status string) error
{ return r.db.Model(&models.Appointment{}).Where("id = ?",
appID).Update("status", status).Error }

func (r *doctorRepo) SetDiagnoses(appID uint, diagnosisIDs []uint) error { var
appointment models.Appointment

if err := r.db.First(&appointment, appID).Error; err != nil {
   return err
}

var diagnoses []models.Diagnosis

if err := r.db.Where("id IN ?", diagnosisIDs).
   Find(&diagnoses).Error; err != nil {
   return err
}

return r.db.Model(&appointment).
   Association("Diagnoses").
   Replace(diagnoses)
}
```

internal/service

service/booking_service.go

```
package service

import ( "api/internal/dto" "api/internal/models" "api/internal/repository" "time" )

type BookingService interface { GetInitData(userID uint)
(*dto.BookingInitResponse, error) GetBusySlots(doctorID uint, date string) ([]string,
error) CreateAppointment(clientID uint, req dto.CreateAppointmentRequest)
```

```
error GetClientHistory(userID uint) ([]dto.RecentAppointmentDTO, error) }

type bookingService struct { repo    repository.AppointmentRepository petRepo
repository.PetRepository }

func NewBookingService(repo repository.AppointmentRepository, petRepo
repository.PetRepository) BookingService { return &bookingService{repo,
petRepo} }

func (s *bookingService) GetInitData(userID uint) (*dto.BookingInitResponse, error)
{ pets, _ := s.petRepo.GetByOwnerID(userID) doctors, _ :=
s.repo.GetDoctors() services, _ := s.repo.GetServices()

res := &dto.BookingInitResponse{}
for _, p := range pets {
   res.Pets = append(res.Pets, dto.PetBriefDTO{ID: p.ID, Name: p.Name, Avatar:
p.Avatar})
}
for _, d := range doctors {
   res.Doctors = append(res.Doctors, dto.DoctorDTO{ID: d.ID, FirstName:
d.FirstName, LastName: d.LastName, Specialization: d.Specialization})
}
for _, sv := range services {
   res.Services = append(res.Services, dto.ServiceDTO{ID: sv.ID, Name: sv.Name,
Price: sv.Price})
}
return res, nil
}

func (s *bookingService) GetBusySlots(doctorID uint, date string) ([]string, error)
{ return s.repo.GetBusySlots(doctorID, date) }

func (s *bookingService) CreateAppointment(clientID uint, req
dto.CreateAppointmentRequest) error { scheduledAt, err :=
time.ParseInLocation("2006-01-02T15:04:05", req.ScheduledAt, time.Local) if err !=
nil { return err }

app := models.Appointment{
   ClientID:    clientID,
   PetID:       req.PetID,
   DoctorID:    req.DoctorID,
   ServiceID:   req.ServiceID,
   ScheduledAt: scheduledAt,
   Status:      models.StatusWaiting,
   Comment:     req.Comment,
}
return s.repo.Create(&app)
```

```
}

func (s *bookingService) GetClientHistory(userID uint)
([]dto.RecentAppointmentDTO, error) { apps, err := s.repo.GetByClientID(userID) if
err != nil { return nil, err }

var history []dto.RecentAppointmentDTO
for _, a := range apps {
   item := dto.RecentAppointmentDTO{
      ID:         a.ID,
      PetName:    a.Pet.Name,
      DoctorName: a.Doctor.LastName + " " + a.Doctor.FirstName,
      Date:       a.ScheduledAt.Format("02.01.2006"),
      Time:       a.ScheduledAt.Format("15:04"),
      Status:     string(a.Status),
      Service:    a.Service.Name,
   }

   if a.Protocol != nil {
      diagnoses := make([]dto.DiagnosisDTO, 0, len(a.Diagnoses))
      for _, d := range a.Diagnoses {
         diagnoses = append(diagnoses, dto.DiagnosisDTO{ID: d.ID, Name:
d.Name})
      }

      item.Protocol = &dto.MedicalProtocolDTO{
         Weight:      a.Protocol.WeightAtVisit,
         Diagnoses:   diagnoses,
         Treatment:   a.Protocol.Treatment,
         Medications: a.Protocol.Medications,
      }
   }

   history = append(history, item)
}
return history, nil
}
```

service/diagnosis_service.go

```
package service



import (

        "api/internal/dto"
```

```
	"api/internal/repository"
)

type DiagnosisService interface {
	GetAll() ([]dto.DiagnosisDTO, error)
}

type diagnosisService struct {
	repo repository.DiagnosisRepository
}

func NewDiagnosisService(repo repository.DiagnosisRepository) DiagnosisService {
	return &diagnosisService{repo}
}

func (s *diagnosisService) GetAll() ([]dto.DiagnosisDTO, error) {
	diagnoses, err := s.repo.GetAll()
	if err != nil {
		return nil, err
	}

	res := make([]dto.DiagnosisDTO, 0, len(diagnoses))
	for _, d := range diagnoses {
		res = append(res, dto.DiagnosisDTO{ID: d.ID, Name: d.Name})
	}
```

```
	return res, nil
}
```

service/doctor_service.go

```
package service

import (
	"api/internal/dto"
	"api/internal/models"
	"api/internal/repository"
)

type DoctorService interface {
	GetSchedule(doctorID uint) []dto.DoctorScheduleDTO
	CompleteVisit(appID uint, req dto.CompleteVisitRequest) error
	GetPatients(search string) []dto.PatientDTO
	GetMedicalHistory(clientID uint) []dto.PetHistoryDTO
	UpdateStatus(appID uint, status string) error
}

type doctorService struct {
	repo repository.DoctorRepository
}

func NewDoctorService(repo repository.DoctorRepository) DoctorService {
	return &doctorService{repo}
```

```
}

func (s *doctorService) GetSchedule(doctorID uint) []dto.DoctorScheduleDTO {
	apps, err := s.repo.GetSchedule(doctorID)
	if err != nil {
		return []dto.DoctorScheduleDTO{}
	}

	res := make([]dto.DoctorScheduleDTO, 0)
	for _, a := range apps {
		res = append(res, dto.DoctorScheduleDTO{
			ID:        a.ID,
			Time:      a.ScheduledAt.Format("15:04"),
			Date:      a.ScheduledAt.Format("2006-01-02"),
			PetName:   a.Pet.Name,
			OwnerName: a.Client.LastName + " " + a.Client.FirstName,
			Service:   a.Service.Name,
			Status:    string(a.Status),
		})
	}
	return res
}

func (s *doctorService) CompleteVisit(appID uint, req dto.CompleteVisitRequest)
error {
```

```
	protocol := &models.MedicalProtocol{
		AppointmentID: appID,
		WeightAtVisit: req.Weight,
		Treatment:     req.Treatment,
		Medications:   req.Medications,
	}

	if err := s.repo.CompleteAppointment(appID, protocol); err != nil {
		return err
	}

	return s.repo.SetDiagnoses(appID, req.DiagnosisIDs)
}

func (s *doctorService) GetPatients(search string) []dto.PatientDTO {
	clients, _ := s.repo.SearchClients(search)
	var res []dto.PatientDTO
	for _, c := range clients {
		res = append(res, dto.PatientDTO{
			ID:        c.ID,
			FullName:  c.LastName + " " + c.FirstName,
			Phone:     c.Phone,
			Email:     c.Email,
			PetsCount: len(c.Pets),
		})
```

```
	}
	return res
}

func (s *doctorService) GetMedicalHistory(clientID uint) []dto.PetHistoryDTO {
	pets, _ := s.repo.GetClientHistory(clientID)
	res := make([]dto.PetHistoryDTO, 0)
	for _, p := range pets {
		var visits []dto.VisitTimelineDTO
		for _, a := range p.Appointments {
			if a.Protocol != nil {
				diagnoses := make([]dto.DiagnosisDTO, 0,
len(a.Diagnoses))
				for _, d := range a.Diagnoses {
					diagnoses = append(diagnoses,
dto.DiagnosisDTO{ID: d.ID, Name: d.Name})
				}

				visits = append(visits, dto.VisitTimelineDTO{
					ID:        a.ID,
					Date:      a.ScheduledAt.Format("02.01.2006"),
					Diagnoses: diagnoses,
					Treatment: a.Protocol.Treatment,
				})
			}
		}
```

```
                res = append(res, dto.PetHistoryDTO{PetName: p.Name, PetIcon:
p.Avatar, Visits: visits})

        }

        return res

}


func (s *doctorService) UpdateStatus(appID uint, status string) error {

        return s.repo.UpdateAppointmentStatus(appID, status)

}
```

internal/dto

dto/booking_dto.go

```
package dto


type BookingInitResponse struct {

        Pets     []PetBriefDTO `json:"pets"`

        Doctors  []DoctorDTO   `json:"doctors"`

        Services []ServiceDTO  `json:"services"`

}


type DoctorDTO struct {

        ID           uint   `json:"doctorId"`

        FirstName      string `json:"firstName"`
```

```
	LastName	string `json:"lastName"`
	Specialization string `json:"specialization"`
}

type ServiceDTO struct {
	ID	uint	`json:"id"`
	Name	string `json:"name"`
	Price uint	`json:"price"`
}

type CreateAppointmentRequest struct {
	PetID	uint	`json:"pet_id" binding:"required"`
	DoctorID	uint	`json:"doctor_id" binding:"required"`
	ServiceID	*uint	`json:"service_id"`
	ScheduledAt	string	`json:"scheduled_at"	binding:"required"`	//	"2024-05-20T09:00:00"
	Comment	string `json:"comment"`
}
```

dto/doctor_dto.go

```
package dto
```

```
type DoctorScheduleDTO struct {
        ID        uint   `json:"id"`
        Time      string `json:"time"`
        Date      string `json:"date"`
        PetName   string `json:"petName"`
        OwnerName string `json:"ownerName"`
        Service   string `json:"service"`
        Status    string `json:"status"`
}

type CompleteVisitRequest struct {
        Weight       float64 `json:"weight" binding:"required"`
        DiagnosisIDs []uint  `json:"diagnosis_ids" binding:"required"`
        Treatment    string  `json:"treatment"`
        Medications  string  `json:"medications"`
}

type PatientDTO struct {
        ID        uint   `json:"id"`
```

```
	FullName  string `json:"fullName"`
	Phone     string `json:"phone"`
	Email     string `json:"email"`
	PetsCount int    `json:"petsCount"`
	LastVisit string `json:"lastVisit"`
}

type PetHistoryDTO struct {
	PetName string            `json:"petName"`
	PetIcon string            `json:"petIcon"`
	Visits  []VisitTimelineDTO `json:"visits"`
}

type VisitTimelineDTO struct {
	ID        uint          `json:"id"`
	Date      string        `json:"date"`
	Diagnoses []DiagnosisDTO `json:"diagnoses"`
	Treatment string        `json:"treatment"`
}
```

dto/diagnosis_dto.go

```
package dto


type DiagnosisDTO struct {
	ID   uint   `json:"id"`
	Name string `json:"name"`
}
```

internal/handler

handler/booking_handler.go

```
package handler


import (
	"api/internal/dto"
	"api/internal/service"
	"net/http"
	"strconv"

	"github.com/gin-gonic/gin"
)


type BookingHandler struct {
```

```
        svc service.BookingService
}

func NewBookingHandler(svc service.BookingService) *BookingHandler {
        return &BookingHandler{svc}
}

// GetInit godoc
// @Summary      Инициализация формы записи
// @Description  Получает списки питомцев, врачей и услуг
// @Tags         booking
// @Security     ApiKeyAuth
// @Produce      json
// @Param        userId  path      int  true  "ID пользователя"
// @Success      200     {object}  dto.BookingInitResponse
// @Router       /book/init/{userId} [get]
func (h *BookingHandler) GetInit(c *gin.Context) {
        uid, _ := strconv.ParseUint(c.Param("userId"), 10, 32)
        data, _ := h.svc.GetInitData(uint(uid))
        c.JSON(http.StatusOK, data)
```

```
}


// GetBusySlots godoc
// @Summary      Получение занятых слотов
// @Description  Возвращает список времени, на которое уже есть запись у врача
// @Tags         booking
// @Security     ApiKeyAuth
// @Produce      json
// @Param        doctor_id  query     int     true  "ID врача"
// @Param        date       query     string  true  "Дата (YYYY-MM-DD)"
// @Success      200        {array}   string
// @Router       /appointments/busy-slots [get]
func (h *BookingHandler) GetBusySlots(c *gin.Context) {
        did, _ := strconv.ParseUint(c.Query("doctor_id"), 10, 32)
        date := c.Query("date") // "2024-05-20"
        slots, _ := h.svc.GetBusySlots(uint(did), date)
        c.JSON(http.StatusOK, slots)
}


// Create godoc
```

```
// @Summary      Создание записи на прием
// @Description  Бронирует визит к врачу
// @Tags         booking
// @Security     ApiKeyAuth
// @Accept       json
// @Produce      json
// @Param        request body dto.CreateAppointmentRequest true "Данные записи"
// @Success      201 {object} map[string]bool
// @Router       /appointments [post]
func (h *BookingHandler) Create(c *gin.Context) {
    var req dto.CreateAppointmentRequest
    if err := c.ShouldBindJSON(&req); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    clientID := c.GetUint("userID")

    if err := h.svc.CreateAppointment(clientID, req); err != nil {
```

```
            c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
            return
        }
        c.JSON(http.StatusCreated, gin.H{"success": true})
}


// GetHistory godoc
// @Summary     История визитов клиента
// @Tags        booking
// @Security    ApiKeyAuth
// @Produce     json
// @Param       userId  path     int  true  "ID пользователя"
// @Success     200     {array}  dto.RecentAppointmentDTO
// @Router      /appointments/client/{userId} [get]
func (h *BookingHandler) GetHistory(c *gin.Context) {
        uid, _ := strconv.ParseUint(c.Param("userId"), 10, 32)
        history, err := h.svc.GetClientHistory(uint(uid))
        if err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
            return
```

```
        }
        c.JSON(http.StatusOK, history)
    }
```

handler/doctor_handler.go

```
package handler

import (
        "api/internal/dto"
        "api/internal/service"
        "net/http"
        "strconv"

        "github.com/gin-gonic/gin"
)

type DoctorHandler struct {
        svc service.DoctorService
}

func NewDoctorHandler(svc service.DoctorService) *DoctorHandler {
```

```
	return &DoctorHandler{svc}
}

// GetSchedule godoc
// @Summary      Расписание врача
// @Description  Список всех записей к конкретному врачу
// @Tags         doctor
// @Security     ApiKeyAuth
// @Produce      json
// @Param        doctor_id  query     int  true  "ID врача"
// @Success      200        {array}   dto.DoctorScheduleDTO
// @Router       /doctor/schedule [get]
func (h *DoctorHandler) GetSchedule(c *gin.Context) {
	did, _ := strconv.ParseUint(c.Query("doctor_id"), 10, 32)
	c.JSON(http.StatusOK, h.svc.GetSchedule(uint(did)))
}

// CompleteVisit godoc
// @Summary      Завершение приема
// @Description  Закрывает визит и сохраняет медицинский протокол (диагноз,
```

```
вес, лечение)
// @Tags        doctor
// @Security    ApiKeyAuth
// @Accept      json
// @Produce     json
// @Param       id      path    int                     true  "ID записи"
// @Param       request body    dto.CompleteVisitRequest true "Данные осмотра"
// @Success     200     {object} map[string]bool
// @Router      /appointments/{id}/complete [patch]
func (h *DoctorHandler) CompleteVisit(c *gin.Context) {
        aid, _ := strconv.ParseUint(c.Param("id"), 10, 32)
        var req dto.CompleteVisitRequest
        if err := c.ShouldBindJSON(&req); err != nil {
                c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
                return
        }
        h.svc.CompleteVisit(uint(aid), req)
        c.JSON(http.StatusOK, gin.H{"success": true})
}
```

```
// GetPatients godoc
// @Summary      Search patients
// @Description  Search for clients by name or phone
// @Tags         doctor
// @Security     ApiKeyAuth
// @Produce      json
// @Param        search  query     string  false  "Search query"
// @Success      200     {array}   dto.PatientDTO
// @Router       /doctor/patients [get]
func (h *DoctorHandler) GetPatients(c *gin.Context) {
        search := c.Query("search")
        c.JSON(http.StatusOK, h.svc.GetPatients(search))
}


// GetHistory godoc
// @Summary      Медицинская история клиента
// @Description  Получает историю всех визитов всех питомцев конкретного клиента
// @Tags         doctor
// @Security     ApiKeyAuth
```

```
// @Produce      json
// @Param        id   path      int  true  "ID клиента"
// @Success      200  {array}   dto.PetHistoryDTO
// @Router       /doctor/patients/{id}/history [get]
func (h *DoctorHandler) GetHistory(c *gin.Context) {
        cid, _ := strconv.ParseUint(c.Param("id"), 10, 32)
        c.JSON(http.StatusOK, h.svc.GetMedicalHistory(uint(cid)))
}

// UpdateStatus godoc
// @Summary      Изменить статус записи
// @Description  Принять (confirmed) или отклонить (rejected) запись
// @Tags         doctor
// @Security     ApiKeyAuth
// @Accept       json
// @Produce      json
// @Param        id      path      int     true  "ID записи"
// @Param        request  body      object{status=string}  true  "Новый статус"
// @Success      200      {object}  map[string]bool
// @Router       /appointments/{id}/status [patch]
```

```
func (h *DoctorHandler) UpdateStatus(c *gin.Context) {
	id, err := strconv.ParseUint(c.Param("id"), 10, 32)
	if err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": "Некорректный ID записи"})
		return
	}

	var req struct {
		Status string `json:"status" binding:"required"`
	}
	if err := c.ShouldBindJSON(&req); err != nil {
		c.JSON(http.StatusBadRequest, gin.H{"error": "Поле status обязательно"})
		return
	}

	if err := h.svc.UpdateStatus(uint(id), req.Status); err != nil {
		c.JSON(http.StatusInternalServerError, gin.H{"error": "Не удалось обновить статус"})
		return
```

```
	}


	c.JSON(http.StatusOK, gin.H{"success": true})
}
```

handler/diagnosis_handler.go

```
package handler


import (
	"api/internal/service"
	"net/http"


	"github.com/gin-gonic/gin"
)


type DiagnosisHandler struct {
	svc service.DiagnosisService
}


func NewDiagnosisHandler(svc service.DiagnosisService) *DiagnosisHandler {
	return &DiagnosisHandler{svc}
```

```
}


// GetAll godoc
// @Summary      Список диагнозов
// @Description  Возвращает справочник диагнозов для выбора врачом
// @Tags         diagnosis
// @Security     ApiKeyAuth
// @Produce      json
// @Success      200  {array}  dto.DiagnosisDTO
// @Router       /diagnoses [get]
func (h *DiagnosisHandler) GetAll(c *gin.Context) {
        diagnoses, err := h.svc.GetAll()
        if err != nil {
                c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
                return
        }
        c.JSON(http.StatusOK, diagnoses)
}
```

internal/middleware

middleware/auth.go

```
package middleware


import (
	"net/http"
	"strings"

	"github.com/gin-gonic/gin"
	"github.com/golang-jwt/jwt/v5"
)


var jwtSecret = []byte("super-secret-key")


func AuthMiddleware() gin.HandlerFunc {
	return func(c *gin.Context) {
		header := c.GetHeader("Authorization")
		if !strings.HasPrefix(header, "Bearer ") {
			c.JSON(http.StatusUnauthorized, gin.H{"error": "Нет токена"})
			c.Abort()
			return
		}
```

```
            tokenStr := strings.TrimPrefix(header, "Bearer ")
            token, err := jwt.Parse(tokenStr, func(t *jwt.Token) (interface{}, error) {
                return jwtSecret, nil
            })
            if err != nil || !token.Valid {
                c.JSON(http.StatusUnauthorized, gin.H{"error": "Неверный
токен"})
                c.Abort()
                return
            }

            claims := token.Claims.(jwt.MapClaims)
            userID := uint(claims["sub"].(float64))
            c.Set("userID", userID)
            c.Next()
        }
}
```

utils

utils/database.go

```
package database


import (
	"api/internal/models"
	"fmt"
	"log"


	"gorm.io/driver/mysql"
	"gorm.io/gorm"
)


func InitDB() *gorm.DB {
	user := "root"
	pass := "password"
	host := "127.0.0.1"
	port := "3306"
	dbName := "vet_clinic"


	dsn := fmt.Sprintf("%s:%s@tcp(%s:%s)/%s?
charset=utf8mb4&parseTime=True&loc=Local",
```

```
        user, pass, host, port, dbName)


db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})
if err != nil {
        log.Fatal("Ошибка подключения к базе: ", err)
}


db.AutoMigrate(
        &models.User{},
        &models.Pet{},
        &models.Service{},
        &models.Appointment{},
        &models.Diagnosis{},
        &models.MedicalProtocol{},
        &models.Product{},
        &models.CartItem{},
        &models.Order{},
        &models.OrderItem{},
)
```

```
        return db

}
```

Клиентское веб-приложение (Vue 3)

components

components/Main.vue

```
<div class="layout">
  <SideBar
    :current-role="userRole"
    :active-page="currentPage"
    :is-open="mobileMenu"
    @navigate="(p) => (currentPage = p)"
    @close="mobileMenu = false"
  />

  <main class="main">
    <div class="page-container">
      <template v-if="userRole === 'client'">
        <DashboardPage
          v-if="currentPage === 'dashboard'"
          @navigate="(p) => (currentPage = p)"
        />
        <PetsPage v-else-if="currentPage === 'pets'" />
        <BookPage v-else-if="currentPage === 'book'" />
        <AppointmentsPage
          v-else-if="currentPage === 'appointments'"
        />
        <StatsPage v-else-if="currentPage === 'stats'" />
        <ShopPage v-else-if="currentPage === 'shop'" />
        <OrdersPage v-else-if="currentPage === 'orders'" />
      </template>

      <template v-else-if="userRole === 'doctor'">
        <SchedulePage v-if="currentPage === 'schedule'" />
        <PatientsPage v-else-if="currentPage === 'patients'" />
      </template>

      <div v-else class="page">
        <div class="page-header">
          <div>
            <div class="page-title">{{ currentPage }}</div>
```

```
                    <div class="page-sub">
                        Страница не найдена или недоступна
                    </div>
                </div>
            </div>
        </div>
    </main>
</div>

<Profile
    :is-open="isProfileOpen"
    @close="isProfileOpen = false"
    @logout="emit('logout')"
/>
```

components/elements

components/elements/Auth.vue

```
ВетКлиника
<div class="auth-box">
 <div class="auth-tabs">
 <div
 class="auth-tab"
 :class="{ active: activeTab === 'login' }"
 @click="activeTab = 'login'"
 >
 Вход
 </div>
 <div
 class="auth-tab"
 :class="{ active: activeTab === 'reg' }"
 @click="activeTab = 'reg'"
 >
 Регистрация
 </div>
 </div>

 <div v-if="activeTab === 'login'" class="auth-form">
 <div class="form-group">
 <label>Email</label>
 <input
 v-model="loginData.email"
```

```
type="email"
placeholder="example@mail.ru"
                :disabled="loading"
              />
            </div>
            <div class="form-group">
              <label>Пароль</label>
              <input
                v-model="loginData.pass"
                type="password"
                placeholder="Минимум 8 символов"
                :disabled="loading"
                @keyup.enter="doLogin"
              />
            </div>
            <button
              class="auth-submit"
              @click="doLogin"
              :disabled="loading"
            >
              {{ loading ? "Вход..." : "Войти" }}
            </button>
            <p class="auth-footer-text">
              Нет аккаунта?
              <span class="auth-link" @click="activeTab = 'reg'"
                >Зарегистрироваться</span
              >
            </p>
          </div>

          <div v-else class="auth-form">
            <div class="form-row">
              <div class="form-group">
                <label>Фамилия</label>
                <input
                  v-model="regData.lastName"
                  type="text"
                  placeholder="Иванов"
                  :disabled="loading"
                />
              </div>
              <div class="form-group">
                <label>Имя</label>
                <input
                  v-model="regData.firstName"
                  type="text"
```

```
            placeholder="Иван"
            :disabled="loading"
          />
        </div>
      </div>
      <div class="form-group">
        <label>Телефон</label>
        <input
          v-model="regData.phone"
          type="text"
          placeholder="+79001234567"
          :disabled="loading"
        />
      </div>
      <div class="form-group">
        <label>Email</label>
        <input
          v-model="regData.email"
          type="email"
          placeholder="example@mail.ru"
          :disabled="loading"
        />
      </div>
      <div class="form-group">
        <label>Пароль</label>
        <input
          v-model="regData.pass"
          type="password"
          placeholder="Минимум 8 символов"
          :disabled="loading"
          @keyup.enter="doRegister"
        />
      </div>
      <button
        class="auth-submit"
        @click="doRegister"
        :disabled="loading"
      >
        {{ loading ? "Регистрация..." : "Зарегистрироваться" }}
      </button>
      <p class="auth-footer-text">
        Уже есть аккаунт?
        <span class="auth-link" @click="activeTab = 'login'"
          >Войти</span
        >
      </p>
```

```
      </div>
    </div>
  </div>
  components/elements/BaseModal.vue
  <script setup lang="ts">
  defineProps<{ show: boolean; title: string; maxWidth?: string }>();
  const emit = defineEmits<{ close: [] }>();
  </script>

  <template>
    <Teleport to="body">
      <div v-if="show" class="modal-overlay" @click.self="emit('close')">
        <div class="modal" :style="maxWidth ? `max-width:${maxWidth}` : ''">
          <div class="modal-header">
            <span class="modal-title">{{ title }}</span>
            <button class="modal-close" @click="emit('close')">
              ×
            </button>
          </div>
          <div class="modal-body">
            <slot />
          </div>
          <div class="modal-footer" v-if="$slots.footer">
            <slot name="footer" />
          </div>
        </div>
      </div>
    </Teleport>
  </template>

  <style scoped>
  .modal-overlay {
    position: fixed;
    inset: 0;
    background: rgba(0, 0, 0, 0.4);
    z-index: 1000;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 20px;
  }
  .modal {
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 10px;
    width: 100%;
```

```
    max-width: 520px;
    overflow: hidden;
    animation: fadeIn 0.15s ease;
}
.modal-header {
    padding: 14px 16px;
    border-bottom: 1px solid var(--border);
    display: flex;
    align-items: center;
    justify-content: space-between;
}
.modal-title {
    font-size: 14px;
    font-weight: 600;
}
.modal-close {
    background: none;
    border: none;
    color: var(--text3);
    cursor: pointer;
    font-size: 14px;
    padding: 2px 6px;
    border-radius: 4px;
    transition: background 0.15s;
}
.modal-close:hover {
    background: var(--surface3);
    color: var(--text);
}
.modal-body {
    padding: 16px;
}
.modal-footer {
    padding: 12px 16px;
    border-top: 1px solid var(--border);
    display: flex;
    justify-content: flex-end;
    gap: 8px;
}
</style>
```

components/user

components/user/BookPage.vue

Записаться на визит

Выберите время для посещения клиники

```
<div class="card booking-card">
 <div class="card-body">
 <div class="booking-row">
 <div class="form-group">
                <label>Питомец *</label>
                <select v-model.number="form.petId">
                  <option :value="null">— Выбрать —</option>
                  <option v-for="p in pets" :key="p.id" :value="p.id">
                    {{ p.avatar }} {{ p.name }}
                  </option>
                </select>
              </div>
              <div class="form-group">
                <label>Цель визита</label>
                <select v-model.number="form.serviceId">
                  <option :value="null">— Осмотр —</option>
                  <option
                    v-for="s in services"
                    :key="s.id"
                    :value="s.id"
                  >
                    {{ s.name }} ({{ s.price }}₽)
                  </option>
                </select>
              </div>
              <div class="form-group">
                <label>Направление *</label>
                <select v-model="form.specialization">
                  <option value="">— Профиль —</option>
                  <option
                    v-for="s in specializations"
                    :key="s"
                    :value="s"
                  >
                    {{ s }}
                  </option>
                </select>
              </div>
              <div class="form-group">
                <label>Врач *</label>
                <select
                  v-model.number="form.doctorId"
                  :disabled="!form.specialization"
                >
```

```
            <option :value="null">— Выберите —</option>
            <option
              v-for="d in filteredDoctors"
              :key="d.doctorId"
              :value="d.doctorId"
            >
              {{ d.lastName }} {{ d.firstName }}
            </option>
          </select>
        </div>
        <div class="form-group">
          <label>Дата *</label>
          <input v-model="form.date" type="date" :min="minDate" />
        </div>
        <div class="form-group">
          <label>Время *</label>
          <select
            v-model="form.time"
            :disabled="!form.date || !form.doctorId"
          >
            <option value="">--:--</option>
            <option
              v-for="s in TIME_SLOTS"
              :key="s"
              :value="s"
              :disabled="busySlots.includes(s)"
            >
              {{ s }}
              {{ busySlots.includes(s) ? "(занято)" : "" }}
            </option>
          </select>
        </div>
      </div>

      <div class="form-group mt-16">
        <label>Комментарий</label>
        <textarea
          v-model="form.comment"
          rows="2"
          placeholder="Опишите жалобы или причину обращения..."
        ></textarea>
      </div>

      <div
        v-if="form.date && form.time && form.doctorId"
        class="summary-plate"
```

```
        >
          <div class="summary-info">
            <div class="summary-date">
              {{ form.date.split("-").reverse().join(".") }} в
              {{ form.time }}
            </div>
            <div class="summary-details">
              Врач: {{ selectedDoctor?.lastName }}
              {{ selectedDoctor?.firstName }}
              <span v-if="selectedService">
                • {{ selectedService.name }}</span
              >
            </div>
          </div>
          <div class="summary-price" v-if="selectedService">
            {{ selectedService.price }} ₽
          </div>
        </div>

        <div class="row mt-20" style="gap: 12px">
          <button
            class="btn btn-primary"
            :disabled="loading"
            @click="sendRequest"
          >
            {{ loading ? "Отправка..." : "√ Подтвердить запись" }}
          </button>
          <button
            class="btn btn-ghost"
            @click="emit('navigate', 'dashboard')"
          >
            Отмена
          </button>
        </div>
      </div>
    </div>
  </div>
components/user/AppointmentsPage.vue

Мои визиты
История посещений и медицинские заключения

    <div v-if="loading" class="text-muted py-40 center">Загрузка...</div>
    <div v-else-if="!appointments.length" class="card py-40 center">
      <p class="text-muted">Записей не найдено</p>
    </div>
```

```
<div v-else class="table-wrap">
  <table>
    <thead>
      <tr>
        <th>Питомец</th>
        <th>Дата и время</th>
        <th>Врач</th>
        <th>Статус</th>
        <th></th>
      </tr>
    </thead>
    <tbody>
      <tr v-for="app in appointments" :key="app.id">
        <td class="td-main">{{ app.petName }}</td>
        <td>
          <div class="mono" style="font-size: 13px">
            {{ app.date }}
          </div>
          <div class="text-muted" style="font-size: 11px">
            {{ app.time }}
          </div>
        </td>
        <td>{{ app.doctorName }}</td>
        <td>
          <span
            v-if="statusMap[app.status]"
            :class="['badge', statusMap[app.status].cls]"
          >
            {{ statusMap[app.status].label }}
          </span>
        </td>
        <td style="text-align: right">
          <button
            v-if="app.status === 'done'"
            class="btn btn-primary btn-sm"
            @click="showProtocol(app)"
          >
            Протокол
          </button>
        </td>
      </tr>
    </tbody>
  </table>
</div>

<BaseModal
```

```
    :show="isProtocolOpen"
    title="Медицинское заключение"
    @close="isProtocolOpen = false"
  >
    <div class="protocol-view" v-if="selectedProtocol">
      <template v-if="selectedProtocol.protocol">
        <div class="p-summary">
          <div class="p-summary-item">
            <label>Пациент</label>
            <span>{{ selectedProtocol.petName }}</span>
          </div>
          <div class="p-summary-item">
            <label>Вес</label>
            <span
              >{{
                selectedProtocol.protocol.weight ||
                selectedProtocol.protocol.weight_at_visit
              }}
              кг</span
            >
          </div>
        </div>
        <div class="p-section">
          <label>Поставленный диагноз</label>
          <div class="p-val diagnosis-tags">
            <span
              v-for="d in selectedProtocol.protocol.diagnoses"
              :key="d.id"
              class="diagnosis-tag-readonly"
            >
              {{ d.name }}
            </span>
            <span
              v-if="
                !selectedProtocol.protocol.diagnoses?.length
              "
              class="text-muted"
            >
              Диагноз не указан
            </span>
          </div>
        </div>
        <div class="p-section">
          <label>Рекомендации и лечение</label>
          <div class="p-val">
            {{ selectedProtocol.protocol.treatment }}
```

```
            </div>
          </div>
          <button
            class="btn btn-accent full mt-20"
            @click="printProtocol"
          >
            🖨️ Сохранить в PDF
          </button>
        </template>
        <div v-else class="center py-20">
          <p class="text-muted">
            Протокол приема еще не заполнен врачом.
          </p>
        </div>
      </div>
    </BaseModal>

    <div style="position: absolute; left: -9999px; top: 0">
      <div ref="protocolRef" class="pdf-document">
        <div class="pdf-header">
          <div class="pdf-logo">🐾</div>
          <div class="pdf-clinic-info">
            <h2>ВЕТЕРИНАРНЫЙ ЦЕНТР "ВетКлиника"</h2>
            <p>Лицензия № ВЕТ-77-01-000842 | petcare-clinic.ru</p>
            <p>
              г. Москва, ул. Ветеринарная, 12 | +7 (495) 000-00-00
            </p>
          </div>
        </div>

        <h1 class="pdf-title">
          МЕДИЦИНСКОЕ ЗАКЛЮЧЕНИЕ №{{ selectedProtocol?.id }}
        </h1>

        <div class="pdf-meta-grid">
          <div class="pdf-meta-item">
            <strong>Пациент:</strong>
            {{ selectedProtocol?.petName }}
          </div>
          <div class="pdf-meta-item">
            <strong>Вид животного:</strong>
            {{
              selectedProtocol?.petSpecies || "Домашнее животное"
            }}
          </div>
          <div class="pdf-meta-item">
```

```
        <strong>Дата приема:</strong>
        {{ selectedProtocol?.date }}
      </div>
      <div class="pdf-meta-item">
        <strong>Вес пациента:</strong>
        {{
          selectedProtocol?.protocol?.weight ||
          selectedProtocol?.protocol?.weight_at_visit
        }}
        кг
      </div>
      <div class="pdf-meta-item">
        <strong>Лечащий врач:</strong>
        {{ selectedProtocol?.doctorName }}
      </div>
    </div>

    <div class="pdf-content-block">
      <h3 class="pdf-section-h">1. Анамнез и диагноз</h3>
      <div class="pdf-text-box">
        {{ diagnosisNames(selectedProtocol?.protocol) }}
      </div>
    </div>

    <div class="pdf-content-block">
      <h3 class="pdf-section-h">
        2. Назначенное лечение и рекомендации
      </h3>
      <div class="pdf-text-box">
        {{ selectedProtocol?.protocol?.treatment }}
      </div>
    </div>

    <div
      class="pdf-content-block"
      v-if="selectedProtocol?.protocol?.medications"
    >
      <h3 class="pdf-section-h">3. Медикаментозные назначения</h3>
      <div class="pdf-text-box highlight">
        {{ selectedProtocol?.protocol?.medications }}
      </div>
    </div>

    <div class="pdf-footer">
      <div class="pdf-signature">
        <div class="pdf-sig-line"></div>
```

```
                <p>
                  Подпись врача: _____ /
                  {{ selectedProtocol?.doctorName }} /
                </p>
              </div>
              <div class="pdf-stamp">ДЛЯ ДОКУМЕНТОВ</div>
            </div>
          </div>
        </div>
```

components/doctor

components/doctor/SchedulePage.vue

```
<script setup lang="ts">

import { ref, reactive, computed, onMounted } from "vue";

import BaseModal from "../elements/BaseModal.vue";

import { useToast } from "../../utils/useToast";

const { showToast } = useToast();



// --- СОСТОЯНИЕ ---

const appointments = ref<any[]>([]);

const diagnosesList = ref<{ id: number; name: string }[]>([]);

const loading = ref(false);

const isReportModalOpen = ref(false);

const selectedApp = ref<any>(null);

const isDiagnosisPickerOpen = ref(false);
```

```
const availableDiagnoses = computed(() =>
    diagnosesList.value.filter((d) => !reportForm.diagnosisIds.includes(d.id)),
);

const reportForm = reactive({
    weight: "",
    diagnosisIds: [] as number[],
    treatment: "",
    medications: "",
});

const statusMap: Record<string, { label: string; cls: string }> = {
    waiting: { label: "Ожидает", cls: "badge-waiting" },
    confirmed: { label: "Принята", cls: "badge-confirmed" },
    rejected: { label: "Отклонена", cls: "badge-rejected" },
    done: { label: "Завершена", cls: "badge-info" },
};

const token = localStorage.getItem("token");
const doctorId = JSON.parse(localStorage.getItem("user") || "{}")?.id;
```

```
async function loadSchedule() {
    loading.value = true;
    try {
        const res = await fetch(`/api/doctor/schedule?doctor_id=${doctorId}`, {
            headers: {
                Authorization: `Bearer ${token}`,
            },
        });
        if (res.ok) {
            appointments.value = await res.json();
        } else {
            appointments.value = [];
        }
    } catch (err) {
        console.error("Ошибка загрузки расписания:", err);
        appointments.value = [];
    } finally {
        loading.value = false;
    }
```

```
}


async function loadDiagnoses() {
    try {
        const res = await fetch(`/api/diagnoses`, {
            headers: { Authorization: `Bearer ${token}` },
        });
        if (res.ok) {
            diagnosesList.value = await res.json();
        }
    } catch (err) {
        console.error("Ошибка загрузки справочника диагнозов:", err);
    }
}


function openCompleteModal(app: any) {
    selectedApp.value = app;
    Object.assign(reportForm, {
        weight: "",
        diagnosisIds: [],
```

```
        treatment: "",
        medications: "",
    });
    isDiagnosisPickerOpen.value = false;
    if (!diagnosesList.value.length) {
        loadDiagnoses();
    }
    isReportModalOpen.value = true;
}
async function saveProtocol() {
    if (!reportForm.weight || !reportForm.diagnosisIds.length)
        return showToast(
            "Поле 'Вес' и хотя бы один диагноз обязательны для заполнения",
            "info",
        );

    try {
        const res = await fetch(
            `/api/appointments/${selectedApp.value.id}/complete`,
            {
```

```
        method: "PATCH",
        headers: {
          "Content-Type": "application/json",
          Authorization: `Bearer ${token}`,
        },
        body: JSON.stringify({
          weight: Number(reportForm.weight),
          diagnosis_ids: reportForm.diagnosisIds,
          treatment: reportForm.treatment,
          medications: reportForm.medications,
        }),
      },
    );

    if (res.ok) {
      isReportModalOpen.value = false;
      await loadSchedule();
    } else {
      const err = await res.json();
      alert(err.message || "Ошибка при сохранении протокола");
```

```
        }
    } catch (err) {
        alert("Ошибка связи с сервером");
    }
}

async function updateStatus(appId: number, status: string) {
    try {
        const res = await fetch(`/api/appointments/${appId}/status`, {
            method: "PATCH",
            headers: {
                "Content-Type": "application/json",
                Authorization: `Bearer ${token}`,
            },
            body: JSON.stringify({ status }),
        });
        if (res.ok) {
            await loadSchedule();
        }
    } catch (err) {
```

```
        alert("Ошибка при обновлении статуса");
    }
}

function getDiagnosisName(id: number) {
    return diagnosesList.value.find((d) => d.id === id)?.name || "";
}

function toggleDiagnosisPicker() {
    isDiagnosisPickerOpen.value = !isDiagnosisPickerOpen.value;
}

function addDiagnosis(id: number) {
    reportForm.diagnosisIds.push(id);
    isDiagnosisPickerOpen.value = false;
}

function removeDiagnosis(id: number) {
    reportForm.diagnosisIds = reportForm.diagnosisIds.filter((x) => x !== id);
}
```

```
onMounted(loadSchedule);
</script>

<template>
  <div class="page">
    <div class="page-header">
      <div>
        <h1 class="page-title">Приём пациентов</h1>
        <p class="page-sub">
          Управление текущими визитами и заполнение мед. карт
        </p>
      </div>
      <button
        class="btn btn-ghost"
        @click="loadSchedule"
        :disabled="loading"
      >
        {{ loading ? "Обновление..." : "🔄 Обновить" }}
      </button>
```

```
</div>


<div
    v-if="loading && !appointments.length"
    class="text-muted py-40 center"
>
    Загрузка расписания...
</div>


<div v-else-if="!appointments.length" class="card py-40 center">
    <p class="text-muted">На сегодня записей нет</p>
</div>


<div v-else class="schedule-list">
    <div
        v-for="app in appointments"
        :key="app.id"
        class="card app-card-doctor"
    >
        <div class="row-between">
```

```
<div class="app-main-info">
    <span class="app-time-tag">{{ app.time }}</span>
    <span class="app-pet-name">{{ app.petName }}</span>
    <span class="app-owner-name"
        >({{ app.ownerName }})</span
    >
</div>

<div class="app-actions">
    <span
        v-if="statusMap[app.status]"
        :class="['badge', statusMap[app.status].cls]"
    >
        {{ statusMap[app.status].label }}
    </span>

    <!-- Действия для новых записей -->
    <div
        v-if="app.status === 'waiting'"
        class="btn-group-mini"
```

```
>
    <button
        class="btn btn-primary btn-sm"
        @click="updateStatus(app.id, 'confirmed')"
    >
        Принять
    </button>
    <button
        class="btn btn-ghost btn-sm text-red"
        @click="updateStatus(app.id, 'rejected')"
    >
        Отклонить
    </button>
</div>

<!-- Действие для подтвержденных (завершение приема) -->
<button
    v-if="app.status === 'confirmed'"
    class="btn btn-primary btn-sm"
    @click="openCompleteModal(app)"
```

```
            >
                Завершить приём
            </button>
          </div>
        </div>
      </div>
    </div>

    <!-- Модалка Протокола -->
    <BaseModal
      :show="isReportModalOpen"
      title="Медицинский протокол"
      @close="isReportModalOpen = false"
    >
      <div class="form-grid" v-if="selectedApp">
        <div class="form-group">
          <label>Вес питомца (кг) *</label>
          <input
            v-model="reportForm.weight"
            type="number"
```

```
            step="0.1"
            placeholder="Например: 5.4"
          />
        </div>
        <div class="form-group full">
          <label>Диагноз *</label>
          <div class="diagnosis-box">
            <span
              v-for="id in reportForm.diagnosisIds"
              :key="id"
              class="diagnosis-tag"
            >
              {{ getDiagnosisName(id) }}
              <button
                type="button"
                class="tag-remove"
                @click="removeDiagnosis(id)"
              >
                ×
              </button>
```

```
</span>


<div class="diagnosis-add">
  <button
    type="button"
    class="tag-add-btn"
    @click="toggleDiagnosisPicker"
  >
    +
  </button>

  <div
    v-if="isDiagnosisPickerOpen"
    class="diagnosis-dropdown"
  >
    <div
      v-for="d in availableDiagnoses"
      :key="d.id"
      class="diagnosis-option"
      @click="addDiagnosis(d.id)"
```

```
                >
                    {{ d.name }}
                </div>
                <div
                    v-if="!availableDiagnoses.length"
                    class="diagnosis-empty"
                >
                    {{
                        diagnosesList.length
                            ? "Все диагнозы добавлены"
                            : "Загрузка..."
                    }}
                </div>
            </div>
        </div>
    </div>
    <div class="form-group full">
        <label>Рекомендации и лечение</label>
        <textarea
```

```
            v-model="reportForm.treatment"
            rows="2"
            placeholder="Манипуляции, советы по уходу..."
          ></textarea>
        </div>
        <div class="form-group full">
          <label>Назначенные препараты</label>
          <input
            v-model="reportForm.medications"
            type="text"
            placeholder="Лекарства, дозировка, длительность"
          />
        </div>
      </div>
      <template #footer>
        <button class="btn btn-primary" @click="saveProtocol">
          Подтвердить и закрыть визит
        </button>
      </template>
    </BaseModal>
```

```
    </div>
</template>


<style scoped>
.center {
    text-align: center;
}
.py-40 {
    padding: 40px 0;
}
.text-red {
    color: var(--red);
}
.btn-group-mini {
    display: flex;
    gap: 8px;
}


.app-card-doctor {
    padding: 16px 20px;
```

```
    margin-bottom: 12px;
    border-radius: 12px;
    transition: transform 0.2s;
}
.app-card-doctor:hover {
    transform: translateX(4px);
}

.app-time-tag {
    font-family: var(--mono);
    font-weight: 800;
    color: var(--accent);
    margin-right: 15px;
    font-size: 16px;
}
.app-pet-name {
    font-weight: 700;
    font-size: 15px;
    margin-right: 5px;
}
```

```
.app-owner-name {
    color: var(--text3);
    font-size: 13px;
}
.app-actions {
    display: flex;
    align-items: center;
    gap: 15px;
}

.diagnosis-box {
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    gap: 8px;
    min-height: 44px;
    padding: 8px;
    border: 1px solid var(--border, #e0e0e0);
    border-radius: 10px;
}
```

```
.diagnosis-tag {
    display: flex;
    align-items: center;
    gap: 6px;
    font-size: 13px;
    padding: 6px 10px;
    border-radius: 999px;
    background: var(--accent-soft, #e6f4ea);
    color: var(--accent, #1a8f4c);
    font-weight: 600;
}

.tag-remove {
    border: none;
    background: none;
    cursor: pointer;
    font-size: 14px;
    line-height: 1;
    color: inherit;
```

```
    opacity: 0.7;
    padding: 0;
}
.tag-remove:hover {
    opacity: 1;
}

.diagnosis-add {
    position: relative;
}

.tag-add-btn {
    width: 32px;
    height: 32px;
    border-radius: 50%;
    border: 2px solid var(--accent, #1a8f4c);
    color: var(--accent, #1a8f4c);
    background: transparent;
    font-size: 18px;
    line-height: 1;
```

```
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
}
.tag-add-btn:hover {
    background: var(--accent-soft, #e6f4ea);
}

.diagnosis-dropdown {
    position: absolute;
    top: 40px;
    left: 0;
    z-index: 20;
    min-width: 220px;
    max-height: 220px;
    overflow-y: auto;
    background: var(--surface, #fff);
    border: 1px solid var(--border, #e0e0e0);
    border-radius: 10px;
```

```
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
    padding: 6px;
}

.diagnosis-option {
    padding: 8px 10px;
    border-radius: 6px;
    font-size: 13px;
    cursor: pointer;
}
.diagnosis-option:hover {
    background: var(--bg2, #f5f5f5);
}

.diagnosis-empty {
    padding: 8px 10px;
    font-size: 13px;
    color: var(--text3, #999);
}
</style>
```

components/doctor/PatientsPage.vue

Картотека пациентов
Поиск владельцев и медицинская история питомцев

```
<!-- Поиск -->
<div class="search-panel card">
  <input
    v-model="search"
    type="text"
    placeholder="Поиск по ФИО или номеру телефона..."
    @input="loadPatients"
  />
</div>

<!-- Таблица -->
<div class="table-wrap mt-20">
  <div v-if="loading" class="text-muted py-40 center">
    Поиск в базе данных...
  </div>

  <table v-else>
    <thead>
      <tr>
        <th>Владелец</th>
        <th>Контакты</th>
        <th>Питомцев</th>
        <th>Последний визит</th>
        <th></th>
      </tr>
    </thead>
    <tbody>
      <tr v-if="patients.length === 0">
        <td colspan="5" class="center py-40 text-muted">
          Совпадений не найдено
        </td>
      </tr>
      <tr v-for="p in patients" :key="p.id">
        <td>
          <div class="td-main">{{ p.fullName }}</div>
        </td>
        <td>
          <div class="td-phone">{{ p.phone }}</div>
          <div class="td-sub">{{ p.email }}</div>
        </td>
```

```
            <td>
              <span class="tag">{{ p.petsCount }}</span>
            </td>
            <td>{{ p.lastVisit || "—" }}</td>
            <td style="text-align: right">
              <button
                class="btn btn-ghost btn-sm"
                @click="openHistory(p)"
              >
                Медкарта
              </button>
            </td>
          </tr>
        </tbody>
      </table>
    </div>

    <!-- Модалка истории (Медкарта) -->
    <BaseModal
      :show="isHistoryOpen"
      title="Медицинская история"
      @close="isHistoryOpen = false"
    >
      <div v-if="selectedOwner" class="history-content">
        <div class="owner-summary mb-20">
          <h3>{{ selectedOwner.fullName }}</h3>
          <p class="text-muted">{{ selectedOwner.phone }}</p>
        </div>

        <div v-if="historyLoading" class="center py-20 text-muted">
          Загрузка медицинских записей...
        </div>

        <div
          v-else-if="ownerHistory.length === 0"
          class="center py-20 text-muted"
        >
          История визитов пуста
        </div>

        <div
          v-for="pet in ownerHistory"
          :key="pet.petName"
          class="pet-history-item"
        >
          <div class="pet-header">
```

```
            <span class="pet-icon">{{ pet.petIcon || "🐾" }}</span>
            <span class="pet-name">{{ pet.petName }}</span>
          </div>

          <div class="history-timeline">
            <div
              v-for="visit in pet.visits"
              :key="visit.id"
              class="timeline-point"
            >
              <div class="t-date">{{ visit.date }}</div>
              <div class="t-info">
                <div class="t-diagnoses">
                  <span
                    v-for="d in visit.diagnoses"
                    :key="d.id"
                    class="t-diagnosis-tag"
                  >
                    {{ d.name }}
                  </span>
                  <span
                    v-if="!visit.diagnoses.length"
                    class="text-muted"
                  >
                    Диагноз не указан
                  </span>
                </div>
                <p>{{ visit.treatment }}</p>
              </div>
            </div>
          </div>
        </div>
      </BaseModal>
    </div>
```

ПРИЛОЖЕНИЕ В

Результат работы программного продукта

В приложении представлены скриншоты экранов разработанного веб-приложения, демонстрирующие результат работы информационной системы.

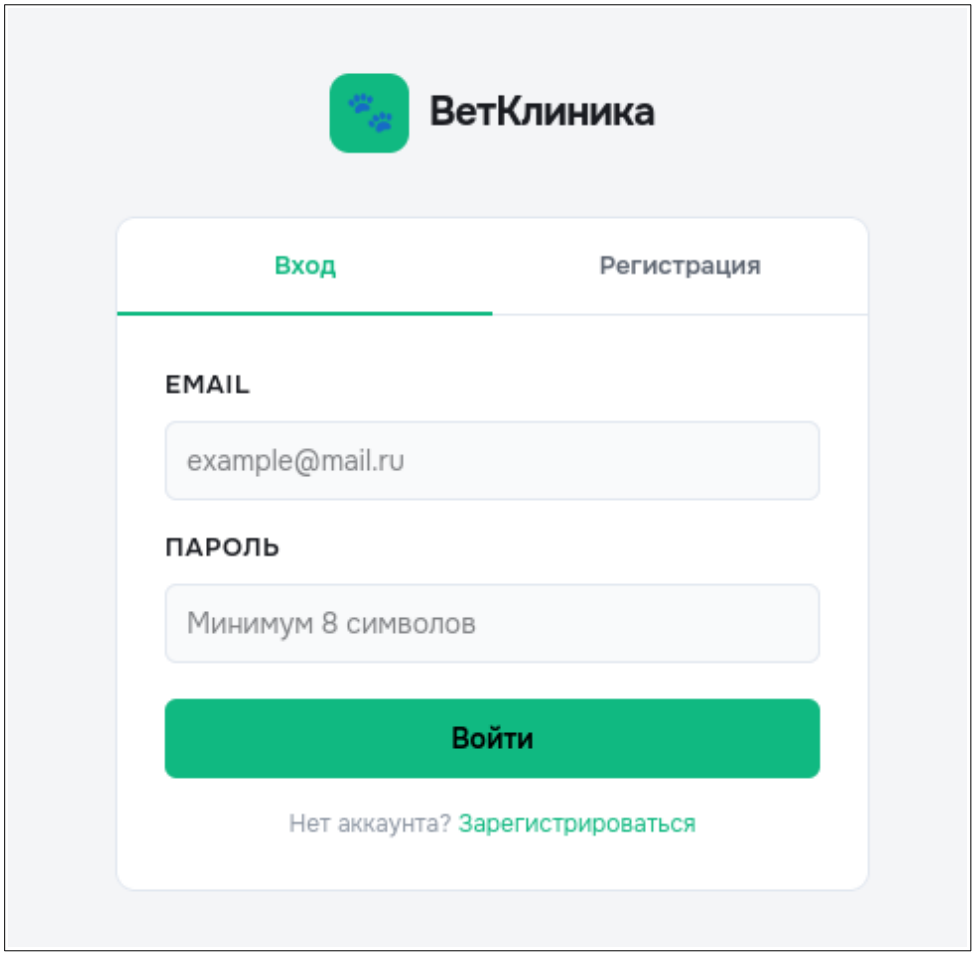


Рисунок В.1 – Форма авторизации пользователя

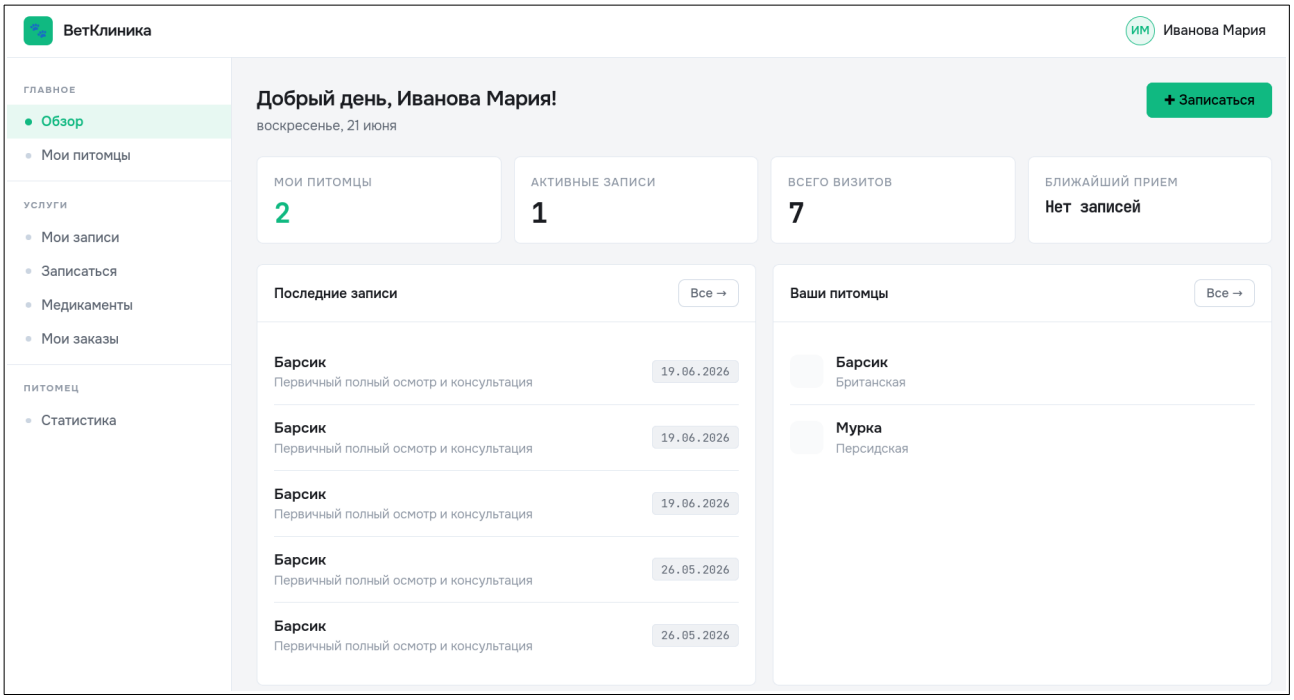


Рисунок В.2 – Дашборд клиента

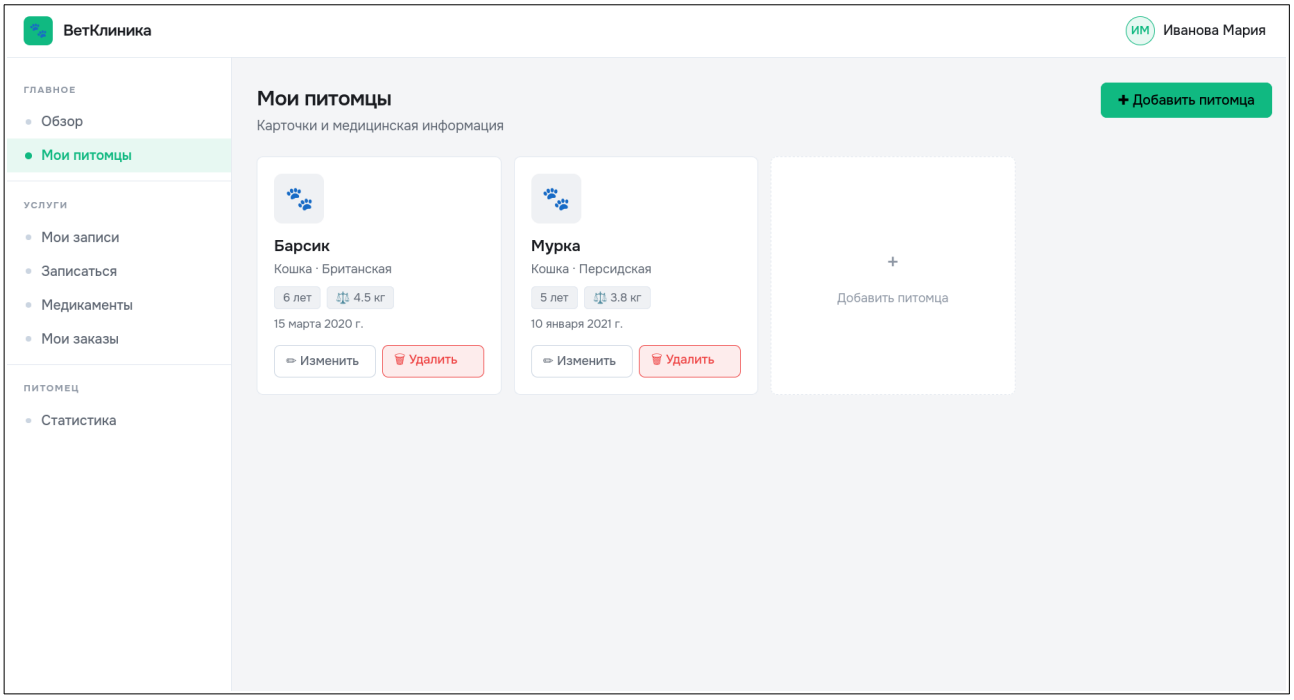


Рисунок В.3 – Страница «Питомцы»

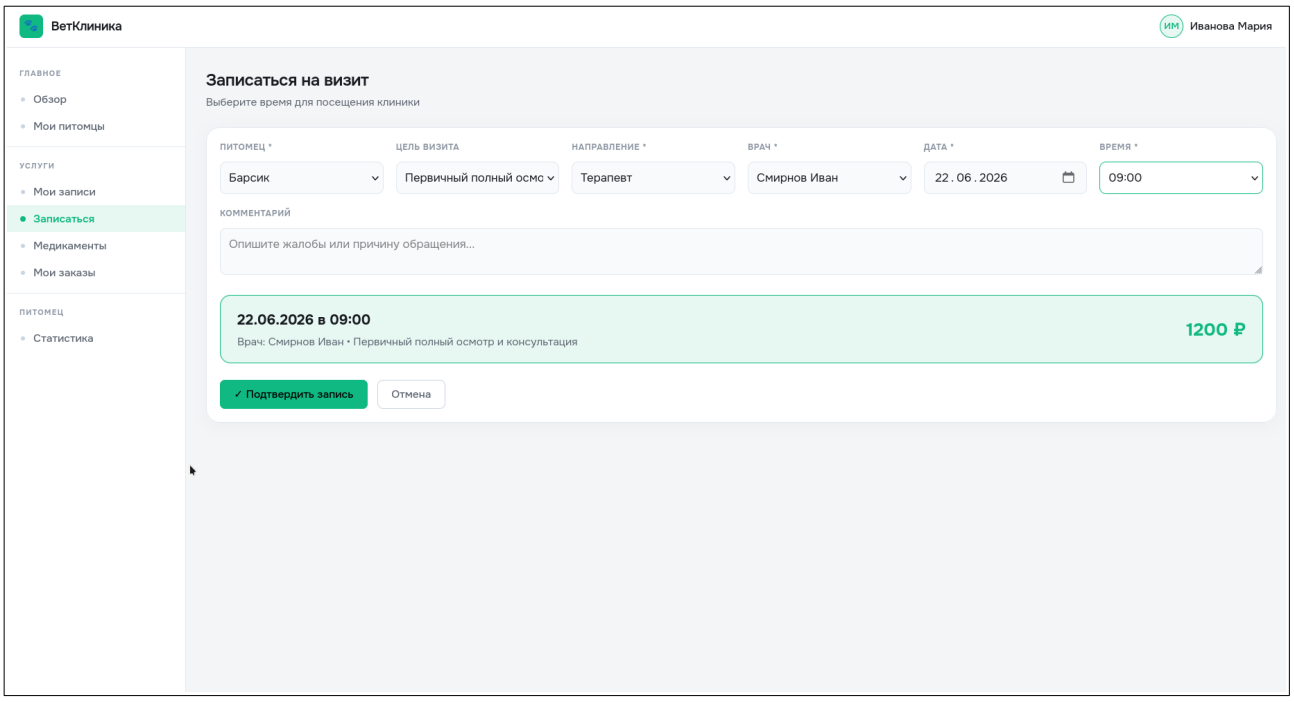


Рисунок В.4 – Форма записи на приём

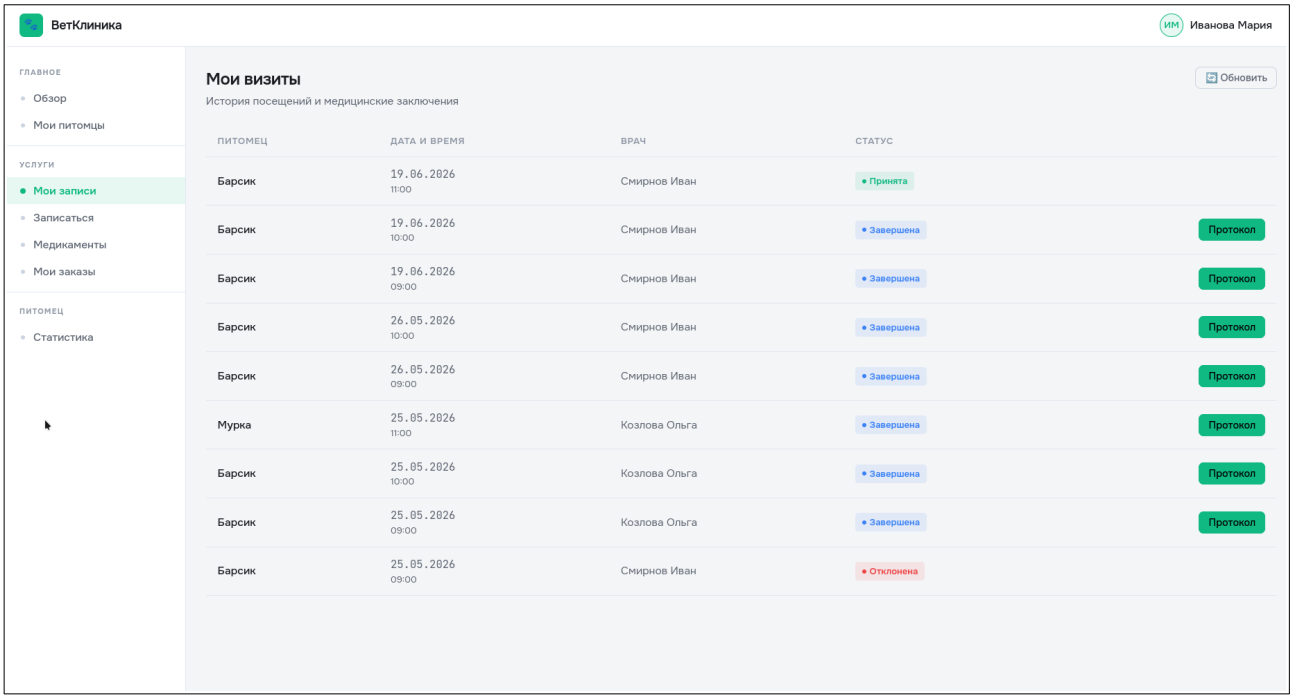


Рисунок В.5 – История визитов клиента

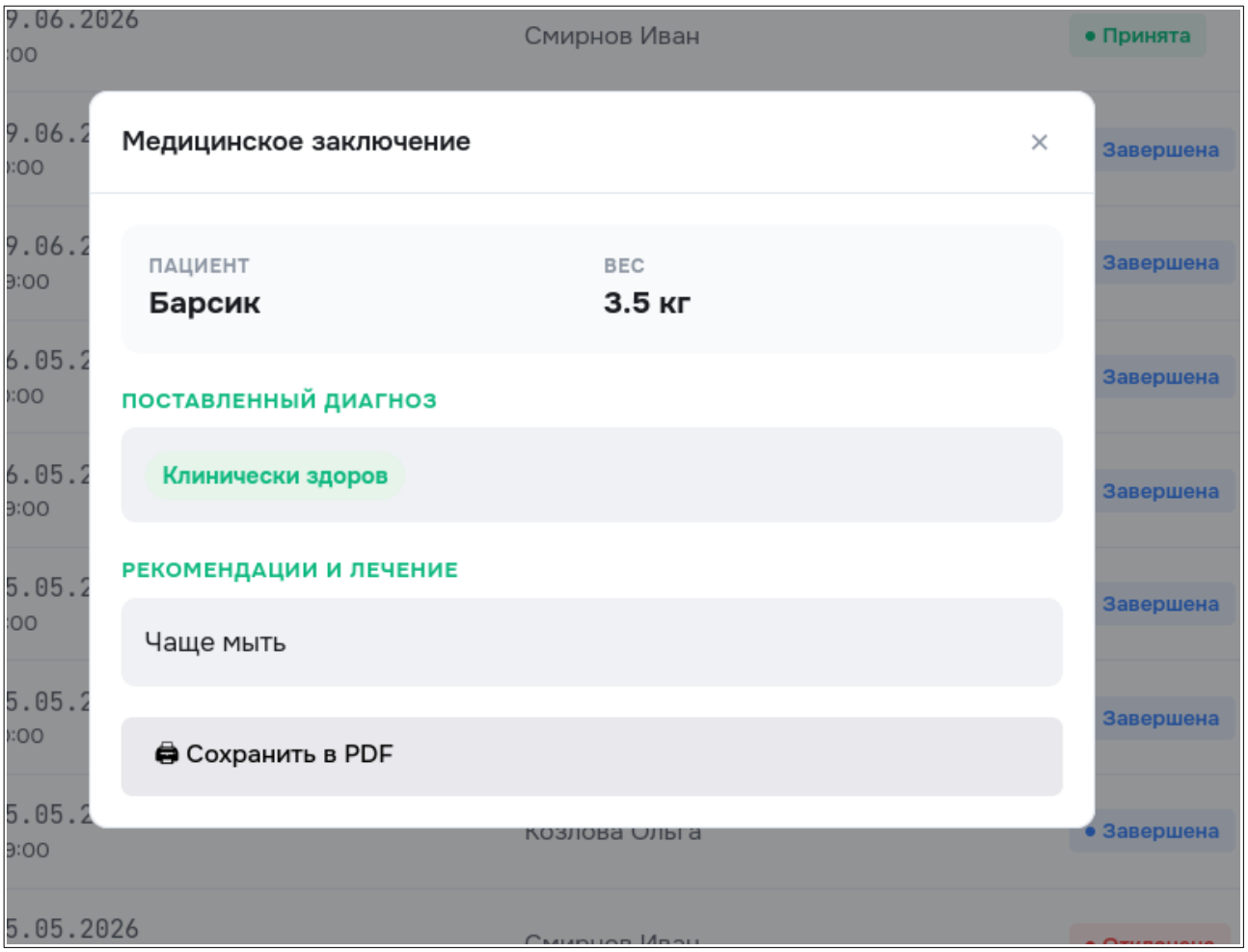


Рисунок В.6 – Медицинское заключение визита



ВЕТЕРИНАРНЫЙ ЦЕНТР "ВетКлиника"

Лицензия № ВЕТ-77-01-000842 | petcare-clinic.ru

г. Москва, ул. Ветеринарная, 12 | +7 (495) 000-00-00

МЕДИЦИНСКОЕ ЗАКЛЮЧЕНИЕ №11

Пациент: Барсик

Вид животного: Домашнее животное

Дата приема: 19.06.2026

Вес пациента: 3.5 кг

Лечащий врач: Смирнов Иван

1. АНАМНЕЗ И ДИАГНОЗ

Клинически здоров

2. НАЗНАЧЕННОЕ ЛЕЧЕНИЕ И РЕКОМЕНДАЦИИ

Чаще мыть

3. МЕДИКАМЕНТОЗНЫЕ НАЗНАЧЕНИЯ

Лекарство 1мл 5д

ДЛЯ ДОКУМЕНТОВ

Подпись врача: _____ / Смирнов Иван /

Рисунок В.7 – Медицинское заключение, экспортированное в PDF

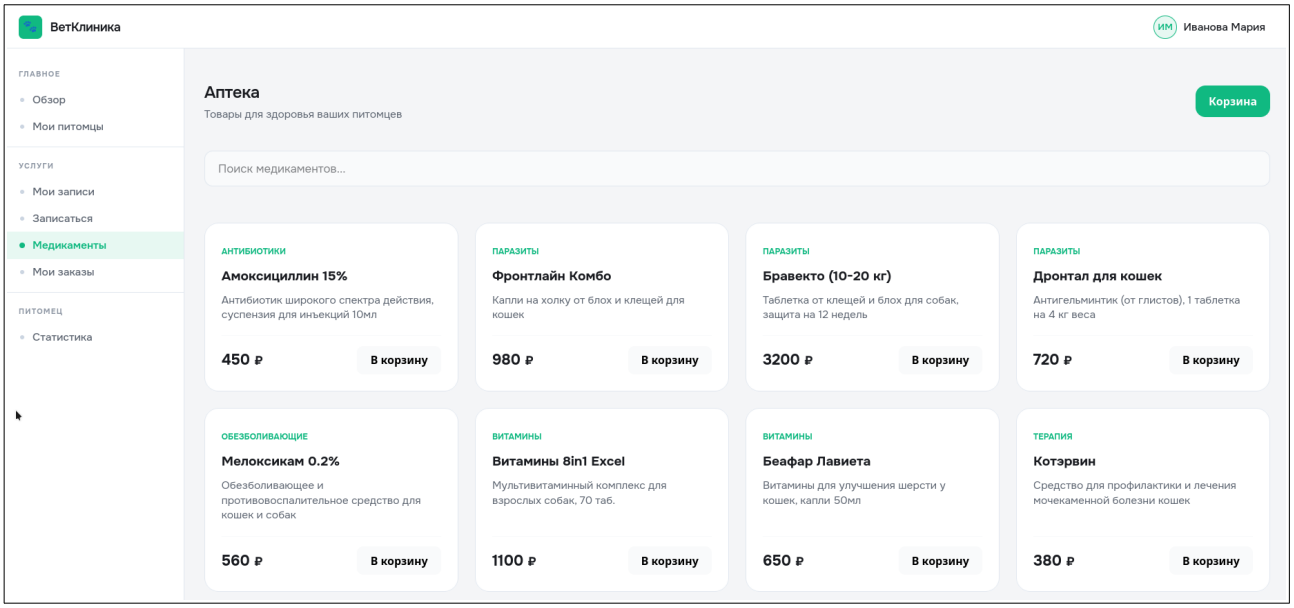


Рисунок В.8 – Страница «Магазин»

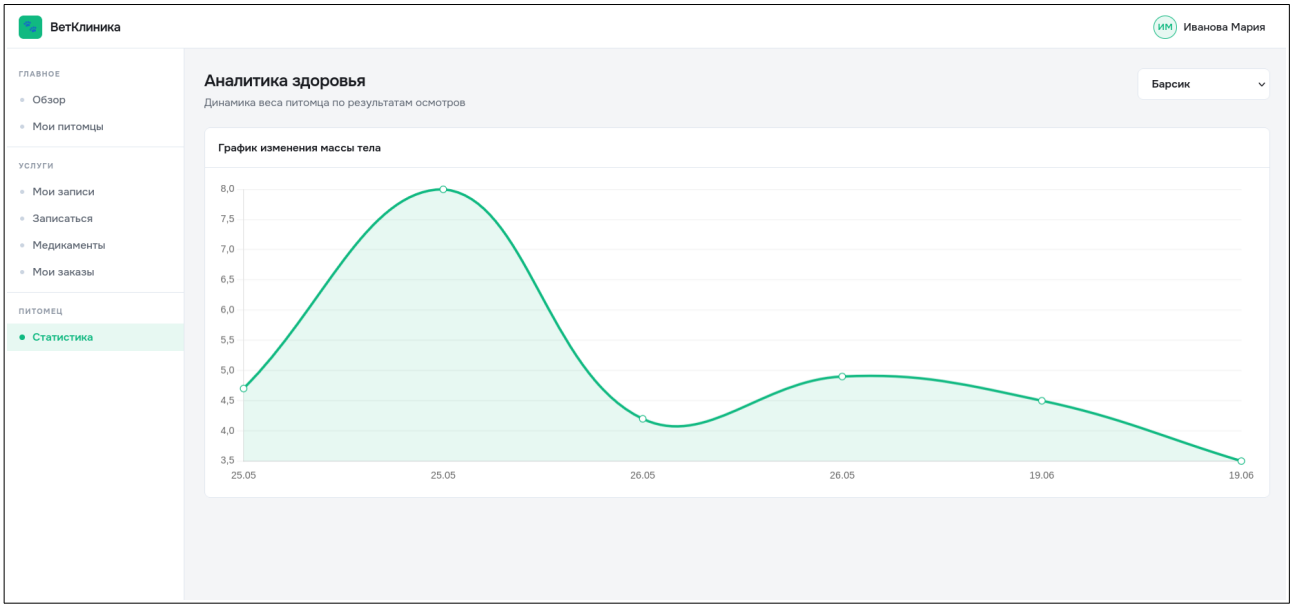


Рисунок В.9 – Статистика динамики веса питомца

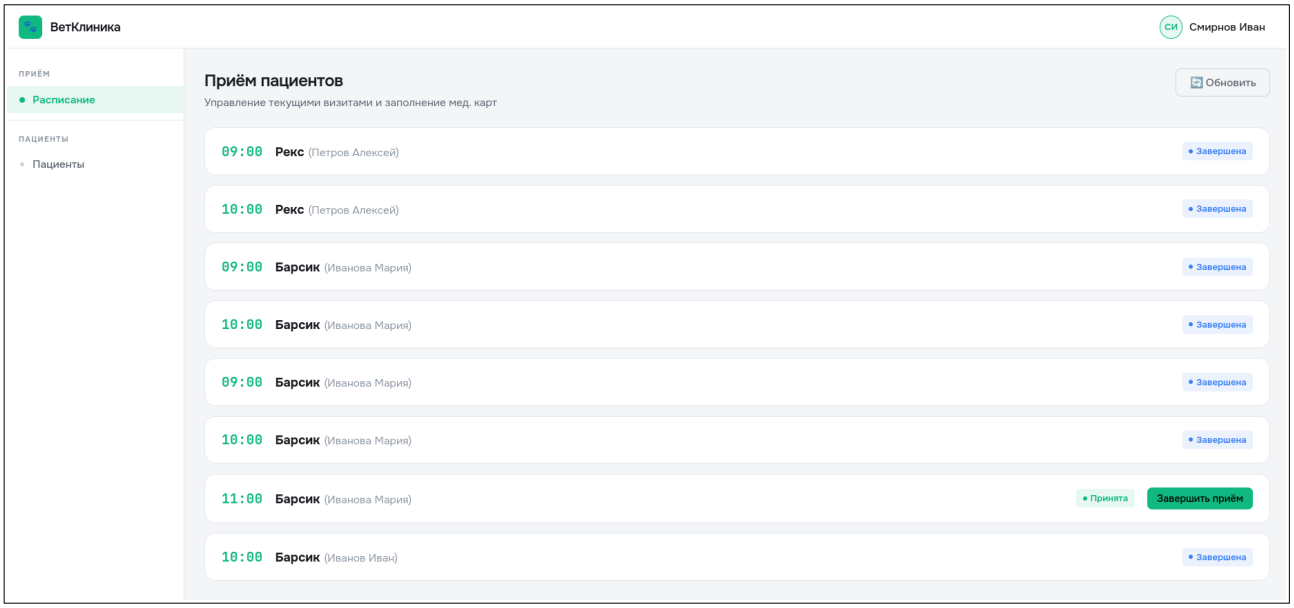


Рисунок В.10 – Расписание приёма врача

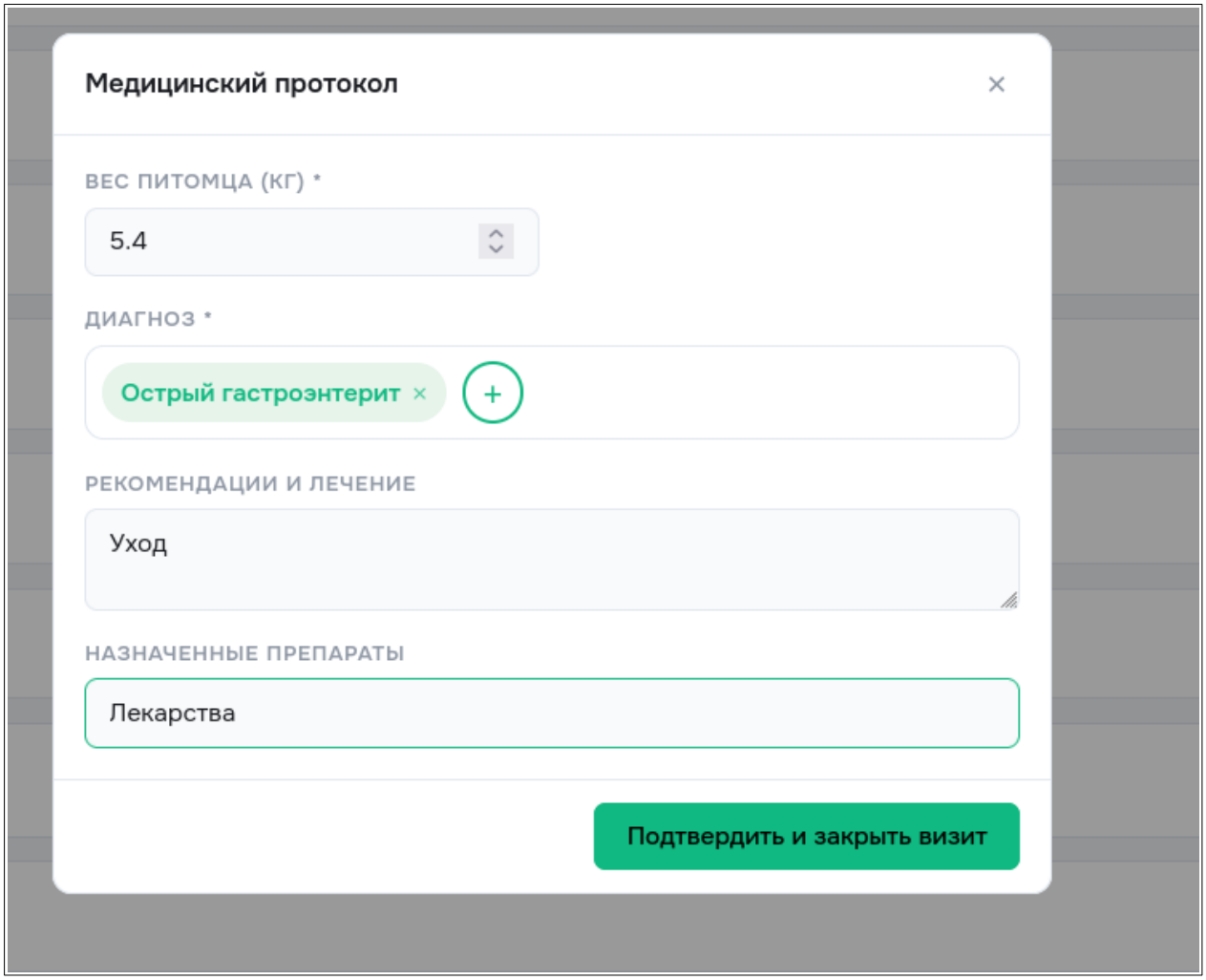


Рисунок В.11 – Завершение приёма с указанием диагнозов

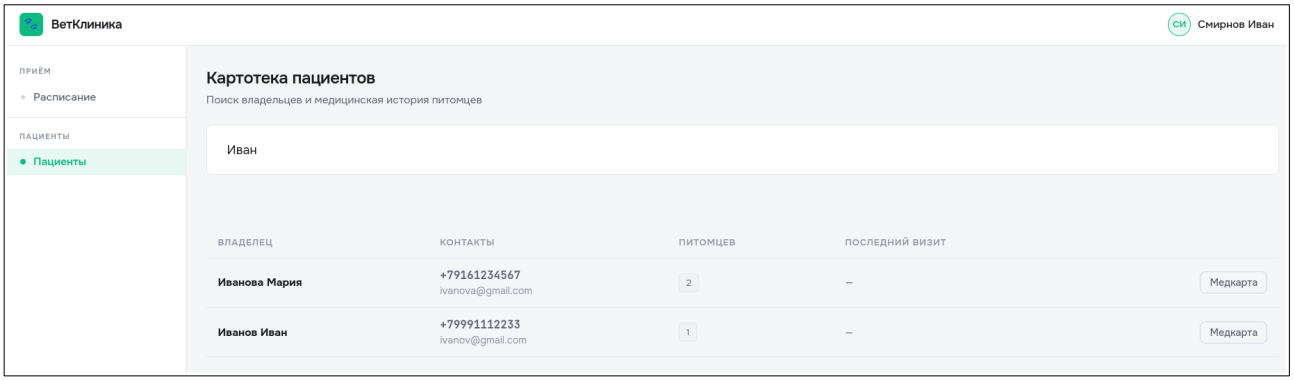


Рисунок В.12 – Картотека пациентов врача

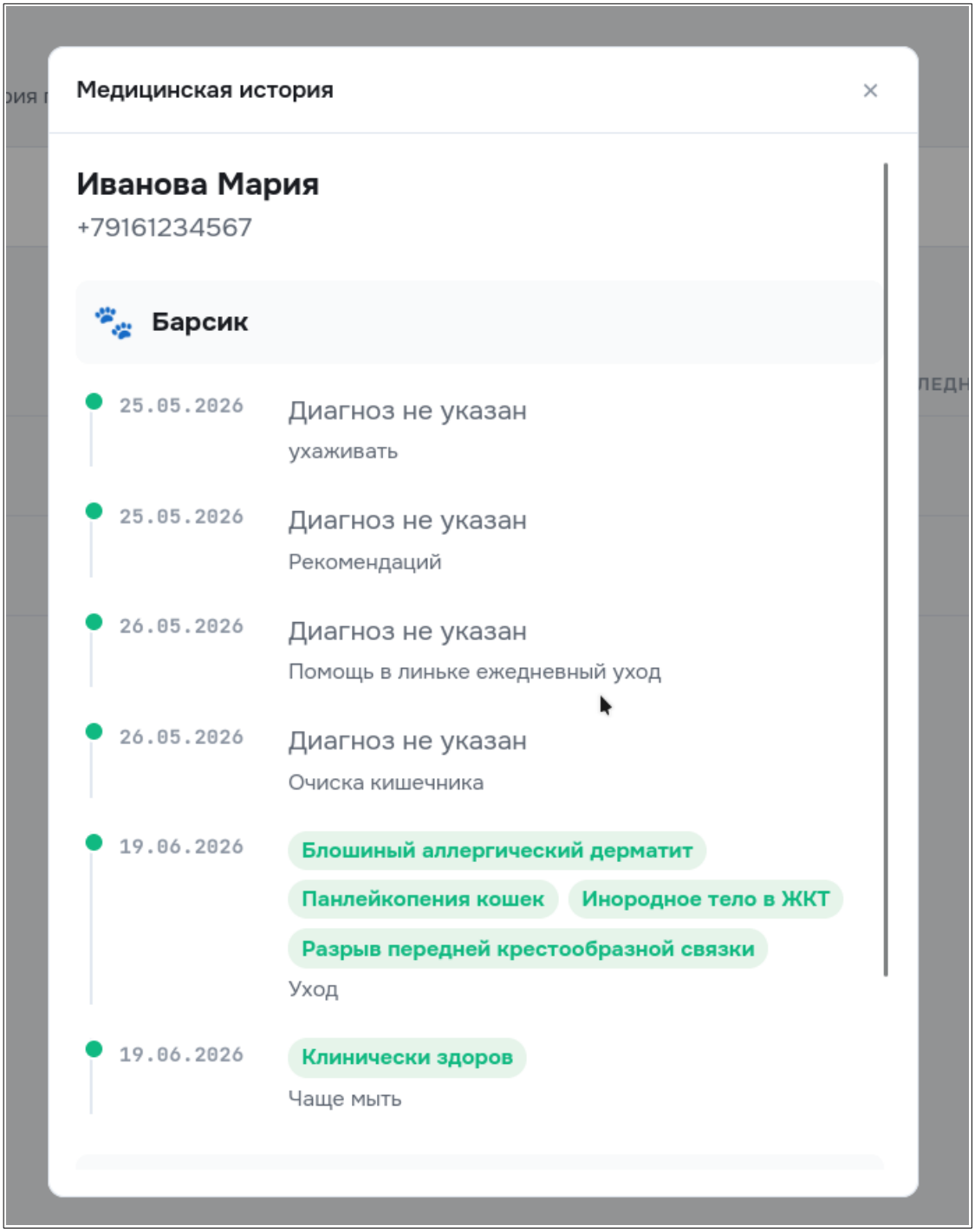


Рисунок В.13 – Медицинская история клиента (вид врача)

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Хорстманн, К. Принципы программирования на Go : учебник / К. Хорстманн. — Москва : ДМК Пресс, 2022. — 480 с.
2. Дронов, В. А. JavaScript и Vue.js : разработка веб-приложений : учебник для студ. учреждений сред. проф. образования / В. А. Дронов. — Москва : Издательский центр «Академия», 2023. — 416 с.
3. Кумскова, И. А. Базы данных : учебник / И. А. Кумскова. — Москва : КноРус, 2021. — 400 с. — ISBN 978-5-406-08303-1. — URL: https://book.ru/book/940108 (дата обращения: 21.06.2026). — Текст : электронный.
4. GORM Documentation [Электронный ресурс]. — URL: https://gorm.io/docs/ (дата обращения: 21.06.2026). — Режим доступа: свободный. — Текст : электронный.
5. Vue.js Documentation [Электронный ресурс]. — URL: https://vuejs.org/guide/ (дата обращения: 21.06.2026). — Режим доступа: свободный. — Текст : электронный.